

—— // ZTMUX // ——

ztmux

v3.7.11

THE COMPANION

the server · the client · the grid · the command language

2026-07-18

MenkeTechnologies

—— MENKETECHNOLOGIES ——

Contents

Before You Begin	3
Chapter 1 — The Pitch	4
Chapter 2 — How the Port Is Built	8
Chapter 3 — The Server	12
Chapter 4 — The Client	16
Chapter 5 — The Grid and Screen Model	20
Chapter 6 — The Input Parser	25
Chapter 7 — Layouts	29
Chapter 8 — The Command Language	33
Chapter 9 — Formats, Config, and Keys	37
Chapter 10 — The Terminal Back End	41
Chapter 11 — The Parity Suite: Correctness as Method	45
Chapter 12 — The Anti-Drift Gate	50
Chapter 13 — Beyond tmux: The Superset	54
Chapter 14 — The Road to “Done Right”: Shrinking <code>unsafe</code>	60
Chapter 15 — Status and the Honest Gaps	64
Chapter 16 — The Format Mini-Language in Depth	68
Chapter 17 — The Wire Protocol and <code>imsg</code> Framing	73
Chapter 18 — Copy Mode and the Grid Reader	76
Chapter 19 — Spawning Panes and Windows	78
Chapter 20 — The Status Line and Screen Redraw	80
Chapter 21 — Hooks, Alerts, and Notifications	82
Chapter 22 — Two Lifecycles: A Keystroke and a Pane	84
Chapter 23 — Performance and the Hot Paths	86
Chapter 24 — Colour: Names, Spaces, and the Nearest Match	88

Chapter 25 — UTF-8, Wide Characters, and Combining Sequences	90
Chapter 26 — Mouse Events and the Window Modes	92
Chapter 27 — Jobs, run-shell, and pipe-pane	94
Chapter 28 — Paste Buffers and the Environment	96
Chapter 29 — Design Decisions and Objections	99
Appendix A — The Command Reference	101
Appendix B — The Format Variable Reference	104
Appendix C — The Option Reference	107
Appendix D — The Parity Methodology and the Current Gap List	110
Appendix E — The Anti-Drift Gate and the Fake-Fn Categories	113
Appendix F — The Source Tree, Module by Module	115
Appendix G — Glossary	118
Appendix H — Building ztmux and the Vendored References	120
Appendix I — The Default Key Bindings	122
Appendix J — Format Recipes	124
Appendix K — The Audit Workflow	126
Appendix L — Coexisting with tmux and Operational Notes	128
Appendix M — A Reading Guide to the Source	130

Before You Begin

This is the book about ztmux — the one you read in a chair, front to back, to understand not only *what* ztmux does but *why* it is shaped the way it is. It is not a man page. The tmux man page enumerates every command, option, and format, and you reach for it when you need one fact fast. This book is the other thing: the continuous account of the program — from the socket a client opens to the cells a pane paints — and of the architecture ztmux inherits, faithfully, from tmux itself.

ztmux is a from-source Rust port of [tmux](#). It is not a wrapper around the [tmux](#) binary and not a control-mode client — it is the tmux program itself, reimplemented in Rust: the server, the client, the grid/screen model, the input parser, the layout engine, the command language, the format system, and the terminal back end. It is the multiplexer, rebuilt, and it is positioned to become a *superset* — a faithful tmux core that can grow features tmux does not have, without breaking the muscle memory of everyone who already lives in tmux. The repository states the ambition on its own front page: [\[TMUX, REWRITTEN IN RUST – DONE RIGHT\]](#), with the two-line creed that the rest of this book unpacks — “*Not a wrapper. Not control mode. The multiplexer itself,*” and “*Ported against the C, verified against the C — byte for byte.*”

This book is honest about that foundation, because the honesty is the point. ztmux stands on two references, both vendored as plain committed copies so the repo is self-contained: [vendor/tmux/](#), the upstream **C sources** that are the source of truth every ported module is validated against; and [vendor/tmux-rs/](#), Collin Richards' substantially-complete Rust port (ISC), which ztmux was *seeded from* and then took over as its own living code. The pristine [tmux-rs](#) copy is kept untouched precisely so the divergence is always legible:

```
# exactly what has changed since the fork - the port's true diff
git diff --no-index vendor/tmux-rs/src src
```

The original work — the work that makes ztmux a MenkeTechnologies project rather than a re-vendor — is the ongoing port itself: taking the seed code, validating every module against the C source of truth, and driving it toward a complete, own-able, extensible multiplexer. This is still-evolving software: the crate version is [v3.7.11](#) at the time of writing — a number that tracks the tmux release ztmux targets for compatibility ([tmux next-3.7](#)), not a claim about its own maturity — and where a subsystem is partial or a feature is still aspirational this book marks it rather than rounding up. ztmux is the work of MenkeTechnologies, built on the foundation of the tmux authors and the tmux-rs port before it, and this book is grounded in that code.

Let's start.

Chapter 1 – The Pitch

A terminal multiplexer is, on paper, a solved problem: `tmux` has multiplexed sessions for two decades, and it is excellent. This chapter states what `ztmux` is for — not “`tmux` but faster,” but *tmux owned in Rust*: a memory-safe reimplementation of the whole program that can be extended past the C original while staying bit-for-bit familiar to its users.

What `ztmux` is — the whole program, not a wrapper or control-mode client

There is a lazy way to build “a `tmux` in language X,” and `ztmux` is not it. The lazy way is to shell out to the real `tmux` binary and drive it — parse its output, send it commands, wrap its control mode (`tmux -CC`) — and call the wrapper a new multiplexer. That produces a client, not a multiplexer; it inherits every bug and every limit of the program it wraps, and it can never be a superset of `tmux` because it *is* `tmux` with a coat of paint. `ztmux` takes the hard road instead. The `ztmux` binary is a complete `tmux`: it contains the server that owns all session state, the client that attaches to it, the grid and screen model that every pane is drawn on, the VT input parser that turns a program’s bytes into cells, the layout engine that tiles panes, the command language that `bind-key` and `split-window` are written in, the `#{...}` format system, and the terminfo-driven terminal back end. Nothing is delegated to an external `tmux`; there is no external `tmux` in the picture at all.

The evidence for that claim is structural, not rhetorical. The source tree has a `src/server.rs` that binds a Unix socket and runs a `libevent` loop; a `src/client.rs` that connects to that socket and performs the attach handshake; a `src/grid.rs` and `src/screen.rs` that maintain the cell matrices; a `src/input.rs` that implements Paul Williams’ DEC ANSI parser state machine; a `src/layout.rs` with the pane-tiling tree; sixty-odd `src/cmd_*.rs` files, one per `tmux` command; a `src/format.rs` that is two hundred kilobytes of format-expansion logic; and a `src/tty.rs` that drives the outside terminal through terminfo. Each of those is a transliteration of the corresponding `tmux` C module, function by function, name for name. A wrapper would have none of them; it would have a subprocess handle and a protocol parser. `ztmux` has the whole machine.

That distinction has a practical consequence you can see from the first command. Because `ztmux` is a server in its own right, it opens its *own* socket namespace — `ztmux-<uid>` rather than `tmux-<uid>` — so it does not collide with a running `tmux` on the same machine (`src/tmux.rs`, `make_label`). You can run `tmux` and `ztmux` side by side, each with its own server, its own sessions, its own detach-survival, and neither knows the other exists. That is only possible because `ztmux` is not a client of `tmux`; it is a peer implementation of the same program.

There is a sibling project, `ztmux-core`, that *is* a native `tmux` client engine — a library that speaks the `tmux` wire protocol to talk to a running `tmux` server — and it is deliberately a different thing from this repository. Keeping the two separate is part of the honesty: `ztmux-core` is the client-of-`tmux` story, and `ztmux` (this book’s subject) is the whole-server-and-client story. When the README says “it is `tmux`, reimplemented,” it means the second thing, and the rest of this book is the detailed account of that reimplementation.

Coexistence in practice: `ztmux` next to `tmux`

The claim that `ztmux` is a peer implementation rather than a client of `tmux` has a concrete, testable consequence you can see in one session. Because `ztmux` binds its own socket namespace (`ztmux-<uid>`, Chapter 3 and Appendix L) rather than `tmux`’s (`tmux-<uid>`), the two servers never touch. Start a `tmux` session and a `ztmux`

session from the same shell and they are entirely independent: `tmux ls` lists the tmux server's sessions, `ztmux ls` lists ztmux's, each has its own detach-survival, its own options tree, its own key bindings, its own set of panes and running programs. Detach from one and the other is unaffected; kill one server and the other keeps running. There is no shared state because there is no shared server — ztmux is not talking to tmux at all. A wrapper could not do this: a wrapper *is* tmux with a coat of paint, so it would share tmux's socket, tmux's sessions, and tmux's bugs. The clean coexistence is the structural proof that ztmux contains its own server, and it is the first thing a skeptical user can check in ten seconds.

Port, not rewrite-from-spec: standing on the C source of truth

There are two ways to reimplement a program, and they produce very different artifacts. One is to read the documentation — the man pages, the protocol specs, the escape-sequence tables — and write a fresh implementation of what they describe. The other is to port: to take the original source, function by function, and translate it into the new language, keeping the same names, the same control flow, the same data structures, and the same defaults, then verify the translation against the original. ztmux is emphatically the second kind. It does not re-derive tmux's behavior from a specification of tmux; it translates tmux's C into Rust and checks the translation against the C.

This is not a stylistic preference; it is the only way to be *faithful* in the strong sense the project demands. tmux's real behavior is not fully captured by any document — it lives in the exact order of operations in `server_loop`, in the exact rounding of the even-horizontal layout, in the exact bytes `format_unescape` writes for a `{1:...}` literal. A spec-driven rewrite would get the documented behavior right and quietly diverge on all the undocumented behavior that real scripts and real terminals depend on. A port cannot diverge silently, because every function carries its provenance: a Rust function in ztmux keeps its C name, and directly above it sits a doc comment citing the exact C file, line, and full signature it was ported from. Open `src/grid.rs` and you find, verbatim:

```
/// C `vendor/tmux/grid.c:374`: `struct grid *grid_create(u_int sx, u_int sy, u_int
    hlimit)`
pub unsafe fn grid_create(sx: u32, sy: u32, hlimit: u32) -> *mut grid {
```

There are more than two thousand such citations across the tree — a line-level map back into upstream tmux, one per ported function. They are not decoration. They are what makes the port auditable: a reviewer can jump from any Rust function to the exact C lines it claims to implement and diff the two by eye, and an automated gate (Chapter 12) fails the build if a Rust function's name has no C counterpart at all. “Standing on the C source of truth” is therefore a literal description of the method: the truth is `vendor/tmux/`, checked into the repository, and every module is diffed against it rather than trusted.

The seed matters too. ztmux did not start from a blank `src/`; it started as a copy of `vendor/tmux-rs/src`, Collin Richards' Rust port, which had already done the enormous first-pass transliteration of tmux's C. Keeping that seed vendored and pristine means the port's *own* contribution is always a single diff away — the `git diff --no-index vendor/tmux-rs/src src` from the introduction shows exactly which functions ztmux has re-reported, fixed, or advanced since the fork. This is the same discipline the whole MenkeTechnologies stack uses when it builds on a foundation: name the foundation, keep it legible, and make your divergence inspectable rather than asserted.

The superset ambition: a faithful core that can grow

The word *superset* is a promise with two halves, and both halves have to hold or the word is dishonest. The first half is that ztmux must be a faithful tmux — every command, every format, every default behaving byte-for-byte as tmux does — so that someone who has lived in tmux for a decade feels nothing change when they switch. The second half is that, being memory-safe Rust that MenkeTechnologies owns rather than frozen C it merely

wraps, ztmux can grow features the C tmux does not have, without forking away from the familiar core. A superset is a strict extension: it does everything the original does, exactly, *and* more.

The reason the first half is non-negotiable is that a superset that breaks parity is not a superset — it is a fork with extra features, and forks lose. The value of being tmux-compatible is muscle memory: the config files, the keybindings, the scripts, the status-line format strings that people have accumulated over years all have to keep working unchanged. The moment `#{pane_current_command}` reports the wrong thing or an even-horizontal split rounds the wrong way, the compatibility promise is broken and the user is on their own. This is why the project spends so much of its energy on a parity suite (Chapter 11) that diffs ztmux against the real tmux byte-for-byte: the superset can only add things once the base is provably identical.

The reason the second half is possible at all is ownership. tmux is superb and it is also a mature C program with a stable feature surface; adding a deep new capability to it means convincing its maintainers and writing it in C against decades of existing code. ztmux is young, it is Rust, and it is the property of the same author who builds the rest of the stack. That means the natural extensions are not speculative — they are the integrations the surrounding MenkeTechnologies tools already want: tighter coupling with the `zterminal` emulator that already speaks a tmux wire protocol, with `zshrs` running inside the panes, with the shared GUI component library that the stack's other apps are built from. Chapter 13 is the honest ledger of which of those are shipped and which are roadmap.

For now the discipline is: parity first, superset second, and never confuse the two. Everything in Chapters 3 through 10 is the faithful core, documented as the port of a specific C module. Chapter 13 is the superset ambition, marked clearly as ambition where it is not yet code. Keeping that line sharp is what lets the book claim faithfulness without overclaiming novelty, and it is the same line every book in this series draws between the foundation it inherits and the work it adds.

The creed, line by line

The repository front page carries a two-line creed, and because the rest of this book is an expansion of it, it is worth reading each clause deliberately. The first line — *“Not a wrapper. Not control mode. The multiplexer itself.”* — is three negations narrowing to a claim. “Not a wrapper” rejects the shell-out approach: ztmux does not drive an external `tmux` binary, parse its output, or send it commands; there is no external tmux in the picture (Chapter 1’s structural argument). “Not control mode” rejects the subtler version of the same shortcut: it is not a `tmux -CC` client either, not a program that speaks tmux’s control protocol to a real tmux server — that is `ztmux-core`’s job, deliberately kept a separate thing. “The multiplexer itself” is the positive claim the negations clear room for: ztmux contains the server, the client, the grid, the parser, the layout engine, the command language, the format system, and the terminal back end — the whole machine, not a view of someone else’s.

The second line — *“Ported against the C, verified against the C — byte for byte.”* — names the method twice, because the method has two halves. “Ported against the C” is the construction: every module is a transliteration of a specific tmux C module, function for function, name for name, with a citation to its origin (Chapter 2). “Verified against the C” is the proof: the parity suite runs the same inputs through the vendored C tmux and ztmux and compares (Chapter 11). “Byte for byte” is the *bar* of that proof — not “behaves reasonably,” not “matches the man page,” but produces the identical bytes, which is the only bar strict enough to guarantee the muscle-memory compatibility the superset rests on (Chapter 24). Read together, the two lines are the whole thesis: ztmux is the multiplexer itself (not a wrapper or a client), built by porting the C and proved by diffing against it to the byte. Everything in the chapters that follow is that thesis worked out in detail.

What v3.7.11 means — the version tracks the tmux target, not ztmux's own maturity

ztmux's crate version is `v3.7.11`, and that number means something specific you should hold onto: it tracks the *tmux release ztmux targets* for compatibility — the parity suite measures against `tmux next-3.7` — not ztmux's own maturity on some independent scale. Read carelessly it would imply a decade-seasoned program; read correctly it says only that this ports the 3.7 line of tmux. Where the port actually stands is not in the version string at all — it is in the parity ledger and the staged roadmap below. The port is no sketch, and the parity suite is why: it first reached 122 of 122 cases byte-for-byte identical to the vendored tmux on both Linux and macOS, and then, having exhausted that first set, was *expanded* to 1107 cases spanning far more of the surface. That larger, adversarial target initially surfaced about twenty specific port gaps — which the port then closed, one by one, until the suite stood at **1107 of 1107 passing (100%), byte-for-byte identical to the vendored tmux on both Linux and macOS** (Chapter 11 and Appendix D walk the whole method). The README calls the result a daily driver. But none of that makes the surface uniform: some subsystems are complete and battle-tested, others are transliterated but lightly exercised, and a number of the configuration options that appear in the options table exist only so that parsing a real `.tmux.conf` does not error — they are accepted and stored but not yet wired to behavior. The Cargo manifest says as much in a comment: most of the feature flags exist “to make matching upstream’s configuration options easier,” with the notable exception that sixel graphics are actually implemented.

Reading against early software means trusting the parity claim exactly as far as it is measured and no further. When this book says a behavior is faithful, it means there is a parity case, or a direct line-for-line port with a C citation, behind the claim. When a subsystem is present but not yet proven, the book says “ported” rather than “verified,” and Chapter 15 keeps the running ledger of which is which. The version-string discipline extends into the artifacts themselves: the parity summary JSON records the exact reference it measured against (`tmux next-3.7`) and the count it achieved, so the claim is never a vibe — it is a number with a provenance.

The upside of early software is that the shape is still legible. A mature program accretes special cases until its architecture is only visible to people who have lived in it for years; a young port, especially one that keeps its C citations inline, wears its structure on the surface. That is the opportunity this book takes: because ztmux is still close to the C it came from and still small enough to hold in your head, the chapters that follow can walk the *actual mechanism* — the real function that binds the socket, the real state that survives a detach, the real loop that turns bytes into cells — rather than an idealized picture of it. The early-software caveat is real; it is also what makes the machine worth reading front to back.

Chapter 2 — How the Port Is Built

Before any subsystem, the method. ztmux stands on two vendored references and a discipline: every ported module is checked against its C counterpart, and the seed port is kept pristine so the fork's divergence is always a single diff away. This chapter documents that machinery — it is what makes “faithful” a claim you can verify rather than trust.

The two references: `vendor/tmux/` (C truth) and `vendor/tmux-rs/` (the seed)

Under `vendor/` sit two complete source trees, both committed as plain read-only copies rather than git submodules, both SHA-pinned, and both documented in `vendor/VENDOR.md`. The first, `vendor/tmux/`, is the upstream tmux C source — the *source of truth*. It is the code every ported Rust module is diffed against, the code the parity suite builds and runs as its reference, and the code the anti-drift gate indexes to decide whether a Rust function name is real. The vendored tree is the `next-3.7` development line of tmux: its own `configure.ac` declares `AC_INIT([tmux], next-3.7)` and its `CHANGES` file opens with “CHANGES FROM 3.7 to 3.7a,” and the parity summary names the reference `tmux next-3.7`. Pinning to a specific upstream commit rather than “whatever tmux the system has” is essential, because tmux’s behavior drifts between versions and a parity suite measured against the wrong version would report false failures — a point Chapter 11 returns to.

The second tree, `vendor/tmux-rs/`, is the *seed*: Collin Richards’ tmux-rs, the Rust port that ztmux’s `src/` was originally copied from. It is vendored for a single, specific reason — to keep the fork’s divergence legible. Because the seed is present and untouched, the exact set of changes ztmux has made since forking is a mechanical diff away, and upstream tmux-rs fixes remain cherry-pickable. Both trees carry their upstream ISC license and their pin metadata; `VENDOR.md` records the commit SHAs and the date they were vendored, and documents the refresh procedure (export a fresh `git archive` with no `.git`, rsync it over the vendored path, record the new `git rev-parse HEAD`).

The two references play complementary roles, and it is worth being precise about which is which. `vendor/tmux-rs/` is a *head start* — it answers “what did the previous Rust transliteration look like?” — but it is not the truth; it can carry its own bugs, and several of the bugs ztmux has fixed (Chapter 11) were inherited from the seed and corrected against the C. `vendor/tmux/` is the *truth* — it answers “what does tmux actually do?” — and it is the only reference the parity suite and the anti-drift gate consult. When those two disagree, the C wins, every time, and the seed’s version is treated as a lead to investigate rather than an answer to trust.

Keeping both in-repo, rather than as submodules or as download steps, is a deliberate self-containment choice. A clone of ztmux carries everything needed to build the port, build the reference, run the parity suite, and audit every function against its C origin, with no network access and no version ambiguity. The cost is repository size; the benefit is that “faithful to tmux” means faithful to a *specific, committed* tmux that ships with the code, not to whatever tmux a contributor happens to have installed.

The divergence diff: `git diff --no-index vendor/tmux-rs/src src`

The single most useful command in the whole methodology is the one from the introduction: a plain `git diff` between the pristine seed and the living source. Because `vendor/tmux-rs/src` is never edited, that diff *is* the port's original contribution — every function ztmux has re-ported, every bug it has fixed against the C, every place it has advanced past where the seed left off. It turns the abstract question “what has this project actually done?” into a concrete artifact you can read. There is no need to trust a changelog or a claim of originality; the diff is the claim, and it is generated from the files themselves.

This matters because the honesty problem for any port is *inflation* — the temptation to present a lightly-modified re-vendor as substantial new work. The divergence diff makes inflation impossible to hide. If the diff is small, the port has done little; if the diff is large and every hunk is a real re-port with a fresh C citation, the port has done a lot. The command is cheap, deterministic, and adversarial in the right direction: it can only ever *under-state* the port's work (by counting untouched seed files as unchanged), never overstate it.

The diff also has a maintenance role. When upstream tmux-rs fixes a bug, the seed tree can be refreshed and the divergence diff recomputed, showing immediately whether ztmux's own changes conflict with the upstream fix or compose cleanly with it. The pristine seed is, in effect, the merge base for a three-way relationship between upstream tmux-rs, upstream tmux (the C truth), and ztmux's living port. Any one of the three can move, and the diffs between them stay meaningful.

Read together with the C citations, the divergence diff gives two independent views of the same work. The citations answer “where did each function come from in the C?” — provenance, per function. The divergence diff answers “what has this project changed since the seed?” — contribution, across the tree. A reviewer who wants to understand ztmux's real work runs the diff; a reviewer who wants to check a single function's fidelity reads its citation and jumps to the cited C lines. Neither view requires taking anything on faith.

Validating a module against its C counterpart

Porting a single module is a repeatable procedure, and ztmux follows it the same way every time. Start from the seed's version of the module (or, where the seed is thin, from the C directly). Translate function by function, keeping the C name unchanged — `grid_create` stays `grid_create`, `input_csi_dispatch` stays `input_csi_dispatch` — because the shared name is what lets everything downstream (citations, the anti-drift gate, the reviewer's eye) line the Rust up against the C. Preserve the control flow, not just the behavior: where the C uses a `for` loop that increments on `continue`, the Rust loop must advance on every path too, and where it does not, subtle bugs follow (Chapter 11's `format_unescape` crash is exactly this mistake). Then annotate each ported function with its C file, line, and full signature, and validate the result against the reference.

Validation happens at three levels, coarse to fine. The coarsest is compilation against the anti-drift gate: does every free function in the module correspond to a real function in the C source, or is it an invented helper inflating the apparent completeness? The middle level is the C-signature citation, generated by a script that records not just the C function's name but its *full parameter list with names*, so a reviewer can diff the Rust parameters against the C parameters and catch a renamed, reordered, dropped, or invented argument. The finest level is the parity suite: feed the same input to the vendored tmux and to ztmux and compare the output byte-for-byte, so that any behavioral divergence — not just a structural one — shows up as a failing case.

The three levels catch different classes of error, and that is the point of having all three. The anti-drift gate catches *structural* dishonesty: functions that exist in Rust but not in tmux, which would otherwise let a port look more complete than it is. The signature citations catch *interface* drift: a function with the right name but the wrong arguments. The parity suite catches *behavioral* drift: a function with the right name and the right signature that nonetheless produces different bytes. A port that passes all three is faithful in the structural, the interface, and the behavioral sense simultaneously, and each gate is cheap enough to run in CI on every push.

What this procedure explicitly refuses is the shortcut of “close enough.” A port that renames functions for Rust

idiom, or invents helpers to make a module read more naturally, or approximates a rounding rule because the difference “won’t matter,” breaks one of the three gates and fails the build. The friction is deliberate: it is much easier to keep a port faithful by refusing every small divergence than to let divergences accumulate and try to reconcile them later. The idiomatic-Rust cleanup — the part where the port *stops* looking like transliterated C — is a later, separate phase, and Chapter 14 explains why it deliberately comes after parity rather than before.

The port report and the doc-count / man-page gates

Beyond the per-function gates, ztmux tracks its progress in aggregate through a generated port report. A script (`scripts/gen_port_report.py`) walks the Rust source and the vendored C, matches functions across the two, and produces a report — published as `port_report.html` on the project’s documentation site — that says, in numbers, how much of tmux has been ported and how much remains. This is the “how complete is it?” question answered mechanically rather than by assertion, and it is the natural companion to the divergence diff: the diff shows what changed since the seed, the report shows how much of the C surface is covered at all.

The report is generated from the same machinery that produces the C-signature citations, so the two stay consistent. The annotation script, `scripts/annotate_c_links.py`, walks the C sources to build a `name → (file, line, signature)` index; the port report reuses that index to compute coverage. Because both are derived from the vendored C rather than from a hand-maintained list, neither can drift out of sync with reality — refresh the vendored tmux, re-run the scripts, and both the citations and the report update to match.

Alongside the coverage report sit the *documentation* gates that the wider stack imposes on every project: a doc-count check and a man-page check that keep the project’s own documentation from falling behind its code. These are the same gates the sibling projects use, and they exist because documentation that is not gated rots. A doc-count gate ensures the ratio of documented to undocumented public surface does not regress; a man-page gate ensures the reference material stays present and buildable. They do not measure correctness — that is the parity suite’s job — but they measure whether the project is still legible, which for a port that sells itself on legibility is its own kind of correctness.

Taken together, the machinery of Chapter 2 forms a closed loop. The seed makes the starting point legible; the divergence diff makes the contribution legible; the C citations make each function’s provenance legible; the anti-drift gate makes structural honesty enforceable; the parity suite makes behavioral fidelity measurable; and the port report and doc gates make the aggregate progress and the documentation legible. None of it depends on trusting the author. That is the whole design: a port whose faithfulness is a property you can check, not a claim you have to believe. With the method established, the rest of the book can walk the machine it produces — starting where tmux itself starts, with the server.

A guided tour of one ported function

To make “port, not rewrite” concrete, take a single function from `src/grid.rs` and read it against its origin. The Rust is:

```
/// C `vendor/tmux/grid.c:374`: `struct grid *grid_create(u_int sx, u_int sy, u_int
    hlimit)`
pub unsafe fn grid_create(sx: u32, sy: u32, hlimit: u32) -> *mut grid {
```

Four things about this one line encode the whole method. The **name** is `grid_create`, identical to the C — not `create_grid`, not `Grid::new`, because the shared name is what lets the anti-drift gate match it to the C index, what lets the citation script find its origin, and what lets a reviewer line the two up. The **signature** takes `sx`, `sy`, `hlimit` in that order with those names, mirroring the C’s `u_int sx`, `u_int sy`, `u_int hlimit`, so the citation’s full-signature form lets a reviewer catch a reordered or renamed argument. The **citation** above it names the exact file and line (`grid.c:374`) and the full C signature, so the jump from Rust to C is mechanical. And the function is `pub unsafe fn` returning `*mut grid` — a raw pointer to a `#[repr(C)]` struct — because the port reproduces tmux’s manual-memory model exactly (Chapter 14), which is what makes the behavior checkable against the C in the first place.

Read the body and the fidelity continues: it allocates the `grid` with the port's `xmalloc`, sets `sx/sy`, sets the `GRID_HISTORY` flag only when `hlimit` is non-zero (the “scrollback of zero keeps no history” rule of Chapter 5), allocates `sy` lines, and returns the pointer — the same order of operations as the C, so any divergence would be a divergence you could see by diffing the two side by side. This is what every one of the two-thousand-plus ported functions looks like: same name, same signature, same control flow, a citation to its origin, and a raw-pointer C-shaped body that the parity suite pins and a later idiomatic pass (Chapter 14) will eventually soften. One function shows the whole discipline in miniature.

Chapter 3 – The Server

tmux is a client/server program: a long-lived server owns all state, and clients attach to it. This chapter walks the ztmux server — the event loop over `libevent`, the socket it binds, the session/window/pane tree it owns, and how it survives every client detaching.

The client/server split and why it exists

The defining architectural decision of tmux — the one that makes it a multiplexer rather than a terminal — is that the state lives in a server, not in the client you type into. When you run `ztmux new-session`, the process you launched does not *become* your session; it connects to a server (starting one if none exists) and attaches to a session the server owns. Your terminal window is a thin client; the sessions, windows, panes, their scrollback, their running programs, and their layouts all live in the server process, which has no controlling terminal of its own and outlives every client that comes and goes. ztmux inherits this split exactly, because the split *is* the feature: it is what lets you detach from a session, close your laptop, and re-attach from another machine to find everything still running.

The split is visible in the source as two entry points that share a binary. `src/main.rs` sets up a custom global allocator (a thin shim over libc `malloc/free/calloc/realloc`, so allocation behaves exactly as the C's does) and calls `tmux_main`, which decides — based on the command and the environment — whether this invocation should act as a client connecting to a server, or should *be* the server. `src/server.rs` is the server half; `src/client.rs` is the client half (Chapter 4). They communicate over a Unix domain socket using an imsg-based protocol with a version number (`PROTOCOL_VERSION = 8`, in `src/tmux_protocol_h.rs`) that both sides check, so a client and a server built from mismatched code refuse to talk rather than corrupt each other's state.

Why a separate server at all, rather than a library the client links? Because the server must survive the client. A library in the client's process dies when the client's terminal closes; a separate process with no controlling terminal does not. The server is deliberately daemonized — forked away from the launching terminal, its signals cleared and re-established — so that the death of a client (a closed window, a dropped SSH connection, a `kill` on the client process) is just a detach, not a loss. Everything the server owns keeps running in the forked daemon until the server itself is told to exit.

This is also why the server is the natural home for *all* shared behavior: key bindings, options, paste buffers, hooks, and the marked pane are single-copy state that any attached client can read and modify. When two clients attach to the same session, they are two windows onto one server-owned world; a `set-option` from one is immediately visible to the other because there is only one options tree, and it lives in the server. Chapter 4's multi-client story and Chapter 9's options story both rest on this: the client is a view, the server is the truth.

The libevent event loop and the server socket

The server is, at bottom, one event loop. tmux is built on `libevent`, the C event-notification library, and ztmux uses the *same* library through FFI rather than reimplementing an async runtime — the whole point of a faithful port is that the event dispatch behaves identically, so ztmux links libevent (`event_core`, wired up in `build.rs`; statically on macOS, dynamically elsewhere by default) and calls its C API directly. Events are C `struct event` values, several of them module-level `static mut` in `server.rs` (`SERVER_EV_ACCEPT` for incoming connections, `SERVER_EV_TIDY` for the hourly housekeeping timer), and callbacks are `unsafe extern "C-unwind" fn` pointers handed to `event_set/event_add` exactly as C would register them.

The loop is entered through `proc_loop(SERVER_PROC, Some(server_loop))`. `proc_loop` runs libevent's dispatch, and `server_loop` (in `server.rs`) is the body called on each iteration: it stamps `CURRENT_TIME`, drains the command queue for the global queue and for every identified client until no work remains (Chapter 8's command queue is pumped here), runs `server_client_loop` to service clients, and then decides whether the server should exit — checking the `exit-empty` and `exit-unattached` options, the count of attached clients, whether any command is still waiting, and whether any background job is still running. Returning 1 from `server_loop` tells `proc_loop` to stop, at which point the server kills its jobs, saves the prompt history, and exits.

The socket the loop accepts on is created by `server_create_socket`. It fills an `AF_UNIX sockaddr_un`, copies the socket path into `sun_path` (guarding against `ENAMETOOLONG` for over-long paths), unlinks any stale socket at that path, creates the socket, sets a umask appropriate to whether this is the default socket, binds, and then calls `listen(fd, 128)` — a backlog of 128 pending connections — before switching the descriptor to non-blocking. The path itself comes from `make_label` in `src/tmux.rs`, which builds "`{base}/ztmux-{uid}`", creates that directory with `S_IRWXU` and the same owner-and-permission safety checks tmux performs (the directory must be owned by the caller and carry no world-accessible bits), and places the per-label socket inside it. With the default label "`default`", the socket is `<dir>/ztmux-<uid>/default` — the `ztmux-` prefix being the one place the path deliberately differs from tmux, so the two servers never share a socket.

Incoming connections are handled by `server_accept`, the callback armed on `SERVER_EV_ACCEPT`. It re-arms itself, accepts the new descriptor, and — importantly — backs off gracefully under file-descriptor pressure: on `ENFILE` or `EMFILE` it disables accept for one second (via `server_add_accept` with a timeout) rather than spinning, and on the transient `EAGAIN/EINTR/ECONNABORTED` it simply returns. A successful accept creates a client for the new fd (`server_client_create`) and runs the access-control join (`server_acl_join`). This is the faithful tmux behavior: a server under fd exhaustion degrades instead of failing, and the loop keeps running.

The state tree: sessions, windows, panes

Everything the server owns hangs off a small number of global containers, declared as module-level statics in `server.rs` and initialized when the server starts. The connected clients live in a `CLIENTS` tail queue; sessions in a `SESSIONS` red-black tree; windows in a `WINDOWS` tree and, separately, all window panes in an `ALL_WINDOW_PANES` tree so any pane can be found by id. These are the same intrusive BSD data structures tmux uses — `TAILQ` for ordered lists, `RB` (red-black) trees for keyed lookup — ported as Rust functions (`tailq_foreach`, `rb_insert`, and the rest) that operate on the same `#[repr(C)]` node structs. The choice to keep the C data structures rather than reach for Rust's `Vec` and `BTreeMap` is faithfulness again: the iteration order, the insertion semantics, and the pointer identity all have to match tmux, and the surest way to match them is to run the same algorithms.

The conceptual hierarchy is the familiar tmux one. A *session* is a collection of windows with one current window; a *window* is a collection of panes tiled by a layout with one active pane; a *pane* is a single pseudo-terminal running a program, backed by a screen, which is backed by a grid of cells. A client attaches to a session and sees its current window; switching windows, splitting panes, and resizing all mutate this tree, and because the tree lives in the server, the mutations persist across detach. Chapter 5 descends into the grid at the bottom of this hierarchy; Chapter 7 covers the layout that tiles the panes; here the point is that the server *owns* the whole tree and every client is a view onto it.

Two pieces of cross-cutting state deserve names because they recur. The *marked pane* — tmux's `select-pane -m` target, used by commands like `join-pane` and `swap-pane` as an implicit source — is a single server-global `cmd_find_state` (`MARKED_PANE`), with a small API (`server_set_marked`, `server_clear_marked`, `server_is_marked`, `server_check_marked`) that ztmux ports directly. The *message log* — the server's recent status messages, surfaced by `show-messages` — is a `MESSAGE_LOG` list trimmed to the `message-limit` option, appended to through a `server_add_message!` macro. Both are examples of the general rule: shared, single-copy state that only makes sense in one place, and that place is the server.

The pane at the leaf of the tree is where the server meets the outside world. Each pane owns a pseudo-terminal

master fd, and the server watches those fds through libevent: when a program running in a pane writes bytes, an event fires, the bytes are read and fed to the input parser (Chapter 6), which mutates the pane's screen, which marks the affected cells dirty, which eventually produces output to every client viewing that pane. The child processes themselves are reaped by the server's `SIGCHLD` handling (`server_child_signal` → `server_child_exited`/`server_child_stopped`), which marks a pane exited or continues a stopped one. The server is thus simultaneously the owner of the abstract session tree and the concrete manager of every child process and every fd — a single loop multiplexing all of it.

Finding anything by walking the trees

The global containers are not just storage; they are the answer to every “which object does this name mean?” question the server fields, and a couple of concrete lookups show how. When a command resolves a target like `work:2.1` (Chapter 8), `cmd_find` walks `SESSIONS` — the red-black tree keyed so a session can be found by name — to locate `work`, then walks that session's window list to index 2, then that window's pane list to pane 1. When a `SIGCHLD` arrives and the server must find *which pane* a dead child belonged to, it has the pid and walks `ALL_WINDOW_PANES` — the separate tree of every pane across every window — to find the pane whose process matches, then marks it dead. When a format expands `{window_panes}`, it walks the current window's pane list and counts. Every one of these is a walk over the same intrusive BSD structures tmux uses (`rb_find`, `tailq_foreach`), ported as functions over the same `#[repr(C)]` nodes, because the *order* these walks visit objects in — the iteration order of a red-black tree, the insertion semantics of a tail queue — is observable through formats like `list-windows -F` and must match tmux exactly. Reaching for Rust's `BTreeMap` would have been more idiomatic and would have iterated in a subtly different order; the port keeps the algorithms so the observable order is tmux's.

Server lifecycle: start, detach-survival, kill

The server comes to life in `server_start`, and the sequence is careful because it is forking a daemon. First it blocks all signals (`sigfillset` + `sigprocmask(SIG_BLOCK)`) so the fork happens in a known signal state, then calls `proc_fork_and_daemon` — the parent (the client that triggered the autostart) returns early with a descriptor, while the child continues as the server. The child installs a Rust panic hook that, on any panic, dumps a backtrace to a `server-panic-<pid>.txt` file — a small original touch that turns a Rust panic in the daemon into a diagnosable artifact rather than a silent death. It clears the inherited signal handlers, re-initializes libevent in the new process (`event_reinit`, fatal if it fails, because a forked libevent base must be reinitialized), starts its own `tmuxproc`, and re-establishes signal handling with `server_signal` as the handler.

With the process set up, the server initializes the world: the RB and TAILQ trees for windows, panes, clients, sessions, and the message log; the key bindings (`key_bindings_init`); the input key tables (`input_key_build`). It creates the listening socket (optionally under systemd socket activation), creates the first client for the connecting descriptor, arms the hourly tidy timer (an `evtimer` set to 3600 seconds, whose callback `server_tidy_event` expires stale format jobs and, off macOS, calls `malloc_trim(0)` to return freed memory to the OS), initializes the access-control list, arms the accept event, and finally enters `proc_loop`. From that point the server is live: it accepts clients, services them, and owns state.

Detach-survival is not a feature the server implements so much as a consequence of the architecture, and the code makes the consequence explicit. When a client detaches, the server receives the detach and tears down that client — but the session the client was viewing is untouched, because the session belongs to the server, not the client. The decision to actually `exit` is made in `server_loop`, deliberately and conservatively: the server exits only when its policy options say it should (`exit-empty` when no sessions remain, `exit-unattached` when no client is attached) *and* nothing is still pending — no command waiting, no job running. Until all of those line up, the server keeps looping, which is exactly why your programs keep running after you close the window.

Killing the server is the deliberate counterpart to surviving a detach. A `SIGINT/SIGTERM` sets `SERVER_EXIT`

and sends every client an exit message; a `kill-server` command walks the same path; and on exit the server runs its cleanup — `job_kill_all` to terminate background jobs, `status_prompt_save_history` to persist the command-prompt history — before calling `exit(0)`. The signal handler `server_signal` also wires the other operational signals faithfully: `SIGCHLD` reaps children, `SIGUSR1` recreates the socket (so a moved or deleted socket can be restored without losing the server), and `SIGUSR2` toggles logging. The lifecycle, end to end, is the tmux lifecycle: start as a daemon, own everything, survive every client, and exit only when asked or when policy says the work is done. With the server understood, the natural next question is the other half of the split — how a client finds this server, attaches to it, and becomes your window onto its state.

A walk through one `server_loop` iteration

It is worth slowing down on a single pass of `server_loop`, because the order of its steps is not arbitrary and the parity discipline reaches all the way down to it. The loop body runs, conceptually, in five stages, and each stage exists to leave the world consistent for the next. First it stamps `CURRENT_TIME` with `gettimeofday`, so every piece of work in this iteration — a format that expands `#{t:...}`, an activity timer, a status redraw — sees one coherent notion of “now” rather than drifting by microseconds across the pass. Second, it drains command queues: the global queue first, then each identified client’s queue, calling `cmdq_next` until no item is ready, so that a `bind-key` that ran three commands has all three applied before anything downstream observes the state. Third, `server_client_loop` walks every client, servicing pane output, recomputing which panes are visible, and flushing redraws. Fourth, alerts and hooks that the command work armed get their chance to fire. Only fifth, at the very end, does the loop ask “should the server exit?”

That final question is a conjunction, and getting the conjunction exactly right is why your programs survive a closed laptop lid. The server exits only if *all* of these hold: the relevant policy option says it may (`exit-empty` with no sessions, or `exit-unattached` with no attached client), *and* no command is still waiting in a queue, *and* no background job is still running, *and* the server is not in the middle of a config load. Miss any conjunct — treat “no clients attached” as sufficient on its own, say — and the server would exit out from under a detached session with a long-running build still going. The C computes this conjunction in a specific order with specific short-circuits, and ztmux ports the order, because “when exactly does the server decide it is done?” is observable behavior a parity `.sh` case can pin: detach from a session running `sleep 100`, confirm the server is still alive, re-attach, confirm the sleep is still there.

Chapter 4 – The Client

The client is the thin half: it connects to the server socket, forwards the terminal, relays keys, and renders what the server sends. This chapter covers the attach handshake, the terminal hand-off, and the client-side of the protocol.

Connecting to the server socket and the attach handshake

A client's first job is to find the server, and `client_connect` in `src/client.rs` does it. It fills an `AF_UNIX sockaddr_un` with the socket path and calls `connect`. If the connection is refused or the path does not exist (`ECONNREFUSED/ENOENT`) and the caller passed the `STARTSERVER` flag — as an attaching command does — the client autostarts a server: it takes a lock file next to the socket ("`{path}.lock`", via `client_get_lock`, an exclusive non-blocking `flock` so two racing clients cannot both start a server) and calls the very `server_start` from Chapter 3, forking the daemon in the background before connecting to the socket it now provides. On a successful connect the descriptor is switched to non-blocking. This is the mechanism behind the everyday experience that `ztmux` “just works” whether or not a server is already running: the first client to arrive brings the server up.

Once connected, the client runs `client_main`, which chooses the message type for this invocation — `MSG_SHELL` for the shell-command path, `MSG_COMMAND` for a normal command — starts its own `tmuxproc`, sets up signal handling with `client_signal`, and registers the peer with `proc_add_peer`, wiring `client_dispatch` as the callback for messages arriving from the server. Then it performs the *identify* handshake, which is the heart of attaching: the client has to tell the server everything about the terminal it is bringing to the party, so the server can drive that terminal correctly.

`client_send_identify` sends that description as a sequence of `msg` messages, one fact per message: `MSG_IDENTIFY_LONGFLAGS` (the 64-bit client flags), `MSG_IDENTIFY_TERM` (the `TERM` value), `MSG_IDENTIFY_FEATURES` (negotiated terminal features), `MSG_IDENTIFY_TTYNAME` (the tty path from `ttyname(STDIN_FILENO)`), `MSG_IDENTIFY_CWD` (the working directory), one `MSG_IDENTIFY_TERMINFO` per terminfo capability the client read, `MSG_IDENTIFY_STDIN` and `MSG_IDENTIFY_STDOUT` (which actually pass *duplicated file descriptors* across the socket, so the server can drive the client's real terminal), `MSG_IDENTIFY_CLIENTPID`, one `MSG_IDENTIFY_ENVIRON` per environment variable, and finally `MSG_IDENTIFY_DONE` to mark the end. The server assembles a client object from this stream. The attach completes when the server replies `MSG_READY`; the client's `client_dispatch_wait` handler sets `CLIENT_ATTACHED` and sends an initial `MSG_RESIZE`, and from that moment the client is a live window onto a server session.

Before any of that, the client also verifies it is speaking to a compatible server. On `MSG_VERSION` it checks the server's protocol version against its own `PROTOCOL_VERSION` (8), and on mismatch prints “protocol version mismatch (client {}, server {})” and refuses to proceed. This is the guard that keeps a client and a server built from different code from corrupting each other — the `msg` protocol is an internal contract, not a stable ABI, and the version check enforces that both halves come from the same build.

Terminal hand-off: raw mode, size, and the render surface

For the server to draw a session into the client's terminal, the client must hand that terminal over in a known state, and `client_main` does the hand-off carefully. It reads the working directory, the `TERM` value, and the tty name; reads the terminal's terminfo capability list (`tty_term_read_list`, Chapter 10); closes the inherited pty-master descriptor it no longer needs; and frees the global options and environment it will not use as a client. Then, for the control-terminal path, it puts the terminal into raw mode — `cfmakeraw` as a base, then the specific tmux flag set: input flags cleared except a controlled `ICRNL/IXANY`, output `OPOST | ONLCR`, control `CREAD | CS8 | HUPCL`, and the canonical single-byte read discipline `VMIN = 1, VTIME = 0`. Raw mode is what lets tmux see every keystroke immediately and unprocessed, which is the precondition for a multiplexer that binds keys and forwards the rest verbatim.

The “render surface” on the client side is deliberately minimal, because the client does not own the grid — the server does. The client's terminal *is* the surface: the server computes what each pane should look like, and drives the client's real terminal through terminfo to make it look that way. The client hands over the raw terminal and the terminfo capabilities describing it; the server does the drawing. This is the opposite of an emulator like `zterminal`, which owns a GPU grid and paints pixels; a tmux client paints nothing itself — it lends its terminal to the server and relays events. Chapter 10's terminal back end is where the server-side drawing lives, and it drives *this* client's terminal through the descriptors the identify handshake passed across.

Terminal size is a first-class part of the hand-off and a recurring event afterward. The initial `MSG_RESIZE` the client sends on attach tells the server how big the terminal is; thereafter, every time the terminal is resized, the client's `SIGWINCH` handler sends a fresh `MSG_RESIZE`, and the server relays out the geometry so the layout engine can re-tile the panes to the new size. Size is negotiated, not assumed, and it is renegotiated continuously — which is why dragging a tmux window larger reflows the panes live.

The hand-off is reversible, and its reversibility is the whole point of a multiplexer. When the client detaches or dies, `client_signal` and the dispatch handlers restore the terminal and tear down the client, but they do *not* touch the server's grid — the session keeps its size and state until the next client attaches and hands over its own terminal. The terminal is borrowed, used, and returned; the session persists across the loan.

Keys in, cells out: the client relay loop

Once attached, the client runs a tight relay loop, and it is genuinely thin: keys go in, cells come out, and almost nothing is decided on the client side. Input from the client's terminal is read and forwarded to the server, which interprets it — against the current key table and bindings (Chapter 9) — and either runs a bound command or forwards the key on to the active pane's program. Output flows the other way: the server draws each pane and sends the client the terminal sequences to make its real terminal match, which the client writes out. The client is a conduit; the intelligence is in the server.

The messages that carry this traffic are dispatched by `client_dispatch`, which splits into two handlers depending on whether the client has attached yet. `client_dispatch_wait` handles the pre-attach messages — `MSG_VERSION`, `MSG_READY`, `MSG_SHELL`, the file read/write messages, and the various exit and detach messages that can arrive before attach completes. `client_dispatch_attached` handles the steady-state traffic — flag changes (`MSG_FLAGS`), detach requests (`MSG_DETACH/MSG_DETACHKILL`), execution requests (`MSG_EXEC`), suspend (`MSG_SUSPEND`, which stops the client with `SIGTSTP`), lock (`MSG_LOCK`, which runs the lock command and replies `MSG_UNLOCK`), and the exit and shutdown messages. The client's whole behavior is this dispatch table; the loop underneath it is libevent's.

Two of these paths are worth naming because they show how much the client deliberately does *not* do itself. `MSG_EXEC` is how the server tells the client to run a program in place of itself — used for things that must happen in the client's own process and terminal — and the client obeys via `client_exec`, which sets up the environment and `exec`s the shell. `MSG_LOCK` similarly has the client run a lock program locally. In both cases

the server decides *what* should happen and the client is the hands that carry it out in the right process context. The division is consistent: policy in the server, mechanism in the client.

Because the client is thin, its failure modes are simple and its exit reasons are enumerated precisely. The `client_exitreason` enum names every way a client can end — detached, detached-and-hung-up, lost-tty, terminated, lost-server, server-exited, exited, and so on — and the client prints the corresponding message on the way out. This is faithful tmux behavior and it is useful behavior: when a client goes away you get a specific reason, not a generic disconnect. The client does one job — relay a terminal to and from a server — and it reports exactly how that job ended.

Detach, re-attach, and multiple clients on one session

Detach is the multiplexer's signature move, and on the client side it is undramatic: the client receives (or initiates) a detach, restores the terminal, and exits, while the server keeps the session running. Re-attach is the mirror image: a new client connects, runs the identify handshake with *its* terminal's characteristics, and the server binds it to the session, resizing the session to the new terminal and redrawing it from the grid the server never stopped maintaining. Nothing about the session's programs, scrollbar, or layout was lost in between, because none of it ever lived in the client. Detach and re-attach are just the loan of the terminal ending and a new loan beginning.

Multiple clients on one session is the same mechanism run concurrently. Two clients can attach to the same session at once; both hand their terminals to the server, both receive the server's drawing of the same window, and both relay keys into the same panes. Because there is exactly one session tree in the server, the two clients are genuinely looking at the same thing — a keystroke or a pane switch from one is seen by the other, mediated by the server's single copy of the state. This is how tmux does pair programming and screen sharing without any special mode: it is just two clients viewing one server-owned session.

The interesting wrinkle multiple clients introduce is *size*, and tmux resolves it with an option `zterm ports faithfully`. When two attached terminals are different sizes, the session cannot be both sizes at once; the `window-size` option decides whether the session tracks the largest client, the smallest, the latest to attach, or a manual size, and the layout is computed against that. Each `MSG_RESIZE` from any client feeds this computation, and the server re-tiles accordingly. The client does not participate in the decision — it just reports its size and draws what the server sends — but the decision exists because clients can differ.

The through-line of the whole chapter is that the client is *defined by what it refuses to own*. It owns no session state, no grid, no layout, no options, no bindings; it owns a socket, a terminal in raw mode, and a dispatch table. That poverty is exactly what makes detach-survival, re-attach, and multi-client sharing fall out for free — there is nothing in the client to lose, so losing the client loses nothing. With both halves of the split now walked, the book descends into the thing every client is ultimately a window onto and every server ultimately maintains: the grid of cells at the bottom of the pane tree.

A worked example: two clients, two sizes

The `window-size` option earns a concrete walk, because it is where the multi-client story gets an arithmetic answer. Suppose client A attaches to session `work` from an 120×40 terminal, and then client B attaches to the same session from an 80×24 terminal. The session cannot be both sizes at once, so the layout has to be computed against *one* geometry, and `window-size` decides which:

- `largest` — the session sizes to 120×40, the union that fits the biggest client. Client B (80×24) sees only the top-left 80×24 of a larger virtual window; the `window_bigger` flag is set and `window_offset_x/window_offset_y` (Appendix B) track what portion B is scrolled to, so B can pan around the larger window.
- `smallest` — the session sizes to 80×24, the intersection every client can show fully. Client A (120×40) sees the 80×24 session centered or corner-aligned in its larger terminal, with the surplus area filled by `fill-character`. Nothing is clipped for anyone.

- `latest` — the session tracks whichever client most recently did something, so it follows the active user's terminal.
- `manual` — the size is fixed by `resize-window` and ignores client sizes entirely.

Every `MSG_RESIZE` from any client (Chapter 17) feeds this computation, and on each one the server recomputes the chosen geometry, re-tiles the layout (Chapter 7), reflows the grids (Chapter 5), and redraws each client to *its own* terminal. The client contributes only its size and its drawing surface; the decision is entirely server-side. This is the precise mechanism behind the everyday experience of two people sharing a tmux session from differently sized windows — and porting the option's four choices faithfully is what makes that experience identical under ztmux, down to which corner the smaller client anchors and what character fills the gap.

Chapter 5 – The Grid and Screen Model

Everything visible is a grid of cells. This chapter covers the data structure at the heart of the multiplexer: the grid, the screen, scrollback history, cell attributes and colors, and how panes are windows onto grids.

The grid: cells, lines, and scrollback history

At the very bottom of the pane tree is the `grid` — the two-dimensional array of character cells that a pane's program writes into and a client eventually sees. Its struct (defined in `src/lib.rs`, with the functions in `src/grid.rs`) is small and telling: `sx` and `sy` are the visible columns and rows; `hsize`, `hlimit`, and `hscrolled` track the scrollback history — how many lines of history exist, the maximum allowed, and how far the viewer has scrolled up into it; `linedata` is a pointer to an array of `grid_lines`. The grid is created by `grid_create(sx, sy, hlimit)`, which allocates `sy` lines and sets the `GRID_HISTORY` flag only when `hlimit` is non-zero — a scrollback of zero is a grid that never keeps history, and the flag records that choice.

A `grid_line` is where the port's attention to memory layout shows. Each line holds `celldata` (a pointer to packed cell entries), `cellused` and `cellsize` (how many of the allocated cells are in use and how many are allocated), a separate `extddata/extdsize` pair for extended cells, a `flags` field, and a `time` stamp. Lines grow on demand: `grid_expand_line` snaps the allocation up through a sequence of sizes — a quarter of the screen width, then half, then the full width — so a line that fills up reallocates a bounded number of times rather than once per character. This is tmux's exact growth policy, ported rather than reinvented, because the allocation pattern is part of the behavior the parity suite pins.

Cells come in two physical forms for space, and understanding the split is the key to the whole grid. Most cells are *packed*: a `grid_cell_entry` whose union holds a compact `grid_cell_entry_data` of four bytes (`attr`, `fg`, `bg`, `data`) plus a flags byte — enough for an ASCII character with a 256-color palette entry and basic attributes. Cells that need more — a wide or combining Unicode character, a 24-bit RGB color, a non-default underscore color, or a hyperlink id — are *extended*: the packed entry stores an offset into the line's `extddata` array of `grid_extd_entry` values, which carry the full `utf8_char`, a 16-bit attribute field, and the wide color and link fields. `grid_need_extended_cell` decides which form a cell requires, and `grid_store_cell/grid_extended_cell` place it. The packed/extended split is how tmux keeps the common case cheap without capping the rich case, and ztmux ports the promotion rules exactly.

Scrollback is the history at the top of the grid, and it is managed by a small family of functions. `grid_scroll_history` allocates a fresh bottom line and grows `hsize` and `hscrolled`; `grid_scroll_history_region` does the same within a scroll region. The bound is enforced by `grid_collect_history`: once `hsize` reaches `hlimit`, it frees the oldest `hlimit / 10` lines (at least one) — the “drop the top ten percent” policy that amortizes the cost of trimming rather than freeing one line per scroll. The default `hlimit` comes from the `history-limit` session option, which defaults to 2000 lines (`src/options_table.rs`) and, per its own help text, applies only to newly created panes when changed. Scrollback is thus a bounded ring of past lines, and everything copy mode navigates (below) is a walk through it.

A worked example: the ten-percent trim

The scrollback bound has a specific amortization policy, and watching it act explains why tmux does not simply free one line per scroll. Suppose a pane's `history-limit` is 2000 and its history is full — `hsize == 2000`. A new line scrolls off the top. A naive implementation would free exactly one line to make room, paying a free

(and the bookkeeping) on every scroll forever. `grid_collect_history` instead waits until `hsize` reaches `hlimit` and then frees the oldest `hlimit / 10` lines at once — 200 lines here, or at least one if the tenth is zero — so the pane now holds 1800 lines and will not trim again for another 200 scrolls. The cost of trimming is amortized across those 200 scrolls rather than paid on each one, which matters when a program is scrolling thousands of lines a second (a `yes` loop, a fast build log). This “drop the top ten percent” policy is tmux’s exact rule, ported rather than reinvented, because the allocation and free pattern is part of the behavior the parity discipline pins — and because getting it wrong (trimming one line at a time) would be a quiet performance regression invisible to a correctness test but obvious under a flood.

Cell attributes and color (`attributes.rs`)

A cell is not just a character; it is a character *styled*, and the style lives in the `grid_cell`’s `attr`, `fg`, `bg`, `us`, and `link` fields. The `attr` field is a 16-bit bitflag set (`grid_attr`, in `src/lib.rs`) with a flag per rendition: `GRID_ATTR_BRIGHT` (which is bold — tmux, like the SGR spec, treats bold and bright as one), `DIM`, `UNDERSCORE`, `BLINK`, `REVERSE`, `HIDDEN`, `ITALICS`, `CHARSET` (the ACS line-drawing charset), and — above the low byte, so a cell carrying them is forced extended — `STRIKETHROUGH`, `OVERLINE`, and the four additional underline styles `UNDERSCORE_2` through `UNDERSCORE_5` (double, curly, dotted, dashed). The `GRID_ATTR_ALL_UNDERSCORE` union groups the five underline flavors so code can ask “is this cell underlined in any style?” in one test.

Color is an `i32` with a tagging scheme, and it repays a close look because it encodes three color spaces in one field. The values 0–7 and 90–97 are direct ANSI palette indices; 8 is “default” (`COLOUR_DEFAULT` also treats 9 as default); an index or-ed with `COLOUR_FLAG_256` (`0x01000000`) is a 256-color palette entry; and a packed `(r << 16) | (g << 8) | b` or-ed with `COLOUR_FLAG_RGB` (`0x02000000`) is a 24-bit truecolor value. The `fg` and `bg` fields carry text and background color this way, and `us` carries the underscore color separately — which is why an underline can be a different color from the text it sits under, a feature modern terminals expose through SGR 58 and tmux supports. `src/colour.rs` holds the pack/unpack helpers; the grid just stores the tagged integer.

The bridge between these numeric representations and the human-readable names that appear in config files and `#{...}` formats is `src/attributes.rs`. Its `attributes_tostring` turns an `attr` bitset into a comma-separated string, and `attributes_fromstring` parses such a string back into flags, against a fifteen-entry table that aliases both “`bright`” and “`bold`” to `GRID_ATTR_BRIGHT` and names the exotic underlines (“`double-underscore`”, “`curly-underscore`”, “`dotted-underscore`”, “`dashed-underscore`”) and “`acs`” for the charset flag. This is the table that lets you write `bold,underline` in a style option and have it mean the right bits, and because it is a direct port of tmux’s table, the accepted names are exactly tmux’s.

The default cell ties the representation together. `GRID_DEFAULT_CELL` is a single space with empty attributes and `fg`, `bg`, and `us` all set to 8 (default) — the cell every fresh grid position starts as. There are a few other canonical cells: `GRID_PADDING_CELL`, a zero-width placeholder (marked with the `PADDING` grid flag) that sits in the second column of a wide character; and `GRID_CLEARED_CELL`, which records that a cell was explicitly cleared rather than merely never written. These canonical cells are what clearing, wide-character placement, and grid comparison (`grid_cells_equal`, `grid_cells_look_equal`) are defined against, and they are ported byte-identically because “cleared” versus “never written” is a distinction tmux’s redraw logic actually depends on.

Screen vs. grid: the visible window onto history

The `grid` is raw storage; the `screen` is the stateful view of it, and separating the two is what lets one grid support scrollbar, alternate screens, and cursor motion cleanly. The `screen` struct (`src/lib.rs`, functions in `src/screen.rs`) owns a `grid` pointer plus everything about how a program is currently drawing into it: the cursor position (`cx`, `cy`), the cursor style and color, the scroll region (`rupper` and `rlower`, the top and bottom rows a scroll is confined to), the mode flags (`MODE_CURSOR`, `MODE_WRAP`, and the rest), the tab stops, the title stack, and the selection. Where the grid answers “what characters are stored?”, the screen answers “where is the cursor, what region scrolls, what mode are we in, and what is selected?”

`screen_init` creates the backing grid with `grid_create` and sets the initial mode to `MODE_CURSOR`; `screen_reinit` resets it, setting the scroll region to the full height and the mode to `MODE_CURSOR | MODE_WRAP`. Tab stops default to every eighth column (`screen_reset_tabs`), stored compactly as a bit string. Resize is where the screen/grid split earns its keep: `screen_resize` (and `screen_resize_cursor`, which takes a `reflow` flag) changes the visible dimensions and, when reflow is on, rewraps the grid's lines to the new width — the machinery in `grid.rs`'s `grid_reflow_*` functions — so that making a pane narrower re-wraps long lines instead of truncating them. The cursor is moved to stay sensible across the resize. This is subtle code and it is a faithful port, because reflow-on-resize is precisely the sort of behavior a spec would under-specify and a parity suite pins.

The *alternate screen* is the screen model's most visible trick and a clean example of why the split exists. Full-screen programs — an editor, a pager, `http` — switch to an alternate screen so that when they exit, the shell's scrollbar is exactly as they left it. `screen_alternate_on` saves the current cursor and, optionally, the whole grid into `saved_grid/saved_cx/saved_cy`; `screen_alternate_off` restores them. Because the alternate screen is a property of the `screen` (the view) rather than a second `grid`, entering and leaving it is a save/restore of view state, and the primary scrollbar is untouched underneath — which is exactly the behavior you rely on every time you quit `less` and your prompt is where you left it.

Panes are windows onto grids in the literal sense this chapter's title promises. Each pane owns a screen, and each screen owns a grid; the pane's program writes into the screen through the screen-write API (below), the screen mutates the grid, and the server draws the visible portion of the grid — offset by `hscrolled` when the viewer has scrolled up — into whatever region of the client's terminal the layout assigns the pane. The `screen_size_x/screen_size_y/screen_hsize/screen_hlimit` accessors are thin wrappers over the grid's dimensions, because the screen's size is the grid's visible size. A pane is thus three nested objects — pane, screen, grid — and the client sees a rectangle of the innermost one.

Copy mode and the selection over the grid

Copy mode is how you read and copy the scrollbar without a mouse, and it is implemented as a *mode* over the grid rather than a separate buffer — which is why it can scroll seamlessly from the visible screen up into history. When a pane enters copy mode, a virtual cursor detaches from the program's cursor and moves over the grid under keyboard control; the grid keeps updating underneath (the program is still running), but the copy-mode cursor navigates the frozen coordinate space of stored lines, including the scrollbar above the visible region. The `grid_reader` struct — a `gd` pointer plus a `cx/cy` position — is the virtual cursor's state, and `src/grid_reader.rs` holds the motions that walk it word by word and line by line.

The selection is a region of the grid, and it lives on the screen as a `screen_sel`. Its fields are exactly what a selection needs: `sx/sy` and `ex/ey` for the start and end coordinates, a `rectangle` flag for block versus stream selection, the `modekeys` (emacs or vi) that govern the motions, and a `cell` template for how selected cells are drawn. `screen_set_selection` establishes it, `screen_clear_selection` and `screen_hide_selection` remove or hide it, and `screen_check_selection(s, px, py)` answers the question the renderer asks for every cell: “is this position selected?” — which is how the highlight follows the selection as it grows.

Copy itself is defined against the grid's string-extraction functions, and this is where color and attributes come back. `grid_string_cells` walks a run of cells and produces a string; with the right flags it can embed the SGR escape sequences that reproduce each cell's color and attributes, using the helper family `grid_string_cells_fg`, `grid_string_cells_bg` (which emit `38;5;n / 38;2;r;g;b` and the 30–37/90–97 forms), `grid_string_cells_us` (the 58-based underscore color), and `grid_string_cells_code` (a table mapping each `grid_attr` flag to its SGR code — bright to 1, italics to 3, and so on). This is why a copy-mode selection can preserve styling: the extractor can regenerate the escape sequences that produced it. Whether a copy keeps styling or strips to plain text is governed by the string flags (`grid_string_flags`: with-sequences, trim-spaces, used-only, and the rest).

The larger point copy mode makes is that the grid is not just a rendering buffer — it is a queryable, extractable, navigable data model, and every higher-level feature is expressed against it. Copy mode is a cursor and a selection over the grid; search in copy mode is a scan over the grid; the status line and `capture-pane` are

string extractions from the grid; even the layout engine ultimately exists to decide which rectangle of which grid goes where on screen. The grid is the substrate, and the rest of the multiplexer is operations on it. The next chapter climbs one level down the input path — to the parser that turns a program's raw byte stream into the grid mutations this chapter has been treating as given.

Reflow: rewrapping wrapped lines when a pane narrows

The screen/grid split earns its keep most dramatically on resize, and the hardest part of resize is *reflow* — rewrapping soft-wrapped lines to a new width instead of truncating them. The distinction the machinery turns on is a single per-line flag. When a program prints past the right margin and the line continues onto the next row, tmux does not store a newline; it stores the continuation and marks the first line `GRID_LINE_WRAPPED`. A real newline (the program pressed Enter, or `screen_write_linefeed` ran) leaves the flag clear. So the grid already records, line by line, which line breaks are *soft* (an artifact of the current width) and which are *hard* (structural). Reflow is the operation that honors that distinction: it may re-break the soft ones at a new width, and it must never merge across a hard one.

`grid_reflow(gd, sx)` (`src/grid.rs`) does the work, and its shape is a rebuild rather than an edit. It walks the existing lines building a *new* line array at the target width `sx`: a run of lines joined by `GRID_LINE_WRAPPED` flags is treated as one logical line, concatenated, and then re-split into as many physical lines of width `sx` as it needs, each but the last carrying the wrapped flag again; a hard newline ends the logical line and starts a fresh one. The helper family `grid_reflow_add` (append a new empty line to the rebuild) and `grid_reflow_move` (move existing cells into it) keep the accounting straight, and the history size `hsize` is adjusted as lines are created or absorbed, because rewrapping a full scrollbar at a narrower width produces *more* lines than it started with. This is why making a pane narrower can lengthen its scrollbar: the same text, wrapped tighter, occupies more rows.

The cursor is the subtle part, because the row the cursor sits on may move during the rebuild — a line above it might split in two, pushing it down. `screen_resize_cursor` carries a `reflow` flag precisely so the resize path can track the cursor's *logical* position through the rewrapping and restore it to the corresponding physical row afterward, rather than leaving it stranded where the old geometry put it. Reflow is expensive — it reallocates the whole line array and touches every line of history — so it runs only on a width change, and only when the `window-size/reflow` policy calls for it; a pure height change resizes without rewrapping. The whole operation is a faithful port because reflow-on-resize is exactly the sort of behavior no specification pins down and every user notices: shrink your terminal, and the long lines in your scrollbar must re-wrap the way tmux re-wraps them, not truncate and not scramble. The parity discipline reaches it through `.sh` cases that resize a pane and read back the wrapped result.

A worked example: what one styled character costs in memory

Follow a single character all the way into the grid and the packed/extended split stops being abstract. Suppose a program prints a plain ASCII `A` in the default style. The screen-write layer builds a `grid_cell` whose `data` is the UTF-8 for `A`, whose `attr` is `0`, and whose `fg/bg/us` are all `8` (default). `grid_need_extended_cell` looks at that cell and answers *no*: the character is a single byte, the colors are palette indices that fit, no attribute above the low byte is set. So `grid_store_cell` writes a *packed* `grid_cell_entry` — five bytes: one flags byte plus the four-byte `grid_cell_entry_data` of `attr`, `fg`, `bg`, and the single data byte. That is the common case, and it is deliberately cheap, because a screenful of text is thousands of exactly this.

Now suppose the program prints a red `A` — SGR `38;2;220;50;47` then `A`. The cell's `fg` is now `(220<<16)|(50<<8)|47` or-ed with `COLOUR_FLAG_RGB` (`0x02000000`), which no longer fits the packed entry's one-byte `fg` slot. `grid_need_extended_cell` answers *yes*, and the cell is promoted: the packed entry

now stores an *offset* into the line's `extddata` array, and a `grid_extd_entry` at that offset carries the full `utf8_char`, a 16-bit `attr`, and the wide `fg/bg/us/link` fields — several times the packed size. The same promotion happens for a wide CJK glyph (which also claims a `GRID_PADDING_CELL` in the next column), a combining sequence, a curly underline (`GRID_ATTR_UNDERSCORE_2`, a flag above the low byte), or a hyperlink id. The lesson the example teaches is the one the whole grid is built on: you pay for richness only in the cells that use it, and `grid_need_extended_cell` is the exact predicate — ported from the C, not re-derived — that decides who pays.

Chapter 6 – The Input Parser

Bytes arrive from the program running in a pane; the input parser turns that VT stream into grid mutations. This chapter walks the state machine — escape sequences, CSI/OSC handling, and how the parser drives the screen.

The VT parser state machine

The input parser in `src/input.rs` is a direct implementation of Paul Williams' well-known DEC ANSI-compatible parser — the state machine that virtually every serious terminal implements — extended, as tmux extends it, to handle UTF-8, to let OSC strings terminate on a bare `BEL` as well as the formal string terminator, and to add the APC, rename, and DCS-escape states tmux needs. Its own module documentation lays out the model, and its structure is faithful to it: a set of *states*, each with an *entry* action, an *exit* action, and a *transition table* keyed by input byte, plus a small set of *anywhere* transitions that apply in every state.

Concretely, there are twenty `input_state` values, each a struct carrying a name, an optional enter hook, an optional exit hook, and a slice of transitions: `GROUND` (the resting state, where printable bytes get written to the screen), `ESC_ENTER` and `ESC_INTERMEDIATE`, the CSI family (`CSI_ENTER`, `CSI_PARAMETER`, `CSI_INTERMEDIATE`, `CSI_IGNORE`), the DCS family (`DCS_ENTER`, `DCS_PARAMETER`, `DCS_INTERMEDIATE`, `DCS_HANDLER`, `DCS_ESCAPE`, `DCS_IGNORE`), the string states (`OSC_STRING`, `APC_STRING`, `RENAME_STRING`), and `CONSUME_ST` for swallowing a trailing string terminator. Each transition (`input_transition`) names a byte range (`first`, `last`), an optional handler function, and an optional next state; the `INPUT_STATE_ANYWHERE` transitions handle the bytes that mean the same thing in any state — `0x18/0x1a` (CAN/SUB) abort the current sequence back to ground, and `0x1b` (ESC) always begins a new escape.

The driver is `input_parse`, and it is a plain, fast loop: for each byte, read the current state's transition table, linear-scan it for the entry whose `first..=last` range contains the byte, run that entry's handler, and — unless the handler suppresses it — switch to the entry's next state, running the old state's exit hook and the new state's enter hook (`input_set_state`). Because printable characters are the overwhelmingly common case, they are special-cased: the ground state's printable range routes to `input_print`, and the loop avoids the collection-flush overhead for that path. The parser has entry points for the different sources it serves — `input_parse_pane` for a pane's program output, `input_parse_buffer` and `input_parse_screen` for other callers — but they all funnel into the same `input_parse` loop.

The parser is defensive in the ways a terminal must be, and the bounds are named constants rather than magic numbers. Intermediate bytes collect into a four-byte buffer, parameters into a sixty-four-byte buffer, the parameter list holds up to twenty-four parsed values, and the OSC/DCS string input grows on demand but is capped at `INPUT_BUF_LIMIT` (1048576 — one megabyte) so a runaway sequence cannot exhaust memory. A five-second timer (`input_start_timer`) guards against a sequence that begins and never completes, resetting the parser to ground so a pane cannot be wedged forever mid-escape by a malicious or broken program. These limits are tmux's, ported directly, and they are the difference between a parser that is merely correct and one that is also robust against hostile input.

CSI, OSC, and the escape-sequence surface

The three big escape families dispatch through three tables, and each table is a sorted, searchable map from the sequence's identifying bytes to a handler. For `ESC` sequences — the short two-byte codes like `ESC 7`

(save cursor), `ESC 8` (restore cursor), `ESC M` (reverse index), `ESC c` (full reset), `ESC #8` (the DEC alignment test) — `input_esc_dispatch` looks the final byte and intermediate up in `INPUT_ESC_TABLE` (keyed through `input_table_compare`) and calls the matching handler; the `input_esc_type` enum names each (DECALN, DECKPAM/PNM, DECRC/DECSC, HTS, IND, NEL, RI, RIS, the charset selectors). These are the terse controls that predate CSI, and they are handled in full.

CSI — the `ESC [` family — is the large one, and it is where a terminal earns its compatibility. `input_csi_dispatch` first parses the accumulated parameter buffer into the parameter list with `input_split` (splitting on `;`, and treating `:` as introducing sub-parameters, which is how the colon-form SGR color sequences are carried), then binary-searches `INPUT_CSI_TABLE` — a forty-entry table keyed by the final byte plus any intermediate — for the sequence’s type from the `input_csi_type` enum. The catalogue is the full working set: cursor motion (CUU/CUD/CUF/CUB/CUP), erase (ED/EL), insert and delete of characters and lines (ICH/DCH/IL/DL), scroll (SU/SD), erase character (ECH), the scroll-region setter (DECSTBM), device status and attributes (DSR/DA), the SGR rendition setter, and the mode setters (SM/RM), each with private-mode variants dispatched through `_sm_private/_rm_private` for the `CSI ?` forms. Parameters are fetched with defaults through `input_get(ictx, idx, minval, defval)`, so a `CSI H` with no arguments correctly means “home.”

Replies are part of the CSI surface, because some sequences are *questions* the terminal must answer, and the parser answers them through an `input_reply!` macro that writes the response back toward the program. A Device Attributes request (`CSI c`) is answered `\x1b[?1;2c` — or `\x1b[?1;2;4c` when the sixel feature is built in, advertising graphics support — and a secondary DA is answered `\x1b[>84;0;0c`. A cursor-position report (`DSR` with parameter 6) is answered with the live cursor position, `\x1b[{row};{col}R`. These replies are how programs probe what the terminal can do, and getting them exactly right — including advertising sixel only when it is actually compiled in — is part of being a faithful terminal rather than one that merely renders.

OSC — the `ESC ]` family — carries the string-valued controls, and it is dispatched when the OSC string state exits (`input_exit_osc`): the leading numeric option is parsed and matched. `0` and `2` set the window title (via `screen_set_title`, gated by the `allow-set-title` option); `4` sets palette entries; `7` reports the working directory/path; `8` carries hyperlinks; `10/11/12` set the foreground, background, and cursor colors; `52` accesses the clipboard; `104/110/111/112` reset palette entries; and `133` carries the shell-integration prompt marks (FinalTerm/iTerm2-style semantic prompts). The neighboring string states handle the rest: APC strings, and the rename string (`ESC k`) that sets a window’s name, gated by `allow-rename`. The gating options matter — they are the guardrails that stop a remote program from silently renaming your windows or repainting your palette, and the parser honors them.

From parsed sequence to grid mutation

The parser does not touch the grid directly; it drives the *screen-write* API, and that indirection is deliberate. Every handler in `input.rs` ultimately calls a `screen_write_*` function (`src/screen_write.rs`) — `screen_write_cursormove`, `screen_write_linefeed`, `screen_write_scrollup`, `screen_write_clearline`, `screen_write_insertcharacter`, and so on — and those functions perform the actual grid mutation, update the cursor, respect the scroll region, and mark the affected cells dirty for redraw. The parser’s job ends at “decide what should happen”; the screen-write layer’s job is “make it happen to the grid correctly.” This is the same policy/mechanism split the client/server relationship showed, one level down: the parser is policy, screen-write is mechanism.

The most common mutation — printing a character — goes through a batched fast path, because it is where throughput lives. `input_print` does not write each character to the grid individually; it accumulates printable characters and hands them to `screen_write_collect_add`, which batches them into a run, and the batch is flushed (`screen_write_collect_end`) when a non-print sequence arrives or the run must be committed. This collection path is why *cat*-ing a large file through a tmux pane is fast: the parser turns a stream of thousands of printable bytes into a handful of batched grid writes rather than thousands of individual cell stores. The SGR state — the current color and attributes — rides along in the parser’s `input_cell`, so each batched character is stamped with the active rendition without re-deriving it.

Character-set state is carried in the parser too, and it is a small but load-bearing example of state the parser

must track across sequences. The `input_cell` holds the G0 and G1 charset selections and which is currently active; the ESC charset selectors (`5C5G0/5C5G1` on and off) set them, and the shift-in/shift-out control bytes switch between them. When the ACS (alternate character set) charset is active, the `GRID_ATTR_CHARSET` flag is set on written cells so the drawing layer knows to map them through the line-drawing glyphs. This is how a TUI's box borders render as lines rather than as the letters `q` and `x`: the charset state, tracked across the byte stream by the parser, tags the cells, and the terminal back end (Chapter 10) maps them on the way out.

The color sequences are the richest mutation and show the whole pipeline in one place. `input_csi_dispatch_sgr` walks the parsed SGR parameters and updates the `input_cell`'s attributes and colors: `0` resets, `1` sets bright, `3` italics, `4` underline, `7` reverse, the `30–37/90–97` set the foreground from the ANSI palette, `38/48/58` introduce extended (256-color or RGB) foreground, background, and underscore colors through `_sgr_256/_sgr_rgb`, and `39/49/59` reset each to default. Both the semicolon form and the colon sub-parameter form (`input_csi_dispatch_sgr_colon`) are handled, because real programs emit both. The result of an SGR sequence is not a grid write at all — it is a change to the *cell template* — and the next printable characters inherit it. That is the model in miniature: the parser maintains style state, and printing applies it.

Semantic prompts and the clipboard: OSC 133 and OSC 52

Two OSC sequences the parser handles are worth singling out because they are how a modern shell and a modern terminal talk to the multiplexer, and because one of them is the raw material for a superset feature (Chapter 13). OSC 133 is the FinalTerm/iTerm2-style *semantic prompt* protocol: a shell emits `OSC 133 ; A` before its prompt, `; B` after it, `; C` before command output, and `; D` after — marking, in the byte stream, where prompts and command outputs begin and end. When the parser sees these, it sets grid line flags (`START_PROMPT`, `START_OUTPUT`) on the affected lines, so the grid itself records the structure of the session's history. Because that structure is stored, features that navigate by command — jump to the previous prompt, select the last command's output — have their data already in the grid; they are extensions the port is *positioned* for rather than ones needing new plumbing. `zshrs` running inside a pane emits exactly these marks, which is why the integration is more than aspirational: the tracking is live.

OSC 52 is the clipboard access sequence: a program inside a pane can set the system clipboard by emitting `OSC 52 ; c ; <base64>`, and tmux gates this behind the `set-clipboard` option, because letting arbitrary pane programs write your clipboard is a security decision, not a default. When enabled and the outside terminal supports it, tmux forwards the clipboard set outward; when disabled, it is ignored. The gating is the recurring theme of the OSC handlers (Chapter 6's `allow-set-title`, `allow-rename`): the parser recognizes the sequence but honors a policy option before acting, so a remote program cannot silently repaint your palette, rename your windows, or hijack your clipboard. Porting the gates, not just the sequences, is what keeps ztmux's security posture identical to tmux's.

Fidelity to xterm and the tmux input model

Fidelity here has a precise meaning: the parser must interpret the byte stream the same way tmux does, which is (very nearly) the way xterm does, so that any program written for a real terminal behaves correctly in a ztmux pane. The way ztmux guarantees this is not by aspiring to xterm compatibility in the abstract but by porting tmux's parser, table for table and handler for handler, and then checking the result against the real tmux with the parity suite. The state machine, the transition tables, the reply strings, the buffer limits, the gating options — all of them are tmux's, and when they diverged during porting, the parity suite caught it.

The tmux input model has one important twist over a bare emulator: tmux is *itself* a program running in someone else's terminal, so it has two input surfaces, not one. The parser in this chapter handles the bytes *from the programs running inside panes* — turning their output into grid mutations. A separate path (`src/input_keys.rs`, `src/tty_keys.rs`) handles the bytes *from the outside terminal's keyboard* — turning the user's

keystrokes into tmux key codes that the key tables (Chapter 9) dispatch. The `input.rs` parser is the inner one; it consumes what pane programs produce, not what the user types. Keeping the two straight matters, because they are different state machines solving different halves of the multiplexer's job.

The parser's faithfulness is what makes everything above it trustworthy. Copy mode (Chapter 5) is only correct if the grid it navigates was mutated correctly; the layout engine (Chapter 7) only tiles panes whose contents are right; the format system (Chapter 9) only reports pane state that was tracked right. All of that rests on the parser interpreting the VT stream exactly as tmux does, which is why this subsystem is one of the most heavily line-for-line ported in the tree and one of the first the parity suite's format and pane cases exercise indirectly. A multiplexer whose parser drifts is a multiplexer that corrupts editors and TUIs subtly; ztmux avoids that by not letting the parser drift from the C.

There is genuine subtlety the port has to preserve that a casual reading would miss — the exact handling of wrap-at-margin (does the cursor sit past the last column, or wrap immediately?), the interaction of the scroll region with insert/delete-line, the precise conditions under which a cell becomes extended, the timing of the auto-margin flag. These are the corners where terminals disagree and where TUIs have quietly grown to depend on one specific behavior. Because ztmux ports tmux's exact resolutions of these corners — with the C citation on each function and the parity suite over the whole — it inherits tmux's exact compatibility, including the parts no document fully describes. That inheritance is the point of porting rather than rewriting, and the parser is where it pays off most visibly.

The pending-wrap corner, in detail

One of those corners deserves to be spelled out, because it is the single most common source of subtle emulator bugs and the parser handles it exactly as tmux does. The question is: when a program prints a character in the last column of a line, does the cursor immediately wrap to the next line, or does it stay put? The naive answer — wrap immediately — is wrong, and TUIs depend on it being wrong. A terminal with the auto-wrap mode on does *not* move the cursor when it writes the last column; instead it sets a *pending-wrap* flag and leaves the cursor visually on the last column. The wrap happens only when the *next* printable character arrives, at which point the terminal first moves to column zero of the next line and then prints. If instead a cursor-motion or an erase sequence arrives, the pending-wrap flag is cleared and no wrap ever happens.

This deferred wrap is what lets a program fill the last column of the last row without scrolling the screen — draw a full-width box border, print a character in the bottom-right corner — which every full-screen TUI does constantly. Get it wrong by wrapping eagerly and the screen scrolls by one line every time a program touches the corner, corrupting the display. The parser and screen-write layer track the pending-wrap state on the screen and apply it at exactly the moment tmux does, and because the behavior is observable, it is the kind of thing the parser's line-for-line port and the parity suite together pin. It is a one-flag detail that no casual reading of “the parser writes characters to the grid” would surface, and it is precisely the sort of undocumented-but-depended-on behavior that porting (rather than rewriting from a spec) inherits for free.

Chapter 7 – Layouts

Panes are tiled by a layout engine: the classic tmux layouts, manual splits, and the layout string that serializes an arrangement. This chapter covers the geometry.

The layout tree and cell geometry

A window's panes are arranged by a tree, not a list, and the tree is the layout engine's central data structure. Each node is a `layout_cell` (defined in `src/lib.rs`, operated on in `src/layout.rs`): it carries a `type_`, a `parent` pointer, a size (`sx`, `sy`) and an offset (`xoff`, `yoff`) in the window's coordinate space, a `wp` pointer to a window pane, and a `cells` tail queue of children. The `type_` is one of just three values (`layout_type`): `LAYOUT_LEFTRIGHT` for a horizontal split (children side by side), `LAYOUT_TOPBOTTOM` for a vertical split (children stacked), and `LAYOUT_WINDOWPANE` for a leaf that holds an actual pane. Interior nodes split space; leaves point at panes. Every arrangement of panes you can make in tmux is some tree of these three node types.

The geometry is exact and it composes. A leaf's `sx/sy/xoff/yoff` are the pane's rectangle within the window; an interior `LAYOUT_LEFTRIGHT` node's width is the sum of its children's widths plus one column per internal border, and its height is each child's height; an interior `LAYOUT_TOPBOTTOM` node is the transpose. Because the offsets are absolute window coordinates rather than relative, drawing a pane is a matter of reading its leaf cell's rectangle, and finding which pane a mouse click landed in is a walk down the tree comparing the click position to each cell's rectangle. `layout_create_cell` builds a node (starting its sizes and offsets at `u32::MAX` as an "unset" sentinel until the layout is computed), and the tree is torn down by `layout_destroy_cell` and `layout_free`.

The one-column-per-border accounting is not a detail you can hand-wave, because it is where a layout's arithmetic either matches tmux or does not. The borders between panes occupy real columns and rows on screen, and the layout engine has to reserve them: an 80-column window split into two side-by-side panes is not two 40-column panes, it is (roughly) a 40 and a 39 with a border column between them. Getting that accounting exactly right — including which pane absorbs the odd column — is what the even-horizontal rounding fix (below, and Chapter 11) is about, and it is why the layout code carries dedicated tests locking the exact split sizes.

The tree also answers the navigation questions that pane-movement commands ask. "What pane is to the right of this one?" is a geometric query over the tree: find panes whose rectangles are adjacent on the relevant edge. "Resize this pane" is a mutation that adjusts a cell's size and propagates the change to its siblings and up to its ancestors. Because the layout is a tree with explicit geometry rather than an implicit tiling rule, all of these operations are concrete pointer-and-arithmetic walks, ported directly from tmux's `layout.c`. The tree *is* the arrangement, and everything you do to your panes is an operation on it.

Preset layouts and manual splits

There are two ways panes get arranged: you split them by hand, or you apply a preset layout, and the engine supports both against the same tree. Manual splitting is `layout_split_pane` (`src/layout.rs`): given a pane and a direction, it takes the pane's leaf cell, turns it into an interior split node, and gives it two children — the original pane and a new one — dividing the space between them. This is what `split-window -h` and `-v` do: grow the tree by one leaf, in place, at the chosen pane. Closing a pane (`layout_close_pane`) does

the reverse, collapsing the split and handing the freed space back to the sibling, and `layout_spread_out` redistributes space when a collapse leaves things lopsided.

The preset layouts — the ones `select-layout` and the `Space` prefix key cycle through — live in `src/layout_set.rs` as a `layout_sets` table mapping names to builder functions. There are five, matching tmux exactly: `even-horizontal` (all panes in one row, equal widths), `even-vertical` (one column, equal heights), `main-horizontal` (one large pane on top, the rest in a row below — plus a `-mirrored` variant that flips top and bottom), `main-vertical` (one large pane on the left, the rest stacked at the right — plus its mirror), and `tiled` (a grid as square as the pane count allows). `layout_set_lookup`, `layout_set_select`, `layout_set_next`, and `layout_set_previous` are the dispatch and cycling functions; applying a preset rebuilds the window's layout tree from scratch to the preset's shape.

The even layouts are where the arithmetic bites, and the code is explicit about it. Distributing N panes across a dimension of size S means each pane gets S/N and there is a remainder of $S \bmod N$ to hand out. The question is *to whom*, and tmux's answer — which ztmux now matches — is that the leftover columns go to the *leading* panes, one at a time, not dumped on the last pane. `layout_spread_cell` computes this, and its comment spells out the case that had to be fixed: for a size-80, two-pane split, each gets 39 and the remainder is 1, so the first pane becomes 40 and the second 39 — `40|39`, matching tmux, not `39|40`. Chapter 11 tells the debugging story; here the point is that a preset layout is not “roughly equal panes,” it is a *specific* distribution that has to match tmux column for column.

Manual splits and presets coexist because applying a preset is just replacing the tree, and manual splitting is just growing it. After you apply `tiled`, you can split a pane by hand, and the layout becomes a custom tree again (its name reported as the serialized string rather than a preset name); apply a preset again and the hand-work is replaced. This is exactly tmux's model, and it falls out naturally from representing every arrangement — preset or hand-built — as the same kind of `layout_cell` tree. There is no separate “preset mode”; there is one tree, built either way.

Resizing, zoom, and the layout string

Resizing a pane is the layout engine's most intricate operation because space is conserved: growing one pane must shrink another, and the change must respect every pane's minimum size and propagate correctly through the tree. `layout_resize_pane` (and `layout_resize_pane_to` for an absolute target) handle a single pane's resize; they walk to find the space to give or take, checking `layout_resize_check` for how much a subtree can yield before hitting minimums, and applying the change through `layout_resize_pane_grow/layout_resize_pane_shrink`. Whole-window resize — triggered when a client attaches at a new size or a terminal is dragged — is `layout_resize`, which scales the tree to the new dimensions, using `layout_resize_adjust` to hand out or reclaim space one column or row at a time in a round-robin so the panes stay proportional and no pane is starved.

That round-robin adjustment is the same leading-cell distribution as the even layouts, and it is tested the same way. `layout_resize_adjust` is the primitive that actually drives an 80-column two-pane window to `40|39` when the window grows into that size, and the layout code carries a test (`spread_via_resize_adjust_yields_40_39`) that drives the real primitive and asserts the first pane ends at 40 and the second at 39. Locking the resize arithmetic with a unit test, in addition to the parity suite's end-to-end check, is belt-and-suspenders on exactly the kind of off-by-one that is easy to reintroduce and invisible until a status-line format prints the wrong width.

Zoom is a lighter-weight operation that does not touch the layout tree at all, and that is by design. Zooming a pane (`resize-pane -Z`) temporarily makes one pane fill the whole window, hiding the others, and un-zooming restores the arrangement. Because this must be perfectly reversible, zoom does not rebuild the tree — it saves the window's layout and swaps in a temporary full-window one, restoring the saved tree on un-zoom (the `window_zoom/window_unzoom` functions in `window.rs`, with the zoom flag on the window). The layout tree is preserved untouched under the zoom, which is why un-zooming lands you back in exactly the arrangement you left, down to the column.

The *layout string* is how an arrangement is serialized — printed by `#{window_layout}`, accepted by `select-layout`, and saved by session-restore tools — and `src/layout_custom.rs` implements it faithfully, checksum and all. `layout_dump` emits a string of the form `"{checksum:04x},{tree}"`, where the tree recursively encodes each cell as `"{sx}x{sy},{xoff},{yoff}"` followed by the pane id for a leaf or a bracketed list of children for a split (`[...]` for top-bottom, `{...}` for left-right). The four-hex-digit checksum is tmux's exact rotate-and-add over the string bytes:

```
csum = (csum >> 1) + ((csum & 1) << 15);
csum += *layout as u16;
```

`layout_parse` verifies that checksum before it will parse a layout string, so a corrupted or hand-mangled layout is rejected rather than misinterpreted. This is why a layout string copied from tmux pastes into ztmux and vice versa: the format, the recursion, and the checksum algorithm are byte-identical.

display-panes and pane navigation

`display-panes` is the small command that overlays each pane with a big number so you can jump to one by pressing its digit, and it is a good lens on how navigation and geometry meet. The overlay draws each pane's index (from the layout geometry — the number is centered in the pane's rectangle) and waits for a key; the digit you press selects the corresponding pane. The command lives in `src/cmd_display_panes.rs`, and it leans on the same tree-to-rectangle mapping the renderer uses: knowing where to paint the “3” is knowing pane 3's cell offset and size. It is a tiny feature, but it shows that “which pane is where” is always answerable from the layout tree.

Directional navigation — `select-pane -L/-R/-U/-D` to move to the pane on a given side — is a geometric query the layout engine answers, not a stored adjacency list. Moving right from the current pane means finding the pane whose rectangle is nearest on the right edge, and the engine computes it from the cells' offsets and sizes. This is why pane navigation feels spatial: it *is* spatial, computed from the on-screen geometry each time, so it does the intuitive thing even in irregular custom layouts that no preset would produce.

The pane-movement commands that restructure the tree — `break-pane` (promote a pane into its own window), `join-pane` (pull a pane from another window into this one), `swap-pane` (exchange two panes' positions), `move-pane` — are all tree surgery, and they route through the layout functions this chapter has named. `break-pane` removes a leaf and rebuilds the source window's layout to close the gap while starting a new window around the removed pane; `join-pane` splits a destination pane and grafts the moved pane's leaf into the new split. Because these operate on the shared tree representation, a pane keeps its screen, its scrollbar, and its running program across the move — only its position in the geometry changes.

The whole layout subsystem, then, is a small pure-geometry engine sitting between the panes (which own content) and the terminal back end (which owns drawing). It knows nothing about what a pane contains or how it is drawn; it knows only rectangles, splits, and the arithmetic of distributing space among them faithfully. That separation is why it can be tested with plain arithmetic assertions and pinned with parity cases independent of any actual terminal — and why the one place it *had* to match tmux exactly, the rounding of leftover space, gets both a unit test and a parity case. With geometry settled, the book turns to the layer that lets a user drive all of this: the command language.

Reading a layout string by hand

Because the layout string is a plain, deterministic serialization, you can read one without any tooling, and doing so once demystifies the whole subsystem. Take an 80×24 window split once vertically into two stacked panes. `#{window_layout}` reports something like:

```
bf3c,80x24,0,0[80x12,0,0,0,80x11,0,13,1]
```

Read it left to right. `bf3c` is the four-hex-digit checksum — the rotate-and-add over the rest of the string that `layout_parse` recomputes and verifies before it will trust the layout. `80x24,0,0` is the root cell: 80 wide, 24

tall, at offset `(0,0)`. The `[...]` brackets mean a top-bottom split (a `LAYOUT_TOPBOTTOM` interior node); `{...}` would mean left-right. Inside are two children: `80x12,0,0,0` is the top pane — 80×12 at `(0,0)`, pane id `0` — and `80x11,0,13,1` is the bottom pane — 80×11 at `(0,13)`, pane id `1`. Notice the arithmetic that Chapter 7 insisted on: the two heights are 12 and 11, not 12 and 12, because the border row between them consumes row 12, and the bottom pane starts at `yoff` 13. Twelve plus one border plus eleven is twenty-four. The layout string is the geometry made legible, and the fact that a checksum guards it is why one copied out of `tmux` pastes into `ztmux` and lays out identically — same recursion, same offsets, same checksum algorithm, byte for byte.

Chapter 8 – The Command Language

tmux is driven by a command language — `new-window`, `bind-key`, `split-window`, and a hundred more — parsed by a real grammar. This chapter covers the command layer: the lalrpop grammar, argument parsing, and the one-command-per-file structure (`cmd_*.rs`) that mirrors tmux's own `cmd-*.c`.

The command grammar (`cmd_parse.lalrpop`) and the parser

Everything a user or a config file tells tmux to do is a command, and commands have real syntax — not just a name and arguments, but sequences separated by semicolons or newlines, brace-delimited blocks that group commands, `if/else` conditionals, the `%if/%elif/%else/%endif` config-file directives, and variable assignments. That syntax deserves a real parser, and ztmux uses one: `src/cmd_parse.lalrpop` is an LALRPOP grammar (LALRPOP is a Rust parser generator) that compiles to a generated `cmd_parse.rs` at build time. The grammar's productions map directly onto the language's structure — `Lines`, `Statements`, `Statement`, `Commands`, `Command`, `Arguments`, `Argument`, the `Condition/Elif` productions for `if/else`, the `Assignment` forms, and the `IfOpen/IfElif/IfElse/IfClose` productions for the `%if` family.

The grammar does not tokenize the raw bytes itself; it sits on top of a hand-written lexer (`crate::cmd_parse::lexer`, producing a `Tok` enum), declared to LALRPOP through its `extern` token block. This split — hand-written lexer, generated parser — is deliberate and it mirrors tmux, whose own command parser (`cmd-parse.y`) is a yacc grammar over a hand-written lexer. The tokens are what you would expect from the language: newline, semicolon, left and right brace, the `%hidden/%if/%elif/%endif/%else` keywords, and the value-carrying tokens `Format` (a `#{...}`-bearing string), `Token` (a bare word or quoted string), and `Equals` (an assignment). A tmux command line is genuinely a small program, and it is parsed like one.

The reason a full grammar is warranted rather than a `split(' ')` becomes obvious the moment you look at a real `.tmux.conf`. Brace blocks let a single `bind-key` map to a sequence of commands; the `if-shell` and `%if` conditionals branch on runtime or build-time conditions; assignments set the environment the following commands run in; and all of this nests. A naive splitter cannot handle a brace block that contains semicolons, or a conditional that spans lines, or a quoted argument with a space in it. The grammar handles all of them because it models the language's actual structure, and it does so the same way tmux does, which is what lets a config file written for tmux parse identically under ztmux.

The conditionals interact with parsing in a way worth calling out, because it is subtle and faithful. A `%if false` block is not merely evaluated to nothing at runtime — the parser, consulting the parse state's scope flag, *discards* the commands inside a false branch as it parses, so they never become command objects at all. This matters for config files that use `%if` to guard version-specific or platform-specific syntax: the guarded commands are only constructed when their condition holds, exactly as tmux does it. The grammar and the scope tracking together make the conditional a parse-time construct, not just a runtime one.

Arguments, flags, and target resolution (`arguments.rs`, `cmd_find.rs`)

Once a command's name and raw argument words are parsed, they have to be interpreted against that command's declared argument specification, and that is `arguments.rs`'s job. Each command declares

what flags it takes, which flags carry values, and how many positional arguments it expects (its `args_parse` spec, part of the command's entry). The argument parser (a faithful port of tmux's `args.c`) walks the words, separates flags from positionals, associates values with the flags that take them, and produces an `args` structure the command's implementation queries with helpers like `args_has`, `args_get`, and `args_count`. This is why `split-window -h -l 30` correctly means “the `-h` flag, and the `-l` flag with value 30, and no positional” — the argument spec drives the interpretation.

Target resolution is the other half of interpreting a command, and it is where tmux commands get their characteristic `-t` targets. Most commands act on a session, window, or pane, named by a target string like `mysession:2.1` (session `mysession`, window 2, pane 1), or `-` for the last, or `{marked}` for the marked pane, or a relative form. `src/cmd_find.rs` resolves these: given a target string and the context (the current client, session, window, and pane), `cmd_find` walks the server's state tree to find the session, window, and pane the target names, applying all of tmux's defaulting and search rules. The result is a `cmd_find_state` — resolved pointers to the actual session, window, and pane — that the command then operates on.

The defaulting rules are more subtle than they look, and porting them faithfully is much of the work. When a command takes a `-t` target but none is given, the target defaults to the current pane of the current window of the current session *of the client that issued the command* — which is why `split-window` with no arguments splits the pane you are in. When a target names a window but not a pane, the pane defaults to the window's active pane. When it names a session but not a window, the window defaults to the session's current window. `cmd_find` encodes all of this, plus the special targets and the search-by-name-or-index logic, so that every command inherits the same target semantics — which is exactly why tmux's targeting feels consistent across a hundred commands: they all resolve targets through one function.

Flags, positionals, and targets together are the command's *interface*, and the faithful port of all three is what makes ztmux's commands drop-in compatible. A script that relies on `-t {last}` or on the defaulting of an omitted target or on a flag taking an optional value behaves identically, because the argument parser and `cmd_find` are tmux's, ported with their C citations and checked by the parity suite's `.sh` cases (Chapter 11), several of which do exactly this — run a command with a target, then query the result — precisely to pin the resolution behavior.

One command per file: the `cmd_*.rs` mirror of tmux's `cmd-*.c`

tmux organizes its commands one per file — `cmd-new-window.c`, `cmd-split-window.c`, `cmd-bind-key.c` — and ztmux mirrors that structure exactly: `src/cmd_new_window.rs`, `src/cmd_split_window.rs`, `src/cmd_bind_key.rs`, and so on, sixty-odd files in all (the tree carries 65 `cmd_*.rs` files). The mapping is mechanical and the C-stem convention is even encoded in the tooling: the annotation script maps `cmd_kill_pane.rs` back to `cmd-kill-pane.c` by replacing underscores with hyphens. This one-to-one structure is not incidental tidiness; it is what makes the port auditable command by command — you can hold `cmd_split_window.rs` next to `cmd-split-window.c` and check the two directly, and the C citation on each function tells you exactly which C lines to compare.

Each command file defines two things: a `cmd_entry` — the command's metadata — and an `exec` function that runs it. The `cmd_entry` struct (`src/lib.rs`) carries the command's `name`, an optional `alias` (so `neww` works as `new-window`), the `args_parse` spec that drives argument parsing, a `usage` string, the source and target specifications, a set of flags, and the `exec` function pointer. The flags (`cmd_flag`) are the command's contract with the server: `CMD_STARTSERVER` means running this command should autostart a server if none exists, `CMD_READONLY` means it is safe in a read-only client, `CMD_AFTERHOOK` governs hook timing, and the `CMD_CLIENT_*` flags govern client-flag handling. These flags are how the command queue knows, generically, how to treat a command without special-casing each one.

All the entries are collected into one table, `CMD_TABLE`, an array of ninety command entries (`&CMD_ATTACH_SESSION_ENTRY` through `&CMD_WAIT_FOR_ENTRY`). Command lookup is `cmd_find(name)` — distinct from the *target-finding* `cmd_find` of the previous section — which scans the table, resolving aliases through `cmd_`

`get_alias`, and returns the matching entry or an error. When you type a command, this is the table consulted to find the code that implements it. Some files hold more than one related entry — `cmd_set_option.rs` carries `set-option`, `set-window-option`, and `set-hook`; `cmd_select_layout.rs` carries `next-layout` and `previous-layout` alongside `select-layout` — mirroring `tmux`, where related commands share a source file.

The `exec` function is where a command finally does its work, and its signature is uniform: it takes the parsed command and the command-queue item it is running under, and returns a `cmd_retval`. Inside, it queries its arguments, resolves its targets through `cmd_find`, and calls into the server's state — creating a window, splitting a pane, setting an option, binding a key. Because every command has the same shape — entry plus `exec`, resolved through the same table, run through the same queue — the command layer is uniform and extensible: adding a command is adding a `cmd_*.rs` file with an entry and an `exec`, and (for the port) checking it against the corresponding `cmd-*.c`. That uniformity is the whole reason `tmux` has a hundred commands that all behave consistently, and `ztmux` inherits it by inheriting the structure.

The target syntax, in detail

The `-t` target string deserves its own reference, because it is the notation every command uses to say *what* to act on, and `cmd_find.rs` resolves all of its forms. The full form is `session:window.pane` — `work:2.1` means session `work`, window index 2, pane 1 — but nearly every part is optional and defaults intelligently. `work:2` names a window (pane defaults to the window's active pane); `work:` names a session (window defaults to its current window); `:2.1` names a window and pane in the current session; a bare `1` in a pane context is a pane index. Beyond the numeric forms there is a vocabulary of special targets:

- `-` — the last (previously current) session, window, or pane, depending on context.
- `{last}`, `{next}`, `{previous}` — relative selectors.
- `{marked}` — the marked pane (`select-pane -m`), the implicit source for `join-pane` and `swap-pane`.
- `+/ -` as window indices — the next/previous window (`:+`, `:-`).
- `:=N` — an *exact* index match (used by the default number-key bindings, `select-window -t:=0`), distinct from a name-or-index search.
- `{top}`, `{bottom}`, `{left}`, `{right}`, `{up-of}`, `{down-of}`, and the directional pane selectors resolved geometrically from the layout tree (Chapter 7).

A target that names a session or window by *name* rather than index is searched (an exact match preferred, then a prefix, then an `fnmatch` pattern), which is why `attach -t wo` can find session `work`. All of this defaulting and searching lives in `cmd_find`, ported with its C citations, so that every one of the ninety commands inherits identical target semantics — which is exactly why `tmux`'s targeting feels consistent across the whole command surface: there is one resolver, and every command routes through it.

The command queue and how commands execute against server state

Commands do not run the instant they are parsed; they are *queued*, and the queue is what serializes every mutation of server state into a single, ordered stream. `src/cmd_queue.rs` implements it: a parsed command list becomes a sequence of `cmdq_items` appended to a queue (`cmdq_append`, `cmdq_insert_after`), and the server's main loop pumps the queue by calling `cmdq_next`, which fires each item's command (`cmdq_fire_command`) or callback (`cmdq_fire_callback`) in turn. This is why order is deterministic and why a `bind-key` that runs three commands runs them in sequence against a consistent state — the queue guarantees one item completes before the next begins, and the server loop drains it (Chapter 3) every iteration.

The queue is not just a list; it carries *state* per item, and that state is what makes commands composable. `cmdq_new_state` builds a `cmdq_state` from a `cmd_find_state` (the resolved target context) and, for key-triggered commands, the key event that triggered it. That state flows to the command's `exec`, so a command

knows the client, session, window, and pane it was invoked against, and knows the key (if any) that fired it. This is how `send-keys` knows which pane to send to, how a bound command knows which client pressed the key, and how hooks know their context. The state travels with the item through the queue rather than being looked up afresh, which keeps a queued sequence coherent even as the world changes around it.

Commands can also *wait*, and the queue models that too, which is essential for the commands that are asynchronous. `if-shell` runs a shell command and branches on its exit status; `run-shell` runs one and optionally captures output; `confirm-before` and `command-prompt` wait for user input; `wait-for` blocks on a named channel. Each of these suspends its queue item until the asynchronous event completes, then resumes the queue. The `cmdq_running` state and the item's continuation machinery are how a queued command can pause without blocking the server's single event loop — the item yields, the loop keeps running, and the item is re-fired when its condition is met. This is the queue's most important property: it serializes state mutation *without* serializing the server, so a slow `if-shell` does not freeze every other client.

The command queue is, in a sense, the server's transaction log: every change to the session tree, every option set, every binding, every window created or pane split passes through it as an ordered item with a resolved context. That centralization is why the server can reason about “is any command still pending?” when deciding whether to exit (Chapter 3), why hooks can fire reliably after the commands that trigger them, and why the whole system stays consistent under concurrent clients. Parsing (the grammar), interpreting (arguments and targets), dispatching (the command table), and executing (the queue) are four clean stages, each a faithful port of its tmux counterpart, and together they are the command language that drives everything the previous chapters built. The next chapter covers the three systems that make that language *configurable*: formats, options, and keys.

A worked example: parsing one `bind-key` with a brace block

Take a single line from a real `.tmux.conf` and watch the four stages act on it:

```
bind-key -T prefix S { split-window -v ; select-layout even-vertical }
```

The lexer (`crate::cmd_parse::lexer`) turns the bytes into tokens: the word `bind-key`, the flag word `-T`, the word `prefix`, the word `S`, a left brace, then the inner tokens, and a right brace. The generated LALRPOP parser reduces those tokens through its productions: the outer `Command` is `bind-key` with arguments `-T prefix S` and a final argument that is itself a `Commands` group — the brace block containing two `Commands` separated by the `;` token. Crucially, the brace block is parsed into a *command list object*, not a string; the semicolon inside it is a command separator, not a character in an argument, which is precisely the case a naive `split(' ')` cannot handle.

Stage three, dispatch: `bind-key`'s entry is found in `CMD_TABLE` (via `cmd_find` on the name, resolving the alias `bind` to the same entry). Stage two, arguments: `arguments.rs` interprets `-T prefix` as the table flag with value `prefix`, `S` as the first positional (the key, parsed by `key_string.rs` into a `key_code`), and the brace block as the command list to bind. Stage four, execution: the queued item fires `bind-key`'s `exec`, which calls `key_bindings_add` to insert a `key_binding` into the `prefix` key table mapping `S` to that two-command list. Press the prefix then `S` later and the *same* command-list object is dropped onto the command queue and run in order — split, then re-layout. One line, four faithful stages, and the binding's value is a parsed program rather than a string, which is why tmux's most elaborate default bindings are possible at all.

Chapter 9 – Formats, Config, and Keys

The glue that makes tmux configurable: the `#{...}` format language, the config file, key tables and bindings, and the options system. This chapter covers how a `.tmux.conf` becomes running behavior.

The `#{...}` format language and its expansions

The most distinctive thing about tmux’s configurability is that it has a small *expression language* embedded in strings — the `#{...}` format syntax that appears in status-line definitions, `display-message` arguments, `if-shell` conditions, and window and pane titles. `src/format.rs` implements it, and it is the single largest module in the tree (over two hundred kilobytes, with more than two hundred C citations), because the format language is genuinely large: it does variable substitution, arithmetic, string operations, conditionals, comparisons, loops over sessions/windows/panes/clients, time formatting, and shell-command interpolation, all inside `#{...}`. A format string is expanded by `format_expand` (with a loop guard, `FORMAT_LOOP_LIMIT = 100`, so a self-referential format cannot recurse forever), and each `#{...}` is handled by `format_replace`.

The power comes from *modifiers* — single characters or operators after a `#{` that transform the expansion — and ztmux ports the full set. The simplest is a bare variable: `#{pane_current_command}` expands to the pane’s foreground command. The modifiers layer transforms on top: `#{b:pane_current_path}` is the basename, `#{d:pane_current_path}` the dirname, `#{t>window_activity}` a formatted time, `#{=10:pane_title}` a width-limited version, `#{s/foo/bar/:...}` a substitution, `#{n:...}` a length, `#{q:...}` a shell-quote. There are comparison and boolean operators (`#{==:a,b}`, `#{&&:...}`, `#{||:...}`), an arithmetic operator (`#{e|+|:2,3}` adds), a conditional (`#{?condition,then,else}`), and loop modifiers (`S`, `W`, `P`, `L`) that expand a sub-format once per session, window, pane, or client. Each is a flag in the `format_modifiers` bitset, parsed in the big modifier `match` and applied during replacement.

The literal modifier `l` is small but instructive, because it is where a real bug lived. `#{l:some text}` means “expand to this literal string, unescaped” — it is how you put characters into a format that would otherwise be interpreted. Its handler calls `format_unescape`, which walks the string turning `#`-escapes back into literal characters. The C original is a `for (; s != end; s++)` loop whose `s++` runs on every iteration, including the ones that `continue`; an earlier version of the Rust port translated it to a `while` loop that forgot to advance `s` on the normal branch, so the loop never progressed — it spun forever, writing past the `xmalloc(strlen + 1)` buffer it had allocated, and crashed the server. Chapter 11 tells how the parity suite caught it; the fix advances `s` explicitly on every path, matching the C’s per-iteration increment. The `format.rs` source carries the whole explanation as a comment above `format_unescape`, citing `vendor/tmux/format.c:4281`, so the trap is documented where the next person will see it.

The format language is also where the multiplexer’s whole state becomes readable as data, and that is worth appreciating. Nearly every field of a session, window, pane, or client is exposed as a format variable — `session_name`, `window_index`, `pane_width`, `client_tty`, `pane_current_command`, and hundreds more — and the format engine resolves them from the live server state at expansion time. This is why the status line can show anything, why `list-panes -F '#{pane_pid}'` can extract a specific field, and why the parity suite (Chapter 11) is *mostly* format cases: a `.fmt` case is a single format string, expanded by `display-message -p` against a fixed session and diffed byte-for-byte against tmux, and because formats read so much of the state, a format that matches is strong evidence the underlying state matches too. The format language is simultaneously tmux’s configuration expression language and its introspection interface, and ztmux ports both roles.

Options: server, session, window, pane scopes

Configuration values live in the *options* system, and its defining feature is that options are *scoped* and *inherited*. `src/options.rs` (with the big default table in `src/options_table.rs`) implements four scopes, defined as bit flags in `src/lib.rs`: `OPTIONS_TABLE_SERVER` (server-wide, like `escape-time`), `OPTIONS_TABLE_SESSION` (per-session, like `status` and `history-limit`), `OPTIONS_TABLE_WINDOW` (per-window, like `window-status-format`), and `OPTIONS_TABLE_PANE` (per-pane, like `allow-rename`). A given option belongs to one or more scopes, and a lookup for, say, a window option checks the window's own options, then falls back to the parent scope's value, then to the built-in default — the inheritance chain that lets you set an option globally and override it per window or per pane.

Each option has a *type*, and the type system (`options_table_type`) is what makes options more than untyped strings. An option is a string, a number, a key, a color, a flag (boolean), a choice (one of an enumerated set), or a command, and some are arrays. The type drives parsing and validation: a number option carries a minimum and maximum and rejects out-of-range values; a choice option carries its list of valid strings; a color option parses `#RRGGBB` and named colors through the same `colour.rs` machinery the grid uses. The default table entry for `history-limit`, for instance, declares it an `OPTIONS_TABLE_NUMBER` in the session scope with a minimum of 0, a maximum of `i32::MAX`, and a default of `2000`, with the help text noting the change applies only to new panes. The whole options table is this kind of declarative entry, ported from tmux's `options-table.c`.

The functions that read options are the ones the rest of the code calls constantly, and they encode the scope-and-inheritance logic. `options_get_only` fetches an option set directly on a given scope; `options_get` walks the inheritance chain to the parent; `options_get_string`, `options_get_number`, and their typed cousins return the value in the form the caller wants; `options_default` produces the built-in default from the table. Setting goes through `options_set_string`, `options_set_number`, and `options_parse` (which parses a value against the option's type). `set-option`, `set-window-option`, and `show-options` are the user-facing commands, and they route through exactly these functions — which is why `set -g` (global) versus `set` (current scope) versus `setw` (window) all behave as tmux documents: they are choosing which scope's options tree to write.

Scope resolution — deciding *which* scope a bare `set-option foo bar` means — is itself a ported function, `options_scope_from_name`, because tmux's rules are specific: an option's name determines its scope (the table says whether `foo` is a server, session, window, or pane option), and the command's flags (`-g`, `-w`, `-p`, `-s`) can further constrain it. Some options live in a combined window-and-pane scope. Getting this right matters because a config file that sets an option in the wrong scope silently does nothing useful, and tmux users have precise expectations here built over years. `options.rs` ports the resolution with its C citation (`vendor/tmux/options.c:863`), so the scope a given `set` lands in is exactly tmux's.

A worked inheritance example

Walk one option lookup to see the inheritance chain in action. Suppose `history-limit` is a session option (it is), and consider a pane in window 2 of session `work`. Nothing has set `history-limit` on that window or session, but the global session tree has it at the default `2000`. When a new pane is spawned and asks “how much scrollback should I keep?”, the lookup is `options_get`, which walks the chain: check the pane's own options — not set; fall back to the window's options — not set; fall back to the session's options — not set; fall back to the *global* session options — found, `2000`; and if even that were absent, `options_default` would produce the built-in table default. The pane gets `2000`.

Now `set -g history-limit 50000` writes the *global* session tree, so every future pane inherits 50,000. But `setw -t work:2 history-limit 100000`... would fail, because `history-limit` is a session option, not a window option — `options_scope_from_name` rejects the wrong-scope set, exactly as tmux does, rather than silently storing a value nothing reads. And `set -t work history-limit 100000` sets it on the `work` session specifically, so panes in `work` inherit 100,000 while panes in other sessions still inherit the global 50,000. The same walk governs every option, which is why “set it globally, override it per session, override that per window or pane” is a single consistent mental model across the whole configuration surface — and

why getting the *order* of the fallback chain byte-identical to tmux matters: a config that relies on a per-session override winning over a global default depends on the chain resolving in exactly this sequence.

Option arrays, user options, and the subscript syntax

Not every option is a single scalar. A handful are *arrays* — indexed collections of values — and they carry some of tmux's most-used configuration: `terminal-overrides` and `terminal-features` (Chapter 10), `command-alias` (the table that lets `set-option` map custom short names to command strings), `update-environment` (the variable list of the next section's environment machinery), and the several `status-format` slots. The table marks these `OPTIONS_TABLE_IS_ARRAY`, and `src/options.rs` stores an array option not as a string but as an `options_array` — a red-black tree of `options_array_items` keyed by integer index, so an array can be sparse and can be addressed one slot at a time. `options_array_get`, `options_array_set`, `options_array_assign`, and `options_array_clear` are the ported operators over it.

The user-facing syntax mirrors that structure exactly, and porting it faithfully is what lets a real `.tmux.conf` full of array manipulation load unchanged. A bare `set-option` on an array replaces the whole thing; `set -a` (append) adds an element after the last, which is why the canonical truecolor incantation `set -ag terminal-overrides " ,xterm-256color:Tc "` adds an override rather than clobbering the existing list. The subscript form `foo[3]` addresses one index — `set -g command-alias[100] 'split-pane=split-window'` writes slot 100 specifically — and `show-options -A` expands an array back into its indexed `foo[0]` ... `foo[n]` lines so the output round-trips into commands that would recreate it. Each of these is a specific behavior a config depends on, and each routes through the same `options_array_*` functions the port carries from `vendor/tmux/options.c`.

User options are the other departure from the scalar table, and they are what makes tmux extensible without a plugin API. Any option whose name begins with `@` — `@my-plugin-state`, `@theme`, `@continuum-restore` — is a user option: it has no entry in `options_table.rs`, it can hold any string, and it is stored and read like any other option through the same scoped tree. `options.rs` special-cases the `@` prefix so that setting or getting an unknown name is not the error it would be for a misspelled built-in but a legitimate write to a user slot. This is the mechanism the entire tmux plugin ecosystem — `tpm`, `tmux-resurrect`, status-line themes — stores its state in, and because ztmux ports the `@`-prefix rule and the scoped storage exactly, those plugins' `set -g @foo ...` lines behave identically: the value is stored on the chosen scope, inherited down the same chain (the worked example above), and read back by whatever `#{@foo}` format or `if-shell` consults it. A superset that broke user options would break the plugin ecosystem it most wants to inherit; keeping them byte-faithful is part of the parity promise.

Key tables and `bind-key (cmd_bind_key.rs)`

Every keystroke you send tmux is looked up in a *key table*, and the key-table system (`src/key_bindings.rs`) is how `bind-key` turns a key into a command. A *key_table* is a named, red-black tree of `key_bindings`, each mapping a `key_code` (a key plus its modifiers, and possibly a repeat flag) to a command list. The default tables are the familiar tmux ones: `prefix` (the table entered after the prefix key, where most bindings live), `root` (bindings that fire without a prefix), and the copy-mode tables (`copy-mode` and `copy-mode-vi`). When you press a key, the server consults the active table — normally `root` first, and `prefix` after the prefix key has been seen — and if the key is bound, runs its command list through the command queue; if not, the key is forwarded to the active pane.

`bind-key (src/cmd_bind_key.rs)` is the command that populates these tables, and its implementation is a thin wrapper over `key_bindings_add`: it parses the target table (from `-T`, defaulting to `prefix`), the key (parsed from its string form by `src/key_string.rs`, which turns `C-b` or `M-Left` into a `key_code`), an optional `-r` repeat flag and `-N` note, and the command list to bind, then adds the binding to the named table. `unbind-key (cmd_unbind_key.rs)` removes one via `key_bindings_remove`, and `list-keys (cmd_list_keys.rs)` walks the tables to print them. Because a binding's value is a full command list — parsed by the

Chapter 8 grammar — a single key can be bound to a brace block of many commands, which is how tmux’s more elaborate default bindings work.

The `key_bindings` module ports the whole table lifecycle, not just add and remove. `key_bindings_init` builds the default tables at server start (Chapter 3); `key_bindings_get_table` finds or creates a table by name; `key_bindings_get/key_bindings_get_default` look up a key’s binding (falling back to a default binding if the user has not overridden it); `key_bindings_first/_next` iterate a table’s bindings for `list-keys`; and `key_bindings_reset` restores a key to its default. The reference counting on tables (`key_bindings_unref_table`) exists because a table can be referenced while it is being modified, and freeing it early would be a use-after-free — the sort of memory-lifetime concern the port handles with the same care the C does, because it is porting the C’s exact ownership model.

The result is that key handling is table-driven and fully faithful: the same `C-b` prefix, the same default bindings, the same `bind-key -T copy-mode-vi` extensions, the same repeat behavior for pane resizing. A user’s `.tmux.conf` full of `bind` and `unbind` commands produces the same key tables under ztmux as under tmux, because the tables, the key parsing, and the lookup are ported line for line. And because a binding’s value is a command list, the key system composes with everything in Chapter 8 — a key is just a queued command with a trigger, which is exactly how the command queue’s per-item state (which carries the triggering key event) is designed to treat it.

Loading config (c f g . r s) and source-file

A config file is just a sequence of commands, and loading one is running them — which is why the config loader (`src/cfg.rs`) is thin: it does not have its own mini-language, it hands the file to the command parser (Chapter 8) and queues the result. `start_cfg` kicks off configuration at server start, and `load_cfg/load_cfg_from_buffer` do the work: read the file (or buffer), parse it with the command grammar, and append the resulting commands to the command queue to run in order against the freshly started server. The `source-file` command (`cmd_source_file.rs`) is the same machinery exposed to the user — it loads and runs another config file at runtime — which is why a `.tmux.conf` can `source-file` a modular set of sub-configs and they compose exactly as if inlined.

Because config loading is command execution, everything in the command language is available in a config file: brace blocks, `%if/%endif` conditionals, `if-shell`, assignments, and of course `set-option` and `bind-key`. This is the unifying insight of tmux’s design that ztmux inherits wholesale — there is no separate “configuration format,” there is only the command language, and a config file is a batch of it. It is why the same line works typed at the command prompt, passed to `tmux` on the command line, sourced from a file, or bound to a key: all four paths parse the same grammar and run through the same queue.

Errors during config loading are collected rather than fatal, and the collection mechanism is itself a nice example of the port’s fidelity. As commands in a config file run, any that produce an error add a *cause* — a message describing what went wrong and where — through `cfg_add_cause`, and after loading, `cfg_print-causes/cfg_show-causes` surface them (a config error shows up as a message you can read, not a silent failure or a crash). This is why a typo in your `.tmux.conf` leaves tmux running with an error message rather than failing to start, and ztmux reproduces the behavior because it ports the cause-collection machinery, not just the happy path.

Config, options, formats, and keys are four faces of one idea: tmux is configured by running commands that set typed, scoped options and populate key tables, with a format language threaded through it all for computed values, and a config file is just a script in that same command language. ztmux ports all four faithfully — the format engine with its modifiers and its hard-won `format_unescape` fix, the scoped options table with its typed defaults, the red-black key tables with their reference counting, and the config loader with its error collection — so that the entire configurability surface of tmux carries over unchanged. The last piece of the faithful core is the layer that turns all of this into actual output on a real terminal: the terminal back end.

Chapter 10 – The Terminal Back End

The other edge of the program: how ztmux drives the outside terminal — terminfo capabilities, the output optimizer, truecolor, and the features it negotiates with the host terminal.

Terminfo and capability negotiation

At the far edge of the server, past the grid and past the layout, is the problem of actually making a real terminal show what tmux computed — and real terminals differ, so tmux (and ztmux) drive them through *terminfo*, the database of terminal capabilities. `src/tty_term.rs` is the terminfo layer, and it reads a terminal's capabilities through the `terminfo-lean` crate (the one non-libc, non-parser dependency doing real work here), building a `tty_term` that answers questions like “how does this terminal move the cursor?” or “does it support truecolor?” The client reads the capability list on its side (`tty_term_read_list`, Chapter 4) and ships it to the server in the identify handshake; the server builds the `tty_term` from it with `tty_term_create`. From then on, every drawing operation asks the `tty_term` for the right escape sequence rather than hard-coding one.

Capabilities are named — `TTYC_CUP` for cursor-position, `TTYC_SMCUP`/`TTYC_RMCUP` for entering and leaving the alternate screen, `TTYC_CLEAR`, `TTYC_SGR0`, and so on — and the back end emits them by name through `tty_term`, which knows each terminal's actual byte sequence. This indirection is why tmux runs correctly across the enormous variety of terminals that have existed: it never assumes a sequence, it looks one up. ztmux ports the capability names, the lookup, and the emission faithfully, so a terminal that works under tmux works under ztmux for the same reason — the capability database says how to talk to it.

Terminfo alone is not enough, because terminal databases are frequently wrong or incomplete, so tmux layers *overrides* on top, and ztmux ports the override machinery. `tty_term_apply` and `tty_term_apply_overrides` apply the `terminal-overrides` and `terminal-features` options — user- or default-supplied corrections that add, remove, or fix capabilities the terminfo entry gets wrong. This is how tmux knows that a terminal advertising a certain `TERM` actually supports truecolor even when its terminfo entry omits it, or how a user forces a capability their terminal has but its database lacks. The override system is a faithful port because it is load-bearing: without it, a large fraction of real terminals would render subtly wrong.

Feature negotiation goes a step further than static terminfo, and it is where the back end talks *back and forth* with the terminal. Some capabilities cannot be known from the database and must be probed at runtime — by sending a query escape sequence and reading the reply. tmux probes for features like truecolor, synchronized output, and the terminal's actual identity during startup, using the Device Attributes and other reports the input parser (Chapter 6) knows how to send and receive. The `tty_features.rs` module tracks the resulting feature set. This runtime negotiation is why tmux increasingly “just knows” what a modern terminal can do without configuration, and porting it keeps ztmux current with that behavior.

The output path and its optimizations

Driving a terminal is bottlenecked by bytes on the wire — an escape sequence to move the cursor and paint a cell is far larger than the cell itself — so the output path is where tmux spends real effort optimizing, and `src/tty.rs` ports that effort. The core idea is *differential redraw*: the server does not repaint the whole screen every frame, it computes what changed since the last draw and emits only the sequences to update those cells. The grid's dirty-tracking (Chapter 5) feeds this: cells the screen-write layer marked changed are the only cells the tty layer redraws, and the `tty_cmd_*` family of functions turns high-level changes (this line scrolled, these cells cleared, this region changed) into minimal terminal sequences.

The output is buffered through libevent, and the buffering is where flow control lives. The tty's output goes into a libevent `evbuffer` (`tty.out`), drained to the client's descriptor by the `tty_write_callback` when the descriptor is writable. When a program floods a pane faster than the terminal can consume — a `yes` loop, a giant `cat` — the output buffer grows, and tmux applies flow control rather than growing it without bound: when the buffered bytes exceed `TTY_BLOCK_START` (computed as $1 + (sx*sy)*8$, a multiple of a screenful), the tty enters a blocked state, stops producing more output, and arms a timer (`TTY_BLOCK_INTERVAL`, 100 milliseconds) to resume. This is what keeps a runaway program in one pane from making the whole terminal unresponsive — the back end throttles it. ztmux ports the exact thresholds because flow-control behavior is observable and the parity discipline extends to it.

The individual output helpers show the optimization mindset at the small scale. `tty_putc`, `tty_puts`, `tty_putn`, and `tty_add` append to the output buffer, and they mirror everything they emit to an optional output log (`tty_create_log`, controlled by the logging machinery) so a developer can capture the exact byte stream tmux sent — a faithful port of tmux's own debugging aid. Cursor movement is optimized: rather than always emitting an absolute cursor-position sequence, `tty_cursor` emits the shortest move from the current position (a single “cursor right” when the target is one cell over, a carriage return when it is column zero), because the current position is tracked and the cheapest sequence is chosen. These micro-optimizations sound minor and they are not — across a full-screen redraw of a busy editor they are the difference between a responsive terminal and a laggy one.

The whole output path is also where the *charset* and *attribute* state from the grid gets turned back into escape sequences, closing the loop the input parser opened. When the back end draws a cell, `tty_attributes` emits the SGR sequences to set the cell's color and attributes (only the changes from the previous cell, again minimizing bytes), and `tty_colours` handles the color-specific part, choosing between the 16-color, 256-color, and truecolor forms based on what the cell holds and what the terminal supports. The ACS charset flag on a cell (Chapter 6) causes the back end to switch the terminal into line-drawing mode via the `TTYC_SMACS`/`TTYC_RMACS` capabilities, mapping the box-drawing cells to the terminal's line glyphs. The grid stores tagged, abstract style; the back end renders it to concrete, minimal, terminal-specific bytes.

Truecolor and terminal features

Color is the feature that most visibly separates a modern terminal from a legacy one, and the back end handles all three color depths a cell can carry (Chapter 5) against whatever the terminal actually supports. If a cell holds a 24-bit RGB color and the terminal supports truecolor (known from terminfo, an override, or runtime probing), the back end emits the `38;2;r;g;b / 48;2;r;g;b` direct-color SGR sequences. If the terminal supports only 256 colors, tmux can be told (or can decide) to approximate, and if only 16, it falls back further. The point is that the *grid* always stores the true color the program requested, and the *back end* renders it as faithfully as the target terminal allows — the fidelity is preserved as far up the pipeline as possible and only degraded at the last step, in the terminal that cannot do better.

Beyond color, the back end negotiates a set of modern terminal features that tmux has grown to support, and `tty_features.rs` is the registry. *Synchronized output* — the private mode that lets an application say “buffer this whole update and show it atomically” — is one; when the host terminal supports it, tmux can wrap a complex redraw in it so the user never sees a half-drawn frame. Others include extended underline styles (the curly, dotted, dashed underlines the grid can store), the overline attribute, bracketed paste, focus reporting, and the various mouse-reporting modes. Each feature is a capability the back end checks before using, so tmux uses a feature exactly when the terminal can handle it and not otherwise — the same conservative discipline as the escape-sequence support surface.

Sixel graphics are the one feature here that is fully implemented in ztmux and worth singling out, because the Cargo manifest calls it out specifically as an exception to its “options exist but aren't wired up yet” caveat. `src/image.rs` and `src/image_sixel.rs` implement sixel image support — the protocol for drawing bitmap graphics in a terminal — and its presence changes an observable behavior noted back in Chapter 6: the Device Attributes reply advertises sixel (`\x1b[?1;2;4c` instead of `\x1b[?1;2c`) when the feature is compiled in. This is the sort of end-to-end consistency the port aims for: a feature is not “supported” until every place

that should reflect it does, including the capability it advertises to the programs running inside panes.

The general shape of feature handling is negotiation, not assumption, and it runs both directions. Outward, tmux advertises to pane programs (through DA replies and terminfo) what *it* supports, so a program inside a pane knows it can send truecolor or sixel. Inward, tmux probes and reads the host terminal's terminfo to know what the *outside* terminal supports, so it knows what it can safely emit. tmux sits in the middle, translating between what pane programs want and what the real terminal can do — which is a large part of why running a program inside tmux can work even when the outside terminal is more limited than the program expects: tmux negotiates the gap. ztmux ports this middle-man negotiation faithfully, feature by feature.

A worked example: one RGB color, three terminals

Follow a single 24-bit color out to three different terminals and the “store true, degrade late” principle becomes concrete. A program prints text in `{e}`-computed RGB `(220, 50, 47)` — a specific red. The grid stores it exactly: the cell is promoted to extended (Chapter 5), its `fg` holding `(220<<16)|(50<<8)|47` or-ed with `COLOUR_FLAG_RGB`. The true color is now in the grid regardless of who is looking.

Client 1 is a modern truecolor terminal. `tty_colours` checks the cell's tag and the terminal's capabilities (terminfo, an override, or runtime probing say truecolor is supported) and emits the direct-color SGR `\x1b[38;2;220;50;47m` — the exact red. Client 2 is a 256-color terminal: no truecolor capability, so tmux maps the RGB to the nearest entry in the 256-color cube and emits `\x1b[38;5;NNNm` — a close approximation, the best that terminal can show. Client 3 is a legacy 16-color terminal: tmux falls back further, to the nearest of the 30–37/90–97 ANSI codes. All three clients can be attached to the *same session* at once (Chapter 4), each rendered to its own capabilities from the one shared grid, because the tty back end is instantiated per client while the grid is singular. The fidelity is preserved as far up the pipeline as possible — the program's exact color lives in the grid untouched — and degraded only at the very last step, in the terminal that cannot do better. A port that stored the *approximated* color in the grid would have thrown away information the truecolor client could have used; storing the true value and degrading per-client at output is the faithful design, and ztmux ports it.

Where the back end meets the client render surface

The terminal back end and the client (Chapter 4) meet at a specific seam, and naming it precisely ties the whole faithful core together. The server owns the grid, computes the differential redraw, and produces the terminal byte stream through the tty layer — but the descriptor those bytes are written to is the *client's* terminal, passed to the server across the socket during the identify handshake as duplicated file descriptors. So the back end in `tty.rs` is driving hardware it does not own: it holds the client's `stdout` fd (dup'd over the socket) and the client's terminfo capabilities, and it draws the server's grid onto the client's screen. The client, meanwhile, does no drawing at all — it lent its terminal and it relays keys. The tty layer is the server-side end of that loan.

This seam is why the same session can render correctly to two different clients at once (Chapter 4's multi-client story). The server maintains one grid, but it can run the tty back end once per attached client, each with that client's own terminfo and output buffer, drawing the same grid to each client's terminal in whatever sequences that specific terminal needs. A truecolor terminal and a 256-color terminal attached to the same session both show the session correctly, each rendered to its own capabilities, because the color and the capability negotiation are per-client on the output side while the grid is shared on the state side. The back end is instantiated per client; the grid is singular.

The seam also explains the resize dance one more time, from the output side. When a client sends `MSG_RESIZE`, the server updates the session's size, the layout re-tiles (Chapter 7), the grids resize and reflow (Chapter 5), and then the back end redraws the new arrangement to that client's terminal. Every step is server-side except the trigger (the client's `SIGWINCH`) and the destination (the client's terminal). The client participates only at the two ends; everything between — the layout arithmetic, the grid reflow, the differential redraw, the capability-correct output — is the server driving the client's borrowed terminal through the tty layer.

This is the architecture in one sentence: the client lends a terminal and relays keys, and the server does everything else, meeting the terminal again only at the tty back end.

With the back end, the faithful core is complete: a socket and an event loop (Chapter 3), a thin client that lends its terminal (Chapter 4), a grid and screen model (Chapter 5), a VT parser feeding it (Chapter 6), a layout tiling the panes (Chapter 7), a command language driving it (Chapter 8), and formats, options, and keys configuring it (Chapter 9), all rendered out through terminfo (Chapter 10). Every one of those is a faithful port of a specific tmux C module, cited line by line and checked against the C. The next two chapters are about *how* that fidelity is proven and defended — the parity suite that measures it and the anti-drift gate that protects it — because in a port, the proof of correctness is as much a part of the work as the code.

A differential redraw, cell by cell

To make the differential-redraw claim concrete, follow what happens when a single cell changes. Suppose the clock in a status line ticks from `:07` to `:08`. The status code re-expands the format and writes the new characters through the screen-write layer, which compares each new cell against what the grid already holds and marks *only the differing cells* dirty — here, the one cell holding the `7/8` digit. When the redraw runs, it does not repaint the status line, let alone the screen; it asks the tty back end to update that one cell. The back end then does its own minimization: `tty_cursor` computes the shortest way to move the terminal's cursor from wherever it currently is to that cell — a bare carriage return if the target is column zero, a single “cursor right” if it is one column over, an absolute `CUP` only if nothing cheaper reaches it — and `tty_attributes` emits SGR only if the cell's color or attributes differ from the last cell drawn. The net wire traffic for a clock tick is a handful of bytes: a short cursor move, maybe an SGR, one digit.

Multiply that discipline across a full-screen editor scrolling one line and the payoff is obvious. A naive terminal would repaint the whole screen — thousands of cells, each with its own positioning and color sequences. tmux emits a scroll region command and repaints only the newly exposed line, and where a real scroll is not possible it repaints only the cells that actually changed. The `tty_cmd_*` family (`tty_cmd_scrollup`, `tty_cmd_clearline`, `tty_cmd_cell`, and the rest) is the vocabulary of these minimal operations, each turning a high-level change into the fewest terminal bytes. This is why a tmux session over a slow SSH link stays responsive: the back end is relentlessly stingy with the wire, and ztmux ports that stinginess exactly because the byte stream it produces is itself observable and, in principle, parity-checkable.

Chapter 11 – The Parity Suite: Correctness as Method

A port’s central claim is fidelity, and a claim of fidelity is worthless unless it is measured. This chapter is about how ztmux measures it: a parity suite that runs the real tmux and ztmux on identical inputs and diffs their output byte-for-byte, treated not as a test folder but as the project’s definition of correct.

Correct means the vendored tmux, byte-for-byte

The first decision the parity suite makes is the most important one, and it is a decision about *what correct means*. For ztmux, correct is not “matches the tmux man page” or “behaves reasonably” or “matches whatever tmux the developer has installed” — correct is “produces the exact same bytes as the specific tmux under `vendor/tmux`,” which is the `next-3.7` development version pinned in the repository. The parity runner (`parity/run_parity.sh`) builds and uses `vendor/tmux/tmux` as its reference, not the system tmux, and its header comment says why in plain terms: the truth being measured against is *that exact tmux*, because version drift causes false diffs — Ubuntu’s tmux 3.4 would report divergences that are really just version differences, not port bugs. Correctness is defined against a committed, versioned reference, so it can never be ambiguous.

Byte-for-byte is the other half of the definition, and it is deliberately unforgiving. For each case, the runner captures the reference tmux’s combined stdout and stderr and ztmux’s combined stdout and stderr, and compares them with `cmp` — a raw byte comparison, not a normalized or fuzzy one. A single differing byte fails the case. This is much stricter than a typical test suite, which asserts on parsed or normalized output, and the strictness is the point: a multiplexer that is “almost” byte-compatible with tmux is one whose status-line format prints a subtly wrong width, or whose layout string has an off-by-one in a checksum, and those are exactly the divergences that break real configs and scripts. Only byte-for-byte identity guarantees the compatibility the superset ambition (Chapter 1) rests on.

The measurement has a three-act history, and telling it straight is part of the honesty. Act one: the suite reached **122 of 122 cases passing**, byte-for-byte identical to the vendored tmux on both Linux and macOS — 100.00 percent green across two platforms, and the strength on which the README first called ztmux a daily driver. Act two, once that first set was exhausted, was to *grow the suite* aggressively — from 122 cases to **1107** — pushing into arithmetic, comparisons and regex, string operations, conditionals, padding, loop modifiers, and multi-command `.sh` scenarios that the first 122 never touched. Against that much larger and more adversarial target the suite did what a good suite is supposed to do: it *broke*, surfacing about twenty deterministic, individually pinned port gaps. Act three was to close them — each of those twenty was traced to its cause in the port and fixed — so the suite now stands at **1107 of 1107 passing (100%)**, byte-for-byte identical to the vendored tmux on both platforms. The parity summary JSON records the count (`total 1107, passed 1107, failed 0`) and the reference (`tmux next-3.7`) with a generation timestamp, so the claim is an artifact, not a boast. The important lesson across the three acts is philosophical: a green 122/122 could mean “the suite is too small to fail,” so the project deliberately expanded the suite until it *did* fail, fixed every failure it found, and then treated each failure as a work item. A parity number that never drops is a parity number that has stopped measuring anything.

Defining correctness this way has a profound effect on how the port is developed, because it makes disagreements *decidable*. When a behavior is in question — does an even split round up or down? does a format modifier trim trailing spaces? — there is no debate and no appeal to intuition: run both binaries on the input

and look at the bytes. The reference tmux is the oracle, and the parity suite is the machine that consults it. This is what “correctness as method” means: correctness is not a property the developer judges, it is a measurement the suite takes, and the developer’s job is to make the measurement pass.

The two case flavors: `.fmt` and `.sh`

The parity cases come in two forms, and the split reflects the two things a multiplexer’s behavior divides into: computing values, and manipulating state. The first and more numerous form is the `.fmt` case — a file containing a single tmux format string. The runner expands it with `display-message -p` against a fresh detached session and compares the two binaries’ expansions. A case named `010_arith_add.fmt`, for instance, contains `{e|+|:2,3}`, and the suite checks that both binaries expand it to `5`. Because the format language reads so much of the server’s state (Chapter 9), a format case is a probe into that state: `{window_panes}` after a split checks the pane count, `{pane_current_command}` checks process tracking, and a matching expansion is evidence the underlying state matches. The format cases are the bulk of the suite for exactly this reason — they are cheap to write and they cover an enormous surface of state and computation through one uniform mechanism.

The second form is the `.sh` case — a shell scenario for behavior that takes more than one command to exercise. The runner exports `$TM` as the binary bound to a private socket, and the script just runs `$TM <command>` lines and prints deterministic output. A case might run `$TM new-window; $TM list-windows -F '#{window_index}'` and check that the window index sequence matches, or split a pane and then query the pane count, or set an option and then read it back. These cases test the *state-manipulating* commands — creating windows, splitting panes, renaming sessions, setting options — by performing an action and then reading the result back through a format, so the assertion is again a byte comparison of output. The two flavors together cover both halves: `.fmt` for the computational and introspective surface, `.sh` for the state-mutating command surface.

The mechanical uniformity of the two flavors is what makes the suite easy to grow, and it is exactly that ease that took the suite from 122 cases to 1107: the `parity/cases/` directory now holds those 1107 `.fmt` and `.sh` files spanning arithmetic, comparisons, string operations, conditionals, padding, loops, and the window/pane/session/option command families. All 1107 report green today; the roughly twenty that the expansion first broke on are documented in Appendix D as *what the growth caught and the port then fixed*, each still guarding its corner as a regression test. Every new behavior the port tackles becomes one or more cases in one of these two forms, and because the forms are so simple — a format string, or a few shell lines — adding coverage is low-friction, which is precisely what you want a correctness suite to be.

The two flavors also map cleanly onto the debugging workflow, because a failing case is a minimal reproduction by construction. A `.fmt` case that fails is a single format string that expands differently — you can paste it into both binaries and see the divergence immediately. A `.sh` case that fails is a short command sequence you can run by hand. There is no large test harness to understand, no fixture to untangle; the case *is* the reproduction. This is why the three bugs the next section describes were each caught and fixed cleanly — the parity case that caught each one was also the minimal input needed to debug it.

Writing a new case

Because the case formats are so simple, adding coverage is a low-friction habit, and the workflow is worth stating so a contributor can extend the suite. To pin a *computed* behavior, write a `.fmt` file — a single format string — named with the next number in its family and a descriptive suffix, e.g. `230_str_upper.fmt` containing the format under test. The runner will expand it under both binaries and diff; if they match, the case is green and the behavior is now pinned against regression. To pin a *state-manipulating* behavior, write a `.sh` file that runs `$TM <cmd>` lines and prints deterministic output — perform an action, then read the result back through a format, so the assertion is a byte comparison of the printed output.

The discipline on a new case is the determinism rule below: no `{host}`, `{version}`, `{pid}`, wall-clock time, or socket path, because those differ between the reference and the port or between runs. A good new

case is one that *fails first* — if a case passes the moment you write it, it is either covering already-correct behavior (fine, as a regression guard) or it is not exercising the corner you meant to. The suite grew from 122 to 1107 precisely by authors writing cases aimed at suspected-weak corners and keeping the ones that failed as pinned gaps (Appendix D). The case is cheap, the reproduction is the case itself, and the byte comparison is the oracle — which is what makes growing the suite the natural way to make a claim about ztmux checkable.

Determinism: fixed geometry, `LC_ALL=C`, no host/time/pid

Byte-for-byte comparison only works if the output is *deterministic*, and a multiplexer’s output is full of things that are not — the hostname, the time, process ids, socket paths, the terminal size. The parity runner controls every one of these, and the controls are as much a part of the methodology as the comparison itself. Geometry is pinned: every server is started at a fixed 80x24 size, so any format that reports a width or height (`{window_width}`, `{pane_height}`) is stable across runs and across the two binaries. Without fixed geometry, a window-width format would depend on the terminal the suite happened to run in, and the comparison would be meaningless.

The locale is pinned to `LC_ALL=C`, because locale affects string collation, number formatting, and character handling, and a suite run under a different locale on a different machine would diverge for reasons that have nothing to do with the port. Each case runs against an isolated server per binary — a unique socket label (`-L <uniq>`), no config file (`-f /dev/null`), and a `timeout` wrapping the run so a hang fails the case rather than wedging the suite — and the server is killed after the case. The isolation means cases cannot interfere with each other or with any real tmux or ztmux the developer is running, and the empty config means a case tests default behavior, not the developer’s `.tmux.conf`.

The non-deterministic *content* is handled by a rule rather than a mechanism: cases must avoid host-, time-, version-, and pid-dependent output. A case cannot use `{host}`, `{version}`, `{pid}`, wall-clock times, or the socket path, because those differ between the reference and the port (the port’s version string and socket prefix are deliberately its own) or between runs. This is a discipline on case authors, documented in the runner and the roadmap, and it is the right place to put the constraint: rather than trying to normalize non-deterministic output after the fact (which would weaken the byte-for-byte comparison), the suite simply excludes non-deterministic *inputs*, keeping the comparison exact.

Determinism is what lets the suite run in CI across platforms and mean the same thing every time, and it is why the parity claim can be qualified with “on both Linux and macOS.” The same fixed geometry, the same locale, the same isolation produce the same reference bytes on both platforms, so a case that passes on Linux and fails on macOS is a real platform-specific port bug — which is exactly how the third bug in the next section was found. The determinism controls turn the parity suite from “a comparison that usually works” into “a comparison that is reproducible and portable,” and that reproducibility is what makes it usable as the definition of correct rather than just a smoke test.

Three bugs the suite caught

The parity suite’s value is best shown by what it caught, and ztmux’s history has three clean examples, each a real divergence the byte comparison exposed and each fixed against the C. The first was a *crash*: the `{l:...}` literal format operator brought the ztmux server down. The parity case was a simple `l:` format string that tmux expanded fine and ztmux died on. Root-causing it led to `format_unescape` (Chapter 9): the C loop’s `s++` runs on every iteration including `continue`, and the Rust port had translated it to a `while` that failed to advance on the normal branch, so it spun forever writing past its `xmalloc(strlen + 1)` buffer until it crashed. The fix advances the pointer on every path, and the function now carries a comment explaining the trap so it cannot be reintroduced silently.

The second was an *arithmetic* divergence: the `405_select_layout` case exposed the even-horizontal layout rounding an 80-column two-pane split to `39|40` where tmux produces `40|39`. The port had carried an older algorithm that dumped the leftover column on the *last* pane; current tmux distributes leftover columns to the

leading panes, one at a time. Re-ported `layout_spread_cell` against the current C fixed it (Chapter 7), and the fix was locked with two unit tests asserting `spread_sizes(80, 2) == [40, 39]` in addition to the parity case — because an off-by-one in space distribution is invisible until a format prints the wrong width, and worth pinning twice. This one is a perfect illustration of why byte-for-byte matters: `39|40` versus `40|39` is a one-column difference no reasonable-behavior test would flag, and it would silently break any config that depends on exact pane widths.

The third was a *platform-specific* bug, and it is the one that shows the cross-platform determinism paying off. The `294_pane_cmd_fmt` case checks `{pane_current_command}`, and on macOS it reported `ztmux` instead of the actual child command (`sleep`), while passing on Linux. The root cause was in the pane fork path (Chapter 3's `spawn`): the child, between `fork` and `exec`, called `closefrom` to close inherited descriptors, and the port's `closefrom` looped `close()` from the low fd up to `RLIMIT_NOFILE` — which the server raises to a huge value — so the child spun through millions of `close()` calls and hung before it could `exec`, leaving `pane_current_command` reporting the server binary that had not yet been replaced. The fix ports tmux's macOS-specific `closefrom` variant, which enumerates the process's actually-open descriptors with `proc_pidinfo(PROC_PIDLISTFDS)` and closes only those, so the child reaches `exec` immediately. The bug was only observable on macOS, and only the cross-platform parity run surfaced it.

Three bugs, three categories — a crash from a dropped pointer increment, an arithmetic off-by-one in space distribution, and a platform-specific hang between fork and exec — and all three were caught by the same mechanism: run both binaries on the same input, diff the bytes, investigate the difference against the C. None of them would have been caught by a test suite that asserted “reasonable” behavior, because each produced output that looked plausible in isolation; they were only visible as *divergences from tmux*. That is the argument for correctness-as-parity in one paragraph: the bugs that matter most in a port are the ones where the port does something reasonable that is nonetheless not what the original does, and only a byte-for-byte comparison against the original finds them. The parity suite is not one test folder among many — it is the operational definition of what it means for ztmux to be tmux.

The expansion to 1107 and closing the last gaps

Reaching 122/122 posed a question the project answered the honest way. A suite that is 100 percent green is either a perfect port or too small to catch the imperfections, and there is no way to tell which from the green bar alone. So rather than bank the number, the project *expanded the suite* until it started failing — from 122 cases to 1107 — deliberately pushing into corners the first set never touched: bitwise arithmetic (`{e|&|:...}`, `{e|\|:...}`), float formatting and precision (`{e|/|f2:...}`), regex substitution with backreferences (`{s/.../\1/:...}`), the logical-not operator (`{!:...}`), left and right numeric padding (`{p-N:...}`), the loop modifiers `{S:}/{W:}/{P:}` with their injected `loop_index/loop_last_flag`, named paste buffers, and layout pinning by `main-pane-width`. The result was **twenty failing cases** — and those twenty were the point: they are what the expansion was *for*.

Every one of the twenty was deterministic — identical across repeated runs, with no host/time/pid dependence — and every one was pinned to the exact case number that exposed it, so each failure was a work item with a reproduction attached rather than a rumor. They clustered into a handful of small, nameable gaps (Appendix D lists each with its case numbers and its fix): `{!:...}` printed empty where tmux prints `1`; `{s/.../.../}` did not substitute `\1\2` backreferences; `{p-N:...}` left-padding padded to `N` *plus* the content length instead of a field width of `N`; `{pane_at_top}/{pane_at_bottom}` formatted as Rust `true/false` (`format!("{}", flag)`) instead of the C's `%d` → `1/0`; the `S/W/P` loop modifiers didn't inject `loop_index/loop_last_flag` (real tmux vars — see `vendor/tmux/format.c:4776`); named buffers (`set-buffer -b <name>`) reported “empty buffer name”; and `main-vertical` ignored `main-pane-width`. Each was a bounded, mechanical fix against the C, not an architectural hole — exactly the profile a good expansion produces: it converted “we don't know what's wrong” into a short, specific list. And then the project worked the list: every one of the twenty was traced to its C counterpart and fixed, and the suite now stands at **1107 of 1107 passing (100%)**.

The policy on those cases is worth stating because it is unusual and it survives their being fixed: **cases stay in the suite forever, never deleted**. A parity case is the most valuable artifact the project can hold — a

byte-exact statement of tmux's behavior — and the twenty that once failed are now the twenty best regression guards in the suite, standing red-turned-green over exactly the corners that once diverged. The roadmap file ([parity/Parity_Roadmap.md](#)) records the arc; the runner reports the count; and the number [1107/1107](#) in [parity/parity_summary.json](#) ([total 1107, passed 1107, failed 0](#)) is reported as-is. This is the difference between a suite that measures a port and a suite that flatters one: the 100% is credible precisely because the suite was allowed — was *made* — to fail first.

Chapter 12 — The Anti-Drift Gate

The parity suite proves behavioral fidelity. A second gate proves *structural* fidelity — that the port has not quietly grown functions tmux does not have, inflating its apparent completeness. This chapter covers the anti-drift machinery: the fake-fn test, the C-function index it checks against, the allowlist that documents the exceptions, and the annotation script that keeps two thousand C citations honest.

The fake-fn problem: inflating completeness

There is a specific way a port can lie about its progress, and it is subtle enough to be worth naming precisely. A port's completeness is naturally measured by how many of the original's functions it has implemented — and that metric is gameable. If you invent helper functions in the port that have no counterpart in the original — a `parse_the_thing` here, a `handle_special_case` there, convenience wrappers and Rust-idiomatic splits — the port *looks* larger and more complete than it is, because it has more functions, even though many of them correspond to nothing in the source of truth. The project calls these “fake fns,” and its documentation is blunt about the risk: a Rust function whose name has no C counterpart is an invented helper that inflates the port's apparent completeness.

This is not a hypothetical concern; it is the natural drift of any porting effort. Writing idiomatic Rust *wants* to introduce helpers — extract a closure, factor a common pattern, split a big C function into smaller typed pieces — and every one of those, however reasonable, moves the port away from a one-to-one correspondence with the C and toward a codebase that is “inspired by” tmux rather than a faithful transliteration of it. Once that correspondence is lost, the C citations stop meaning anything (there is no C function to cite), the parity comparison can't localize a divergence to a function, and the port report's completeness number becomes fiction. The whole auditability apparatus of Chapter 2 depends on the port's functions mapping to the C's functions, and fake fns break the mapping.

ztmux's answer is to make the fake-fn problem a *build failure* rather than a matter of discipline. The rule is simple and absolute: every module-level free function in the Rust source must correspond to a real function in the vendored tmux C source. If a Rust function's name has no C counterpart, the build fails — unless the function is explicitly listed in an allowlist that documents why it is a legitimate exception. This turns “don't invent functions” from a guideline a reviewer might miss into a gate that CI enforces on every push, and it makes the anti-drift discipline mechanical, the same way the parity suite makes the fidelity discipline mechanical.

The reason this deserves its own gate, separate from parity, is that it catches a different failure mode. Parity catches *behavioral* drift — code that does the wrong thing. The anti-drift gate catches *structural* drift — code that is organized wrong, that has wandered from the C's decomposition even while (perhaps) behaving correctly. A port can pass parity while failing anti-drift (it produces the right bytes but through an invented structure), and such a port is a maintenance and auditability trap even though it “works.” Both gates are necessary because faithfulness is both behavioral and structural, and a port that is faithful in only one sense is not the thing ztmux is trying to be.

ported_fn_names_match_c.rs and the C index

The gate is a single test, `tests/ported_fn_names_match_c.rs`, and it runs in CI like any other test — but what it tests is the whole source tree's structural honesty. It works in two halves: collect every free function name in the Rust source, build an index of every function name in the vendored C source, and check that

each Rust name is in the C index or the allowlist. The Rust half (`collect_free_fns`) parses the source character by character, tracking brace depth so it counts only functions at depth zero — free functions, not methods inside `impl` or `trait` blocks, since those correspond to C’s struct-of-function- pointers idiom rather than to free C functions. It correctly skips braces inside comments, strings (including raw strings), and char literals, strips the `pub/unsafe/async/ extern/const` qualifiers, normalizes raw identifiers (`r#loop` to `loop`), and skips `mod tests` blocks wholesale. This is a small hand-written parser, and it is careful because a sloppy one would produce false violations.

The C half (`c_fn_names`) builds the index heuristically but robustly, by scanning `vendor/tmux/*.c`, `vendor/tmux/compat/*.c`, and the `cmd-parse.y` grammar for line-initial identifiers immediately followed by `(` with an opening brace on that line or within a few lines — the shape of a C function *definition*, as opposed to a `;`-terminated *prototype*, which it excludes. It excludes C keywords, and it asserts the resulting index has more than 500 names as a sanity check that the vendored tmux is actually present (a missing `vendor/tmux/` would otherwise make every Rust function look fake). The heuristic does not need to be a full C parser because it only needs the set of function *names*, and names are cheap to extract reliably.

The comparison itself is strict in a way worth noting: there is *no* trailing-underscore leniency. A Rust function named `foo_` is not automatically accepted as an implementation of C’s `foo` — it is treated as a deviation and must be explicitly allowlisted. This matters because the trailing underscore is exactly the convention the port uses when it has to *split* a C function or wrap a macro (below), and those splits are precisely the deviations the project wants to track rather than wave through. By refusing to auto-accept `foo_` as `foo`, the gate forces every such split to be a documented, listed exception, so the count of deviations is always visible and always shrinking-toward-zero as a goal.

When the gate fails, it fails helpfully. It prints one greppable line per violation — `FAKE-FN <name> (<file>:<line>) - no tmux C counterpart` — and panics, so a developer who introduced an unlisted function sees exactly which functions are the problem and where. And the fix is either to rename the function to match its C counterpart (if one exists and the name simply drifted) or, if the function is a legitimate exception, to add it to the allowlist with the understanding that the allowlist is an audit trail, not a dumping ground. The gate does not just say “no”; it points at the exact functions and leaves the developer a clear, honest path forward.

A worked example: what the gate catches

To see the gate working, imagine a well-meaning refactor. A contributor porting `input_csi_dispatch` finds it long and, in good Rust taste, extracts a helper: `fn parse_csi_params(...)`. The code compiles, the parity cases still pass (the behavior is unchanged), and everything looks fine — except that `parse_csi_params` is a function that exists in `ztmux` and *nowhere in tmux*. On the next push, `cargo test --test ported_fn_names_match_c` collects `parse_csi_params`, fails to find it in the C index or the allowlist, and prints:

```
FAKE-FN parse_csi_params (src/input.rs:1423) - no tmux C counterpart
```

and panics, failing the build. The contributor now has a clear, honest choice. If the logic belongs inside `input_csi_dispatch` (it does — the C keeps it there), inline it back and the gate goes green. If a genuine, unavoidable helper is truly needed, add it to the allowlist *with the understanding that the allowlist is an audit trail to shrink, not a dumping ground*. Either way the drift is surfaced at the moment it is introduced, not discovered months later when the C citations have quietly stopped meaning anything. This is the whole value of making anti-drift a build failure rather than a review guideline: the reviewer who would have caught `parse_csi_params` by eye might miss it, but CI never does, and the port’s one-to-one correspondence with the C is protected on every push. The example is invented, but the mechanism is exactly what runs, and it is why the port’s function set has not silently drifted into a codebase merely “inspired by” tmux.

The allowlist as an audit trail, not a free pass

No real port can achieve a *perfect* one-to-one function correspondence, and pretending otherwise would make the gate useless — so there is an escape hatch, `tests/data/ fake_fn_allowlist.txt`, but its framing is

what makes it work. The file's own header calls it “a frozen snapshot of pre-existing, accepted exceptions” and “an AUDIT TRAIL, not a free pass,” with the explicit goal to *shrink* it, never grow it. It holds roughly three hundred entries, and — crucially — they are not arbitrary; they fall into a small number of documented, legitimate categories, so that a reader can see not just *that* a function is excepted but *why* its exception is reasonable.

The categories are specific. First, *macro-paired helpers*: C's variadic functions (a `format_add(fmt, ...)`) are ported as a Rust `macro_rules!` carrying the real C name plus a `_`-suffixed helper the macro forwards to (`format_add_`), because Rust macros and functions occupy different namespaces — the macro can keep the C name, so the helper takes the underscore. Second, *C-function splits*: one C function (like `xstrdup`) ported as several typed Rust overloads (`xstrdup_`, `xstrdup__`, `xstrdup___`) to be merged back later. Third, *libc and libevent shims*: wrappers around library functions that have no `vendor/tmux` definition at all (`evtimer_*`, `evbuffer_*`, `fnmatch`, `basename`), which are real functions, just not tmux's. Fourth, *genuine Rust glue*: the handful of truly Rust-only helpers (`cstr_to_str`, `assume_init_ref`) that exist because Rust needs them and C did not.

A large share of the entries are the BSD intrusive-data-structure macros that tmux uses throughout — the red-black tree family (`rb_insert`, `rb_remove`, `rb_foreach`, and the rest), the tail-queue family (`tailq_foreach`, `tailq_insert_tail`), the list family — which are C *macros* in tmux (so they have no function definition to match) but are ported as Rust functions. These are the data structures Chapters 3, 5, 7, and 9 all rely on (`WINDOWS` is an `RB` tree, `CLIENTS` a `TAILQ`, key tables are `RB` trees), and porting the macros as functions is unavoidable — Rust has no equivalent of C's token-pasting intrusive-structure macros. Listing them documents exactly which “functions” are really ported macros, keeping the fake-fn count honest without pretending they are tmux functions.

The regeneration story reinforces that the allowlist is meant to be curated, not auto-grown. The file can be regenerated from the gate's own **FAKE-FN** output — run the test, pipe the **FAKE-FN** lines through **sort -u** into the allowlist — but doing so blindly would also require manually re-adding the header, and the header is the whole point: it is the statement that these are accepted exceptions to be shrunk over time. The intended workflow is not “regenerate to make the gate pass,” it is “understand each new exception, decide if it is legitimate, and if so add it deliberately.” The allowlist is version-controlled, so its growth or shrinkage over time is itself a visible signal of whether the port is drifting toward or away from clean C correspondence.

annotate_c_links.py and the C-signature citations

The anti-drift gate checks function *names*; a companion script checks function *signatures*, and together they close the structural-fidelity loop. `scripts/ annotate_c_links.py` walks the Rust source and, for every function whose name matches a tmux C function, writes (or refreshes) the doc-comment citation this book has quoted throughout — the `/// c \vendor/tmux/c.:` “ line. The citation includes the C function's *full signature*, return type and parameter types and parameter *names*, not just its name — and that is deliberate. A reviewer diffing the Rust parameters against the cited C parameters can catch a renamed, reordered, dropped, or invented argument: a “fake param,” the argument-level analog of a fake fn. There are more than two thousand of these citations across the tree, one per ported function.

The script is careful about which C definition it cites, because a name can appear in more than one C file. It prefers the C file whose stem matches the Rust file's stem — `grid.rs` cites `grid.c`, `cmd_kill_pane.rs` cites `cmd-kill-pane.c` (mapping underscores to hyphens) — so a function is cited from its natural home. Failing a stem match, if the name is defined in exactly one C file, it cites that; if the name is ambiguous across several files with no stem match, it *skips* rather than guessing, on the principle that no citation is better than a wrong one. This conservative citation policy is why the citations can be trusted: a citation that exists is one the script was confident about.

The script is idempotent and self-updating, which is what keeps two thousand citations from rotting. It recognizes an existing citation directly above a function and *replaces* it in place rather than duplicating it, so re-running the script after a vendored-tmux refresh updates every citation's line number and signature to match the new C — the citations track the C automatically rather than being hand-maintained. A `--check` mode writes nothing

and exits non-zero if any citation is missing or outdated, printing how many functions need attention, so the citation freshness can itself be a CI gate: the build can require that every ported function's citation is present and current.

Between them, the three tools form a structural-fidelity system that parallels the behavioral-fidelity system of Chapter 11. The name gate (`ported_fn_names_match_c.rs`) ensures every function corresponds to something real in the C. The signature citations (`annotate_c_links.py`) ensure every function's interface matches the C's, and stay current as the C moves. The allowlist documents and bounds the exceptions, with a stated goal of shrinking. And all three are derived from the vendored C rather than from hand-maintained lists, so none can drift out of sync with the source of truth. This is what lets ztmux claim to be a *faithful* port and not merely a *working* one: fidelity is measured behaviorally by parity and enforced structurally by the anti-drift gate, on every push, against a committed reference. With correctness both proven and defended, the book can finally turn to the forward-looking question — what a port this faithful is *for*, beyond reproducing tmux.

Chapter 13 – Beyond tmux: The Superset

The chapter that earns the word *superset*. ztmux’s goal is not to stop at parity: a memory-safe, own-able core is a platform for features the C tmux does not have. With the core now provably identical to tmux — 1107/1107 byte-for-byte — that platform is open, and the first extensions have shipped: structured (JSON/CSV/table) output for the `list-*` commands, a live `ztmux dashboard`, and an interactive `ztmux switch` picker, all living in a dedicated `src/extensions/` tree that the anti-drift gate leaves free. Those three were the beachhead; the tree has since grown to over a hundred named `ztmux <name>` subcommands and a set of interactive surfaces catalogued later in this chapter. This chapter is honest about the line between shipped and planned — it marks what exists today and what is roadmap — and lays out where a Rust-owned multiplexer can go that a frozen C program cannot.

Parity first: what “faithful” commits the superset to pre-serve

The superset ambition has an order of operations, and it is not negotiable: parity comes first, extensions come second, and the second must never break the first. A superset is a *strict* extension — it does everything the original does, exactly, and then more — so every feature ztmux adds is constrained by a promise it made in Chapter 1: the config files, the keybindings, the format strings, and the muscle memory of every tmux user must keep working unchanged. That promise is what the parity suite (Chapter 11) and the anti-drift gate (Chapter 12) exist to protect, and it is why they had to come before this chapter. You cannot credibly offer “tmux plus extras” until the “tmux” part is provably identical.

This ordering is also a design constraint on the extensions themselves, not just a sequencing of work. A feature that changed the meaning of an existing format variable, or altered a default, or reinterpreted a command’s arguments, would break parity even if it added capability — and so it is disallowed by construction. The extensions ztmux can add are the ones that are *additive*: new format variables that don’t collide with existing ones, new commands that don’t shadow existing ones, new options that default to the existing behavior, integrations that are invisible unless invoked. The superset grows the surface without moving any of the existing surface, which is a stricter discipline than “add features” and the only one compatible with the parity promise.

The commitment cuts deeper than the user-visible surface, too. Because the port keeps its C correspondence (Chapter 12), a superset feature ideally slots in *alongside* the ported functions rather than tangling into them — an added capability should be legible as an addition, not a modification of a cited-from-C function, so that the port’s faithfulness stays auditable even as it grows. This is the same principle the divergence diff (Chapter 2) enforces: keep the ported core legible and keep your own additions distinguishable from it. A superset built this way can always answer “what have you changed about tmux?” with “nothing — here is what I’ve *added*.”

The reward for accepting this discipline is that ztmux’s extensions inherit tmux’s entire ecosystem for free. Every plugin, every dotfile, every tutorial, every piece of tmux muscle memory continues to work, and the extensions are pure upside on top. This is the opposite of a fork, which asks users to give up the ecosystem in exchange for new features; a faithful superset asks them to give up nothing. Parity-first is not a constraint that limits the superset — it is the strategy that makes the superset worth using, because it means adopting ztmux costs a tmux user exactly nothing and the new capabilities are all gain.

The extensions tree: `ported/` and `extensions/`

The discipline above is enforced by the source layout, not just by intent. As the port matured, `src/` split into two trees. `src/ported/` holds the C-faithful port — every file mirrors a `vendor/tmux/*.c` module, every free function is held to its C name by the anti-drift gate (Chapter 12). `src/extensions/` holds original ztmux features that have *no* tmux counterpart, and it is deliberately **exempt** from the gate: the check skips any path containing `src/extensions/`, and `extensions/mod.rs` says so in as many words — “features with no upstream tmux counterpart... deliberately NOT a port of tmux.” That exemption is what lets extensions exist at all without tripping the fake-fn rule, and the split is what keeps the exemption honest: a function is either in `ported/` (and must cite a C origin) or in `extensions/` (and must be a genuine addition, off to the side). There is no third place to hide an invented helper. The extension modules are also written in a self-denying style — their logic lives in inherent methods and associated functions rather than free `fns` — so even inside the exempt tree they add no module-level names a reviewer must vet.

Shipped: structured output for the `list-*` commands

tmux’s `list-sessions`, `list-windows`, `list-panes`, `list-clients`, and `list-buffers` print one formatted line per item, shaped by a `-F` format string. That is perfect for a status bar and awkward for a script, which must then parse the text back apart. ztmux’s first shipped extension (`src/extensions/structured.rs`) adds a `-o <format>` flag to those commands that emits the rows as *data* instead: `json`, `jsonl` (a.k.a. `ndjson`), `csv`, `tsv`, and the human-facing `table`. The formats are parsed by `OutputFormat::parse`; an unrecognized name is a reported error, and no `-o` at all leaves the classic text output untouched — additive, by construction.

The mechanism reuses the ported core rather than duplicating it. Each list command owns a static schema — a slice of `Field { key, fmt, kind }`, where `key` is the column/JSON name, `fmt` is an ordinary `#{...}` format string, and `kind` is `Str`, `Int`, or `Bool`. So the *same* format engine (Chapter 8) that produces the text line produces the structured value; every format variable is available, and any `-f` filter still applies. A `Structured` accumulator expands each field against the item’s `format_tree`, coercing per `kind` (an unparseable-but-empty integer becomes JSON `null` rather than silently dropping data), then `render()` dispatches on the format: `render_json` (array or line-delimited), `render_sv` (with `,` or `\t`, quoting per RFC 4180 when a field contains the separator, a quote, or a newline), and `render_table` (measure each column’s width, left-align, pad with spaces, trim the trailing pad, and underline the header). The same rows, five renderings:

```
$ ztmux list-sessions -o table
NAME      WINDOWS  ATTACHED
work       3         1
scratch   1         0

$ ztmux list-sessions -o json
[{"name":"work","windows":3,"attached":true},
 {"name":"scratch","windows":1,"attached":false}]

$ ztmux list-sessions -o csv
name,windows,attached
work,3,true
scratch,1,false
```

This is the completion of the “json / csv / ...” family with the aligned, human-readable `table` view — and, not incidentally, the machine-readable side is what the two TUI extensions below consume to read the server without linking against its internals.

Shipped: ztmux dashboard

`ztmux dashboard` (`src/extensions/dashboard.rs`) is a live, full-screen overview of the running server — sessions, windows, and panes at a glance, refreshing in place — built on `ratatui` 0.29. It is a *client* subcommand, not a server command: it does not reach into server memory but polls the server exactly the way any other client would, by re-invoking its own binary (`ztmux -S <socket> list-* -o json`) through the shared query layer and deserializing the JSON with `serde`. `ratatui::init()` puts the terminal into the alternate screen and raw mode, the draw loop renders the session/window/pane tree with key handling for navigation and quit, and `ratatui::restore()` guarantees the terminal is put back on the way out. It is the sort of thing `tmux` has never had — a birds-eye console for a server you are not attached to — and it is cheap precisely because the structured-output extension already exposes the state as clean JSON.

Shipped: `ztmux switch` — the interactive picker

`ztmux switch` (`src/extensions/switch.rs`) is an interactive fuzzy picker to jump to any session, window, or pane. It polls the same JSON snapshot, flattens it into a flat list of *targets* — every session, every window, every pane, each with a display label — and then you type to filter (a case-insensitive subsequence match, so `wrk` finds `work`), move with the arrow keys, and press Enter. On Enter it runs the ordered commands that select the target against the chosen socket: a session runs `switch-client -t <session>`; a window runs `switch-client` then `select-window -t <session>:<index>`; a pane runs `switch-client`, `select-window`, then `select-pane -t <pane-id>`. Esc (or Ctrl-C) cancels. It is the picker `fzf`-driven `tmux` users hand-roll with shell glue and a dozen `tmux` calls — here it is one subcommand, and the target-building and fuzzy-matching logic ships with unit tests that assert a pane target selects session → window → pane in order and that `nvim` matches only the pane running it.

The two TUIs share a small read-only module, `src/extensions/tmux_query.rs`, which is the whole reason they stay clean: it defines the `Session/Window/Pane/Client` structs, the `ztmux_cmd` helper that builds a `ztmux -S <socket> ...` invocation of the current binary, and `poll()` which fetches the full tree via the `-o json` list commands. The extensions read the server the way a script would — through the public, structured interface the first extension added — never through a private back door. That is the additive discipline made literal: a new capability built entirely on top of the ported surface, touching none of it.

Shipped: the extension catalog and the interactive surfaces

The three extensions above are the ones this chapter dissects, but they are no longer the whole of what has shipped. `src/extensions/` now carries over a hundred named `ztmux <name>` subcommands, listed once in an `EXTENSION_COMMANDS` table that is the single source of truth for the CLI, the `zsh` completion, and the man page. Most are read-only *inspections* — pipeable, `-o json`-capable views over the live server built on the same structured `list-* -o json` output the dashboard consumes: process tables (`ps`, `pstree`, `mem`), geometry (`size`, `density`, `layouts`), repository state (`git`, `cwd`, `changes`, `worktree`), network (`ssh`, `net`, `ports`), and client and configuration queries. A smaller set are *actions* (`prune`, `equalize`, `retitle`, `bcast`, `pick`). Every one is additive and gate-exempt by the same rule the three flagship extensions follow — there is no third kind.

Four larger systems reach past one-shot subcommands, and each is opt-in and invisible until invoked. `ztmux triggers` revives `tmux`'s removed `monitor-content` as a general sense → act loop: a rule in `~/.ztmux/triggers.json` runs any `ztmux` command when a regex matches a pane's output, armed with `ztmux triggers arm` and authorable through an inline wizard. `ztmux resurrect save / restore` persist the whole server — every session, window, and pane with its layout, `cwd`, and command — to `~/.ztmux/resurrect/` and recreate it across restarts, never clobbering a live session; `set -g @ztmux-resurrect-auto on` adds continuum-style automatic re-saving every fifteen minutes from a pidfile-guarded daemon. `ztmux open` scans the focused pane for URLs and file paths and offers a picker that opens the selection (a URL through `open/xdg-open`, a file in `$EDITOR` at its `file:line`) or copies it to the `tmux` buffer

and OS clipboard. And an optional **ratatui UI layer** — on by default, disableable with `@ztmux-ratatui off` to fall straight back to the byte-for-byte parity draw path — renders interactive surfaces the server draw never had: a which-key hint bar on the prefix, a completing command palette, a ratatui clock and display-panes, and a family of zellij-style behaviors gated behind `@ztmux-zellij-mode on` (off by default) — inset pane frames a program can never overwrite, a collapsing pane stack (`prefix +`), a window tab bar (`ztmux tabs`), a filterable session manager (`ztmux sessions`), a persistent floating pane (`prefix C-f`), and prefix-free modal keybindings (`ztmux modal`).

None of this moves the shipped/roadmap line the rest of the chapter draws. Every one of these surfaces is *additive*, sitting beside the ported core rather than modifying it, and every one leaves the classic tmux surface — and the parity suite — untouched when its setting is off. What remains roadmap is the deeper stack integration the next sections describe.

What an additive feature looks like

The rule that every extension must be *additive* (never moving existing surface) is easier to believe with a concrete shape. Suppose ztmux wanted to add prompt-aware navigation — a command to jump the copy-mode cursor to the previous shell prompt. Every piece it needs already exists and none of it disturbs parity: the OSC 133 marks are already parsed into grid line flags (`START_PROMPT`, Chapter 6); the grid reader already walks stored lines (Chapter 18); the copy mode already has a cursor and key tables. The feature is a *new* `cmd_*.rs` command with a *new* default `copy-mode` binding on a key tmux leaves unbound, whose `exec` walks the grid reader backward to the nearest `START_PROMPT` line. It adds a command, adds a binding on a free key, and reads state that is already tracked — it shadows no existing command, redefines no existing format variable, and changes no default. A tmux user who never presses the new key sees exactly tmux; a user who does gets pure upside. That is what “additive” means operationally: the new surface sits *alongside* the ported core (Chapter 12’s principle), legible as an addition rather than a modification of a cited-from-C function, so the port’s faithfulness stays auditable even as it grows. The feature is roadmap, not shipped (the next section is emphatic about that) — but its *shape* is the template every superset extension follows.

The MenkeTechnologies-stack integrations ztmux is positioned for

The natural extensions for ztmux are not speculative features dreamed up in the abstract; they are the integrations the surrounding MenkeTechnologies stack already wants, and being a Rust program that the same author owns is what makes them reachable. The clearest is `zterminal`, the stack’s GPU terminal emulator, which already speaks a tmux wire protocol and already live-edits a running tmux server’s options, buffers, and keybindings from its control panel. A tmux server it can talk to *natively*, built in the same language by the same hand, is a tighter and more capable target than the real tmux’s protocol allows — the two can share types, share a socket convention, and evolve the protocol together rather than one reverse-engineering the other.

The `ztmux-core` sibling is the other half of that picture. Where this repository is the whole server and client, `ztmux-core` is a native tmux *client engine* — a library for speaking the tmux protocol — and the two are deliberately separated so the client- of-tmux story and the whole-multiplexer story stay distinct. But they are built to meet: a client engine and a server that share a codebase can offer an integration neither could alone, letting a stack application embed multiplexer functionality without shelling out to a separate `tmux` process. This is the kind of integration that is trivial when you own both sides and nearly impossible when one side is a frozen external binary.

Then there is the shell and the panes’ contents. `zshrs` — the stack’s Rust rewrite of `zsh` — runs inside the panes, and `zpw` paints inside them; a multiplexer that is aware of the shell integration protocols (the OSC 133 prompt marks the input parser already recognizes, Chapter 6) can offer prompt-aware features that a shell-agnostic multiplexer cannot. Because ztmux already parses the semantic-prompt sequences into grid line flags (`START_PROMPT`, `START_OUTPUT`), the raw material for prompt-jumping, command-output selection,

and per-command navigation is already in the grid — these are extensions the port is *positioned* for because the underlying state is already tracked, not features that would require new plumbing.

The honest framing of all this is that these are *positions*, not shipped products, and this section is careful to say so. What is real today is that ztmux is a faithful, own-able tmux in the same language and stack as the tools that would integrate with it — which is the precondition for the integrations, not the integrations themselves. The value of the position is that each of these becomes a normal engineering task rather than a cross-project protocol negotiation with a frozen dependency. The superset's promise is that when the stack wants tighter multiplexer integration, ztmux is the piece that can grow to provide it, because it is code the stack controls.

Shipped vs. roadmap — kept honest

The single most important thing this chapter can do is draw the shipped/roadmap line sharply, because a superset chapter is exactly where a project is tempted to describe ambitions as if they were features. So, plainly: what is *shipped* today is the faithful core, plus the additive extension surface catalogued above. The server, client, grid, screen, input parser, layout engine, command language, format system, options, key bindings, config loading, and terminfo back end are ported and, to the extent the parity suite measures, verified — the suite reached 122 of 122 byte-for-byte, was then expanded to 1107 cases (surfacing twenty gaps the port then closed), and stands at **1107 of 1107 passing** on Linux and macOS. Sixel graphics are implemented. The socket is namespaced so ztmux coexists with tmux. That is the real, usable multiplexer this book has spent ten chapters describing.

What is *partial* is also stated in the code itself, and the Cargo manifest is candid about it: many of the feature flags and options exist “to make matching upstream’s configuration options easier” — they are accepted and stored so a real `.tmux.conf` parses without error, but not every one is wired to behavior yet. This is a specific, honest kind of incompleteness: the *configuration surface* is broad (because parsing a tmux config faithfully requires accepting every option tmux accepts) while the *behavior behind some options* is still being connected. A user’s config will load; some of its more exotic options may not yet do anything. Chapter 15 keeps the running ledger of which subsystems are complete versus partial.

What is *roadmap* is the superset proper — the stack integrations of the previous section, and the deeper “done right” work of the next chapter — and none of it should be read as present today. There is no shipped `zterminal` native-integration feature, no shipped prompt-navigation UI, no shipped cross-stack multiplexer embedding. Those are where a Rust-owned core *can* go, and the argument of this chapter is that the port’s faithfulness and ownership make them reachable — but “reachable” is not “shipped,” and this book marks the difference rather than rounding up, because the whole series’ claim to honesty depends on drawing exactly this line.

The reason to be this careful is that the superset’s credibility is cumulative with the parity claim. A project that overstated its extensions would invite doubt about its parity numbers too; a project that is scrupulous about saying “this is roadmap” earns trust that its “this is measured” claims are equally exact. The 1107/1107 is real *because* the roadmap is labeled roadmap and the twenty gaps were printed rather than hidden — named, pinned, and fixed in the open, not quietly deleted — the same discipline produces all three. So the honest ledger is not a hedge that weakens the superset story; it is what makes the superset story believable when its features do ship, because the project will have a track record of describing its own state accurately.

Why owning the code in Rust is what makes the superset possible

Step back and ask why a superset of tmux is even a sensible goal, when tmux itself is mature, excellent, and actively maintained. The answer is ownership, and it has two components. The first is *language*: tmux is C, and C’s memory model means every new feature is written against the same manual-memory discipline, with the same class of bugs (use-after-free, buffer overrun — the very bug the `format_unescape` crash was) one

mistake away. ztmux is Rust, and while the port is currently full of `unsafe` (the subject of Chapter 14), its trajectory is toward safe Rust, where whole categories of those bugs become impossible. A superset built on a memory-safe core can add features without the per-feature risk that C imposes.

The second component is *control*. tmux is governed by its maintainers, and rightly so — a widely-depended-on tool should be conservative about what it absorbs. That conservatism is a virtue for tmux and a wall for anyone who wants to extend it deeply: a stack-specific integration, a new architecture for embedding, a feature that only makes sense inside the MenkeTechnologies ecosystem has no path into upstream tmux, nor should it. ztmux removes the wall by being owned by the same project that wants the extensions. It can move at the stack's speed, adopt the stack's conventions, and integrate with the stack's other tools, because it is not asking anyone's permission — it is the stack's own multiplexer.

Ownership without faithfulness would be worthless, which is why the whole first two-thirds of this book had to come first. A fork that diverged from tmux would be owned but would have thrown away the ecosystem and the muscle memory — it would be a *different* multiplexer, competing with tmux on features while lacking tmux's maturity, which is exactly the losing proposition the project's own bar rejects. The value is in the combination: faithful *and* owned. Faithful means the ecosystem transfers for free; owned means the extensions are reachable. Neither alone is a superset; together they are.

This is the deepest reason ztmux is a port and not a rewrite-from-spec, and it closes the loop back to Chapter 1. A rewrite could have been owned, but it would have diverged in a thousand undocumented behaviors and lost faithfulness. A wrapper could have been faithful (it *is* tmux), but it would have owned nothing and could extend nothing. Only a *faithful port* — tmux's exact behavior, in code the stack controls — is simultaneously compatible enough to inherit the ecosystem and own-able enough to grow past it. The superset is not a feature ztmux has; it is the strategic position the faithful port creates, and everything in the roadmap is a move from that position. The next chapter is about the work that turns the position from a transliteration into something worthy of the word in the README: *done right*.

Chapter 14 – The Road to “Done Right”: Shrinking `unsafe`

The README’s tagline is `[TMUX, REWRITTEN IN RUST – DONE RIGHT]`, and “done right” is a claim the current code has only partly earned. This chapter is honest about the gap: what “done right” means past parity, why the port is full of `unsafe` today, and why the order of operations is parity first, safety second, features third.

What “done right” means past parity

A faithful port can be *correct* without being *done right*, and the distinction is the whole subject of this chapter. Correct means the parity suite is green — ztmux produces tmux’s exact bytes. Done right means something more: that the code is idiomatic, memory-safe Rust that takes advantage of the language’s guarantees rather than merely transliterating C’s patterns into Rust syntax. Today’s ztmux is emphatically the first and only partway to the second. It is a near-mechanical transliteration of tmux’s C — same function names, same control flow, same intrusive data structures, same libevent calls through FFI, same raw pointers — which is exactly what made it possible to verify against the C, and exactly what “done right” has not yet finished.

The evidence is visible in the code’s texture. Nearly every ported function is `pub unsafe fn`; the data structures are `#[repr(C)]` structs threaded on raw pointers through ported BSD tree and tailq macros; the globals are `static mut`; strings are often `*mut u8` and null-terminated rather than `String` or `&str`; callbacks are `extern "C-unwind" fn` pointers. This is not accidental and it is not sloppy — it is the *honest starting point* of a port that must first match C exactly. A grid cell that is a `#[repr(C)]` struct with the same field layout as tmux’s `grid_cell` can be reasoned about against the C directly; a grid cell rewritten as an idiomatic Rust enum could not. The transliteration is a phase, and “done right” is the phase after it.

So “done right” is best understood as a *direction*, not a current state, and the tagline is a statement of intent that the rest of this chapter qualifies. The direction is: take a subsystem that is faithful (green on parity, clean on the anti-drift gate) and rewrite it from C-shaped Rust into Rust-shaped Rust — replacing raw pointers with references and owned types, `static mut` with safe interior mutability, null-terminated `*mut u8` with `str`, unsafe FFI patterns with safe wrappers — *without breaking parity*. Each such rewrite is a step from “correct” toward “done right,” and each is gated by the same parity suite that proved the subsystem correct in the first place. The tagline will be fully earned when that process has swept the tree; today it names the goal.

It is worth being clear that this is a legitimate and common shape for a serious port, not a confession of a half-finished project. Rewriting a large C program in Rust in one pass, idiomatically, with no reference to check against, is how ports fail — they diverge in a thousand places and no one can tell where. Transliterate-then-idiomatize is the disciplined alternative: get to faithful first, with the C as a checkable reference, then improve the code’s *form* while the parity suite guards its *behavior*. ztmux is a project that has finished the first phase for its core and is early in the second, and saying so plainly is exactly the honesty the series demands.

The `unsafe` surface today and why it exists

The `unsafe` in ztmux is not incidental; it is structural, and understanding *why* it is there is the key to understanding what shrinking it entails. It exists because the port mirrors tmux’s memory model, and tmux’s memory model is manual: it allocates with `malloc`, threads objects on intrusive linked structures via raw pointers, shares mutable global state, and hands raw buffers to libevent and libc. To match tmux’s behavior exactly

— including allocation patterns the parity suite can observe, and pointer identities the code relies on — the port initially reproduces that model rather than replacing it, and reproducing a manual-memory model in Rust means `unsafe`. The custom global allocator in `main.rs` (a shim over libc `malloc/free`) is the clearest tell: the port even routes Rust's allocations through libc so that memory behaves as C's does.

There are a few distinct sources of `unsafe`, and separating them clarifies which are temporary and which are more fundamental. The largest source is the *data model*: raw- pointer-threaded `#[repr(C)]` structs and the intrusive `rb_*/tailq_*` structures that every subsystem uses. This is the most reducible — Rust has safe equivalents (owned trees, `Vec`, slotmaps), and converting a subsystem's data model to them is the bulk of the “done right” work. A second source is *global state*: the `static mut` server globals (`CLIENTS`, `SESSIONS`, `SOCKET_PATH`), which safe Rust would express with interior mutability or by threading a context. A third is *FFI*: the libevent and libc calls, which are inherently `unsafe` at the boundary but can be wrapped in safe interfaces so the `unsafe` is confined to thin shims rather than smeared across every caller.

The FFI source deserves a note because it will never go fully to zero, and that is fine. ztmux links the real libevent and calls libc directly — deliberately, because a faithful port of an event-loop program should use the same event loop, not a reimplemented async runtime that might dispatch differently. Calls across that boundary are `unsafe` by definition; C has no Rust guarantees to offer. But the *right* amount of FFI `unsafe` is a small, audited set of wrapper functions, each encapsulating one C call behind a safe Rust signature, so that the rest of the code calls safe wrappers. “Done right” for the FFI is not eliminating it but *confining* it — turning a tree-wide scattering of raw C calls into a thin, reviewable boundary layer.

Seen this way, the current `unsafe` surface is a map of the remaining work, and it is a map the anti-drift allowlist already partly draws — the `rb_*`, `tailq_*`, `evbuffer_*`, and `evtimer_*` entries in `fake_fn_allowlist.txt` (Chapter 12) are exactly the ported macros and FFI shims that carry much of the `unsafe`, and shrinking that allowlist and shrinking the `unsafe` surface are nearly the same project. That is a satisfying convergence: the structural-fidelity gate's “shrink the allowlist” goal and the “done right” chapter's “shrink the `unsafe`” goal point at the same code, so progress on one is progress on the other. The port has, in effect, already instrumented the work that remains.

Idiomatic Rust as the endpoint, not the starting point

The endpoint the tagline names is a ztmux whose code a Rust programmer would recognize as Rust — safe references and owned types instead of raw pointers, `enum` and pattern matching instead of tagged-integer unions, `Result` and `?` instead of C-style error returns and out-parameters, `RAll` instead of manual free — but reaching it is a *migration*, subsystem by subsystem, each verified by parity. This is why idiomatic Rust is the endpoint and not the starting point: had ztmux started idiomatic, it could never have been checked against the C, because idiomatic Rust and C have no line-for-line correspondence. The faithful transliteration is the bridge that makes the idiomatic destination *reachable with a safety net*.

The migration order is not arbitrary; it follows the same logic as everything else in the project. A subsystem is a candidate for idiomatization once it is faithful (green on parity, clean on the gate) and once its correspondence to the C is no longer the thing being actively verified — that is, once the C has been fully mined for its behavior and the parity cases capture it. At that point the C's *structure* has done its job (guiding the transliteration) and can be left behind in favor of Rust's structure, with the parity cases now serving as the specification that the idiomatic rewrite must satisfy. The parity suite, built to *prove* faithfulness, becomes the tool that *preserves* faithfulness across the idiomatic rewrite — the same green cases, a very different implementation underneath.

There is a real tension the migration has to manage, and honesty requires naming it: making a subsystem idiomatic can *reduce* its auditability against the C, because the C citations that mapped each function to its origin become less meaningful once the function no longer mirrors the C. The project's answer is sequencing — audit fully *first*, against the transliterated version, then idiomatize — so that the auditable phase and the idiomatic phase are distinct in time. The C citations and the anti-drift gate are most valuable while a subsystem is still being verified; once it is verified and pinned by parity, the value shifts to code quality, and the citations can relax. Trying to have both maximal C correspondence and maximal Rust idiom *simultaneously* is the mistake; having them in sequence is the plan.

The payoff of reaching the endpoint is what makes the whole migration worth the care. An idiomatic, safe-Rust `ztmux` is a multiplexer where the class of bugs that produced the `format_unescape` crash — pointer arithmetic overrunning a manual buffer — is impossible by construction, where the superset's new features can be written without the per-feature memory risk C imposes, and where a contributor who knows Rust but not `tmux`'s C idioms can work on the code. That is the multiplexer the tagline promises: not just `tmux`'s behavior in Rust syntax, but `tmux`'s behavior in *Rust*, with everything the language's guarantees buy. The transliteration got the behavior right; the idiomatic migration is how it becomes done right.

What one idiomatization step actually looks like

To make stage two concrete rather than abstract, picture the first safe-Rust conversion of a small subsystem — say the tab-stop storage. Today it is a `bitstr` (a ported C bit-string, [src/bitstr.rs](#)) threaded through the screen as a raw pointer, with `bit_set`/`bit_clear`/`bit_test` operations and manual allocation. The idiomatic version would be a `Vec<bool>` or a `bitvec`-style owned bitset with methods, no raw pointer, no manual free, no `unsafe` at the call sites. The conversion is not a rewrite from scratch: the tab-stop parity behavior (default every eighth column, `HTS` sets a stop, `TBC` clears, cursor tab motion honors them) is already pinned, so the parity suite becomes the specification the new implementation must satisfy. You change the storage type, update the handful of call sites, keep the behavior, and run the parity cases; green means the idiomatic version is behaviorally identical to the C-shaped one it replaced.

Two things make this safe to do. First, the anti-drift allowlist already names the `bit_*` shims (Chapter 12), so retiring them and shrinking the allowlist are the same action — progress is measurable. Second, the C citations on the old code can *relax* once the subsystem is no longer being verified against the C, because the parity cases now carry the fidelity that the citations carried during the porting phase. That is the whole sequencing argument (below) in one subsystem: audit against the C first, with the transliteration and its citations; then idiomatize, with the parity cases as the guard. The tab stops are a tiny example; the format engine, the grid, and the layout tree are the same story at larger scale, and each is a discrete, parity-guarded step from correct toward done right.

The order of operations: parity, then safety, then features

The whole project resolves into a single ordering, and stating it plainly is the best summary of `ztmux`'s method. First, *parity*: match `tmux` byte-for-byte, verified by the parity suite and defended by the anti-drift gate, using a faithful transliteration whose C correspondence makes the matching checkable. Second, *safety*: with parity green and pinned as a specification, migrate the transliterated core toward idiomatic, memory-safe Rust, subsystem by subsystem, with the parity suite guaranteeing no behavior changes. Third, *features*: from the faithful, safe, owned core, grow the superset extensions the stack wants, each additive so parity is preserved. Parity, then safety, then features — in that order, never skipping.

The order is not a preference; it is a set of dependencies, and reversing any two links breaks the chain. Safety before parity would mean idiomatizing code before its behavior was verified against the C — the divergent-rewrite trap, where you can no longer tell whether a difference from `tmux` is a bug or an intended change. Features before parity would mean building extensions on an unverified base, so a parity failure later could be in the core or the extension and you could not tell which. Features before safety would mean writing new `unsafe` code for every extension, inheriting C's per-feature bug risk into the new work. Each stage is the precondition for the next: parity gives safety a specification to preserve, and safety gives features a foundation that does not multiply risk.

This ordering also explains why the book is shaped the way it is, and why the port looks the way it does. Ten chapters on the faithful core, two on the machinery that proves and defends its fidelity, and only then the forward-looking chapters — because that is the actual order of the work. At this stage, stage one (parity) is done for the core and measured at 1107/1107 (a suite grown from a green 122/122 until it was hard enough to

fail, then each failure pinned and fixed); stage two (safety) is early, its remaining work mapped by the **unsafe** surface and the anti-drift allowlist; stage three (features) is roadmap. A reader who wants to know “where is ztmux?” can read it off the ordering: at the boundary between stage one and stage two, with stage three clearly labeled ahead.

There is a discipline in refusing to do the stages out of order that is easy to underestimate. It is tempting, at every point, to jump ahead — to idiomatize a subsystem that is *almost* verified, to add a compelling feature before the safety migration reaches its part of the tree. The project’s answer is that the ordering is what makes each claim trustworthy: the parity number means something because features did not come first; the eventual safety will mean something because it came after verification; the eventual features will mean something because they sat on a verified, safe base. Rushing a stage would forfeit exactly the trust the careful ordering builds. “Done right” is, in the end, as much about the *order* the work is done in as about the work itself — and the last chapter is the honest accounting of exactly how far down that order the project has come.

Chapter 15 – Status and the Honest Gaps

Every book in this series ends by walking its own boundaries. This chapter states where the port stands, which subsystems are complete and which are partial, and how the port report and doc-count gates keep that picture accurate rather than aspirational.

The port report: what's ported, what's pending

The first honest answer to “how complete is ztmux?” is a generated one, not a claimed one. The port report (`scripts/gen_port_report.py`, published as `port_report.html`) walks the Rust source and the vendored C, matches functions across the two using the same C index the annotation script builds, and reports the coverage — how much of tmux's function surface has a ported counterpart and how much remains. Because it is derived from the vendored C rather than from a hand-maintained checklist, it cannot flatter the port: refresh the vendored tmux, re-run the script, and the report reflects reality. This is the same anti-inflation discipline as the divergence diff and the anti-drift gate, applied to the aggregate progress number.

What the report shows, today, is a core that is substantially ported — every subsystem this book walked has its functions present and cited — with the pending surface being the long tail of tmux's less-central commands, the deeper corners of features whose common paths are done, and the platform and build variations behind the feature flags. The report is the right place to look for the *specific* pending items, because it is per-function and current; this book does not reproduce its list, because a list transcribed into prose would be stale the moment the next function is ported. The report is live; the book points at it.

The report pairs with the divergence diff (Chapter 2) to answer two different completeness questions. The report answers “how much of tmux is covered?” — coverage against the C. The diff answers “how much has ztmux changed since the seed?” — contribution against the tmux-rs starting point. A reader who wants the full honest picture of the port's state runs both: the report for breadth of coverage, the diff for depth of original work. Neither is a number the author asserts; both are artifacts the tooling generates from the committed references.

The value of having the completeness question answered mechanically is that it removes the temptation to round up, which is the single most common dishonesty in port projects. A project that describes its completeness in prose will, however well-intentioned, drift toward optimism. A project whose completeness is a generated report against a committed reference cannot — the number is what it is, and it moves only when real code moves it. ztmux's status, in the aggregate, is whatever the port report says today, and the book's job is to point you at the live artifact rather than to freeze a number that will be wrong tomorrow.

Complete vs. partial subsystems today

Zooming from the per-function report to the subsystem level, the picture is legible and worth stating directly. The subsystems that are *complete* in the sense that matters — ported and verified byte-for-byte by the parity suite on both Linux and macOS — are the ones this book spent its core chapters on and the ones the 122 parity cases exercise: the format language and its modifiers (the bulk of the cases), arithmetic and comparison and string operations within formats, the core session/window/pane commands (`new-window`, `split-window`, `list-windows`, `select-layout`, and their kin), the options get/set surface, and the layout arithmetic. Where a `.fmt` or `.sh` case pins a behavior, that behavior is complete in the strong sense.

The subsystems that are *ported but partial* are the ones present in the tree and transliterated but not yet exhaustively pinned by parity or not yet fully wired to behavior. The configuration surface is the clearest example: the options table accepts every option tmux accepts — it must, to parse real config files — but, as the Cargo manifest admits, some options are stored without yet driving behavior, existing “to make matching upstream’s configuration options easier.” Sixel is the notable counter-example the manifest calls out as genuinely implemented. The right way to read a partial subsystem is: the common paths are ported and the structure is faithful, but the coverage of edge cases and the wiring of every option is still being completed, and only a parity case makes a specific behavior a guarantee.

The `unsafe` dimension (Chapter 14) is orthogonal to the ported/verified dimension and worth holding separately in mind. Every subsystem, complete or partial, is currently `unsafe`-heavy — being verified against the C does not make a subsystem idiomatic Rust, it makes it faithful C-shaped Rust. So “complete” here means “faithful and verified,” not “done right” in the Chapter 14 sense. A subsystem can be fully parity-verified and still be entirely raw pointers and `static mut`; the format engine is exactly this — thoroughly exercised by the parity suite, and thoroughly `unsafe`. Keeping the two axes distinct is part of the honesty: verified is one claim, idiomatic is another, and at this stage the core is far along the first axis and early on the second.

The one number that summarizes the verified axis is the parity count, and it deserves its caveats stated once more so it is never overread. 1107/1107 means: of 1107 specific cases, each a format string or a short command scenario, all 1107 produce byte-identical output between the vendored `next-3.7` tmux and ztmux, on Linux and macOS, under fixed geometry and `LC_ALL=C` (twenty of them guarding gaps the expansion once caught and the port has since fixed; Appendix D). It does *not* mean every tmux behavior is covered — 1107 cases is a large sample, not the whole of tmux’s behavior. It is a strong, growing, reproducible measurement of a real and central slice of fidelity, and it is exactly as strong as its case count, no more. Read that way, it is the most trustworthy status indicator the project has, precisely because it claims neither more nor less than it measures.

The doc-count and man-page gates

A port that is legible today can become illegible tomorrow if its documentation is not gated, and ztmux inherits the stack-wide documentation gates for exactly that reason. The doc-count gate watches the ratio of documented to undocumented public surface and fails the build if it regresses; the man-page gate ensures the reference material stays present and buildable. These do not measure correctness — parity does that — but they measure whether the project stays *understandable*, which for a port whose entire pitch is auditability is its own kind of correctness. A faithful port that no one can read is a faithful port that no one can verify, and the doc gates keep that from happening quietly.

These gates compose with the C-signature citations (Chapter 12) to give the project an unusually high documentation floor for its age. Every ported function already carries a citation to its C origin — over two thousand of them, kept fresh by `annotate_c_links.py`’s `--check` mode — so the codebase is self-documenting in the specific dimension that matters most for a port: provenance. The doc-count gate ensures that beyond the citations, the public API carries real documentation, and the man-page gate ensures the user-facing reference exists. Together they mean a reader arriving at any level — a function, a module, a command — finds either a C citation, a doc comment, or a man page pointing them at what they need.

The gates also serve the same anti-drift purpose in documentation that the fake-fn gate serves in code. Just as code can drift from the C by growing unmatched functions, documentation can drift from the code by falling behind it — new functions with no docs, changed behavior with stale man pages. Gating the documentation ratio and the man-page presence turns “keep the docs current” from a chore that slips into a build requirement that cannot. This is the same philosophy as the rest of the project: do not rely on discipline where a gate will do, because gates run on every push and discipline does not.

That the documentation is gated at all, this early in the port, is itself a status signal. A project that gates its docs this early is a project that intends the code to be read — by reviewers checking the port against the C, by contributors learning the subsystem, by the author six months from now. The gates are a bet that legibility is worth enforcing before the code is even feature- complete, which is the right bet for a port: the whole value

proposition is that the fidelity is *checkable*, and checkable requires readable. The doc gates are how the project keeps the promise that its faithfulness can be audited rather than merely believed.

A subsystem-by-subsystem ledger

To make the complete-versus-partial distinction concrete, here is the honest state of the subsystems this book walked, read against the two axes that matter — *verified* (pinned by parity) and *idiomatic* (the Chapter 14 axis). Every subsystem below is *unsafe*-heavy, so the idiomatic axis is uniformly “early”; the interesting variation is on the verified axis:

- **Format engine** — heavily verified (the bulk of the 1107 cases), fully green, including the four corners the expansion once caught and the port then fixed (`#{!:...}`, `#{s/.../\1/}`, `#{p-N:...}`, loop-var injection; Appendix D). The most exercised subsystem in the tree.
- **Layout** — verified on the arithmetic that matters (even-split rounding, resize distribution), pinned by both unit tests and parity cases, with `main-vertical` + `main-pane-width` a named gap.
- **Options / commands** — the core get/set surface and the common session/window/pane commands are pinned by `.sh` cases; named buffers are a current gap.
- **Server / client / protocol** — exercised indirectly by every `.sh` case (each spins up a real server and client), and directly by the coexistence and detach behaviors.
- **Grid / screen / input parser** — exercised indirectly (every format reads grid-derived state; every `.sh` scenario drives the parser), and structurally faithful with full C citations, but with fewer *direct* byte-comparison cases than the format engine.
- **Terminfo back end** — ported with the capability lookup, overrides, and output minimization; its output byte stream is observable but less directly parity-cased than formats.
- **Copy mode, status line, spawn, hooks/alerts** — ported and functional, with the common paths covered and the long tail of edge cases still being pinned.

The honest one-line reading: the *computational and command surface* is thoroughly verified, the *rendering and I/O surface* is faithfully ported and indirectly exercised, and *everything* is still C-shaped Rust awaiting the idiomatic pass. Where a `.fmt` or `.sh` case pins a behavior, it is complete in the strong sense; elsewhere the structure is faithful and the coverage is growing. That is the ledger, and it is exactly as current as the parity count it is read against.

From port to superset: the road ahead

The honest summary of where ztmux stands is a single sentence: it is a faithful, parity-verified, *unsafe*-heavy transliteration of tmux’s core, at the boundary between the “achieve parity” stage and the “become idiomatic and safe” stage, with the superset features labeled clearly as the stage after that. Every claim in this book resolves to that sentence. The core chapters describe the transliteration; the parity and anti-drift chapters describe how its fidelity is proven and defended; the superset and “done right” chapters describe the two stages ahead. Real, usable, and still early in its own roadmap — even as the crate version tracks tmux’s 3.7 line for compatibility — is precisely the state this book describes.

The road ahead is the ordering of Chapter 14, walked forward. Stage two, safety, is the near-term work: sweep the tree subsystem by subsystem, converting faithful C-shaped Rust into idiomatic safe Rust, each conversion guarded by the parity suite so behavior is preserved and each shrinking both the *unsafe* surface and the anti-drift allowlist (the two being nearly the same set). Stage three, features, is the longer horizon: from the faithful, safe, owned core, grow the additive superset extensions the stack is positioned for — the `zterminal` and `ztmux-core` integrations, the prompt-aware features the OSC 133 tracking already lays groundwork for — each preserving parity by construction. Neither stage is a rewrite; both are migrations from a base that is already correct.

What will *not* change on that road is the discipline that got the project here, and that is the most important thing to carry out of this book. The parity suite will keep defining correct; the anti-drift gate will keep defending

structure; the C citations will keep provenance legible; the divergence diff will keep contribution honest; the port report will keep completeness mechanical; and the doc gates will keep the whole thing readable. These are not scaffolding to be removed once the port matures — they are the permanent instrumentation that lets ztmux claim faithfulness as a measured property rather than an asserted one, at every stage, forever. A superset that abandoned them would be a fork wearing tmux's name; a superset that keeps them is what the README means by done right.

And so the book ends where it began, with the honesty that is the point. ztmux is tmux, reimplemented in Rust, ported against the C and verified against the C, byte for byte — a faithful core today, an idiomatic and safe core tomorrow, a superset the day after. It is early software, and this book has marked the early parts as early rather than rounding up, because the project itself does: it measures its parity, gates its drift, cites its sources, and labels its roadmap. Read against that standard, ztmux is exactly what it claims to be and nothing it does not — the multiplexer itself, rebuilt in Rust, done carefully enough that every claim about it is one you can check. That checkability is the real inheritance from tmux, and the real gift to whoever reads the code next.

Chapter 16 — The Format Mini-Language in Depth

Chapter 9 introduced the `#{...}` format language as one of the three systems that make `tmux` configurable. It deserves a chapter of its own, because it is the single largest module in the tree (`src/format.rs`, over two hundred kilobytes, more than two hundred C citations), it is where the bulk of the parity suite lives, and it is a genuine little programming language embedded in every status string, `display-message`, pane title, and `if-shell` condition. This chapter walks the operators one family at a time — with real inputs and the exact bytes they expand to — and marks, honestly, the handful of operators that are the current parity gaps.

The expansion pipeline: `format_expand` and `format_replace`

A format string is expanded by `format_expand` (`src/format.rs`), which scans for `#{`, `#[`, and `#` escapes and hands each `#{...}` to `format_replace`. Two design details govern everything else. First, expansion is *recursive with a guard*: a variable's value can itself contain `#{...}`, and the engine re-expands it, but a counter capped at `FORMAT_LOOP_LIMIT = 100` stops a self-referential format (`set -g x '#{x}'; display -p '#{x}'`) from recursing forever — it expands a hundred deep and stops, exactly as `tmux` does, rather than blowing the stack. Second, the engine keeps a `format_expand_state` carrying flags like `FORMAT_EXPAND_TIME` (evaluate `#{t:...}` time formats) and `FORMAT_EXPAND_NOJOBS` (do not spawn `#(shell)` jobs — used where a job would be unsafe or non-deterministic), threaded through the recursion so a nested expansion inherits the right evaluation mode. The parity suite runs with jobs effectively inert and time excluded precisely because `#(date)` and `#{t:...}` are non-deterministic; the state flags are how `tmux` itself expresses that distinction.

The `#{` handler parses, in order: an optional block of *modifiers* (the operators this chapter catalogues), then a *variable name or sub-expression*, then, for the operators that take them, comma-separated *arguments*. The modifiers are accumulated into a `format_modifiers` bitset (`FORMAT_BASENAME`, `FORMAT_DIRNAME`, `FORMAT_LITERAL`, `FORMAT_QUOTE_SHELL`, `FORMAT_TIMESTRING`, `FORMAT_SESSIONS`, `FORMAT_WINDOWS`, `FORMAT_PANES`, `FORMAT_CLIENTS`, `FORMAT_LENGTH`, `FORMAT_WIDTH`, `FORMAT_NOT`, and the rest), each corresponding to one letter or symbol after the `#{`. The big `match` that parses them is the heart of the module, and every case below is one arm of it.

Variable substitution and the `FORMAT_TABLE`

The simplest format is a bare variable: `#{pane_current_command}` expands to the pane's foreground process name, `#{session_name}` to the session's name, `#{window_index}` to the window's index. These names resolve two ways. Dynamic per-object fields (`session_name`, `window_width`, `pane_pid`, and hundreds more) are computed from the live server state at expansion time by `format_create_add`-populated entries and the big resolver. A second set — the *callback* variables — live in `FORMAT_TABLE`, a sorted array binary-searched by `format_table_get`; these are the ones whose value is computed by a small function rather than stored, like `host`, `host_short`, `pid`, `version`, `cursor_x`, `client_termname`, `mouse_x`, `pane_mode`, and the alternate-screen and scroll-region reporters. The table is kept sorted so lookup is a `binary_search_by`, and Appendix B lists the full variable set.

Worked example, against a fresh 80×24 detached session (which is exactly the parity harness's environment):

```
$ ztmux display-message -p '#{window_width}x#{window_height}'
80x24
$ ztmux display-message -p 'panes=#{window_panes} idx=#{window_index}'
panes=1 idx=0
```

Because so much of the server's state is reachable this way, a format string is also an *introspection query* — `list-panes -F '#{pane_pid} #{pane_current_command}'` is a structured dump of the pane table — and that dual role is why the parity suite is mostly format cases: a matching expansion of `#{window_panes}` after a split is evidence the pane tree itself matches.

Arithmetic: `#{e|op|:a,b}`

The `e` modifier introduces arithmetic, and its syntax is `#{e|OP|FLAGS:A,B}` where `OP` is the operator, the optional `FLAGS` control float precision, and `A,B` are the operands (themselves formats, so they can be nested). The operators map to an `Operator` enum evaluated in `f64`: `+` add, `-` subtract, `*` multiply, `/` divide, `%` modulus, `&` bitwise-and, `|` bitwise-or, and the comparison operators (`==`, `!=`, `<`, `>`, `<=`, `>=`) which yield `1` or `0`. Worked examples straight from the parity cases:

```
#{e|+:2,3}      → 5      (010_arith_add.fmt)
#{e|-:10,4}     → 6
#{e|*:6,7}      → 42
#{e|/:7,2}      → 3      (integer division by default)
#{e|=:17,5}     → 2
#{e|/f2:22,7}   → 3.14   (f2 = two decimal places)
#{e|&:6,3}      → 2      (bitwise and)
```

Two faithful subtleties the parity suite pins. Division and modulus by zero must match tmux's exact output rather than crash or print `inf` (`202_arith_divzero.fmt`, `203_arith_modzero.fmt`) — tmux has a defined result and ztmux reproduces the bytes. And equality on floats is not `==` on IEEE bits but an epsilon compare — `(mleft - mright).abs() < 1e-9` — because tmux's `#{e}` treats near-equal floats as equal, and a naive port that used exact float equality would diverge on `#{e|=:0.1,0.1}` computed two different ways. These are the corners a spec would miss and a byte comparison catches.

Comparisons and booleans: `#{==:a,b}`, `#{&&:...}`, `#{||:...}`, `#{m:...}`

Outside arithmetic, formats have direct comparison operators that work on strings and numbers: `#{==:a,b}`, `#{!=:a,b}`, `#{<:a,b}`, `#{>:a,b}`, `#{<=:...}`, `#{>=:...}`, plus the boolean combinators `#{&&:a,b}` and `#{||:a,b}`, and the pattern matcher `#{m:pattern, string}` (fnmatch-style glob) with a `#{m/r:...}` regex variant. Each yields `1` (true) or empty (false) — note *empty*, not `0`, for the boolean/conditional forms, which matters because conditionals test for emptiness. Examples:

```
#{==:abc,abc}   → 1
#{==:abc,xyz}   → (empty)
#{&&:1,1}        → 1
#{m:*.rs,main.rs} → 1      (fnmatch glob)
#{m:*.rs,main.txt} → (empty)
```

The `!` logical-not operator, `#{!:...}`, inverts a truth value, and this is one of the current parity gaps: tmux prints `1` for `#{!:}` on an empty (false) argument, and ztmux prints empty because the `!` operator is not yet wired into the modifier parse (cases 548–550, Appendix D). It is a small, bounded fix — one arm of the modifier `match` — and it is kept red in the suite until it lands.

Conditionals: `#{?condition,then,else}`

The conditional is `#{?COND,THEN,ELSE}`: if `COND` expands to a non-empty string it yields `THEN`, otherwise `ELSE`. The condition is commonly a variable (truthy when non-empty) or a comparison. This is the workhorse of status-line configuration — showing one thing when a pane is zoomed and another when it is not, one color when a window has activity and another when it does not:

```
#{?pane_active,[active],[inactive]}    → [active]    (in the active pane)
#{?window_zoomed_flag,Z,}              → Z or empty
#{?#{==:#{window_panes},1},single,multi} → single      (one pane)
```

The last example shows nesting: the condition is itself a `#{==:...}` over `#{window_panes}`, and the whole thing composes because every part is a format re-expanded by the same engine. The comma is the argument separator, and a literal comma inside a branch is escaped as `#,` — a detail the parity string-op cases exercise.

String operations: `#{s/.../.../:}`, `#{=N:}`, `#{b:}`, `#{d:}`, `#{q:}`, `#{n:}`

The string family is large. `#{b:path}` is basename, `#{d:path}` is dirname — both implemented against the ported `basename/dirname` shims:

```
#{b:/usr/local/bin/ztmux} → ztmux
#{d:/usr/local/bin/ztmux} → /usr/local/bin
```

`#{=N:string}` limits width to `N` columns from the left (`#{=-N:string}` from the right), respecting wide characters; `#{=/N/...:string}` adds a trailing marker when truncated. `#{n:string}` is the length. `#{q:string}` shell-quotes (escaping so the result is safe to hand to a shell), and `#{q:...}` in `FORMAT_QUOTE_STYLE` form escapes for a style string instead. `#{t:variable}` formats a time-valued variable through `strftime` (with `#{t/f/FORMAT:...}` choosing the format and a `p` flag for a “pretty”/relative form).

The substitution operator `#{s/PATTERN/REPLACEMENT/:string}` runs a regex substitute (`src/regsub.rs`), with a trailing `g` flag for global replace:

```
#{s/foo/bar/:foobar}      → barbar    (first match)
#{s/o/0/g:foobar}         → f00bar    (global)
```

Regex *backreferences* in the replacement — `\1`, `\2` referring to capture groups — are the second current parity gap: on input `abcd` with a pattern capturing pairs, `#{s/(.)(.)/\2\1/g:abcd}` should yield `badc`, but `ztmux` emits the literal digit next to the capture (`b2a1d2c1`) because the backreference substitution is not wired in the replacement builder (cases 566–568, 572, Appendix D). Like `#{!:...}`, it is a localized fix in one function, held red until done.

Padding: `#{pN:}` and the left-pad gap

The `p` modifier pads a field to a fixed width: `#{pN:string}` right-aligns to width `N` (padding on the left), and the fill character defaults to a space. This is how status lines keep columns aligned regardless of content width. The right-pad and left-pad forms are `#{pN:}` and `#{p-N:}`. The left-pad form `#{p-N:...}` is the third current parity gap: `tmux` treats `N` as the *total field width*, but `ztmux` currently pads to `N` *plus* the content length, so a `#{p-5:ab}` that should produce a five-wide field produces a seven-wide one (cases 589, 592, Appendix D). Again bounded, again kept red until the arithmetic is corrected to a total-width computation.

Loops: `#{S:...}`, `#{W:...}`, `#{P:...}`, `#{L:...}`

The loop modifiers expand a sub-format once per object: `#{S:fmt}` once per session, `#{W:fmt}` once per window in the current session, `#{P:fmt}` once per pane in the current window, and `#{L:fmt}` once per client. They are implemented by `format_loop_sessions`, `format_loop_windows`, `format_loop_panes`, and the client loop, each of which iterates the relevant tree and re-expands `fmt` with that object pushed as the current context:

```
#{W:#{window_index}:#{window_name} } → 0:zsh 1:vim 2:logs
```

Inside a loop, tmux injects two synthetic variables per iteration — `loop_index` (the zero-based position) and `loop_last_flag` (1 on the final iteration) — which `format_loop_sessions` sets via `format_add!(nft, "loop_index", ...)` and `format_add!(nft, "loop_last_flag", ...)`. The `#{S:}/#{W:}/#{P:}` capital-letter loop modifiers are the fourth current parity gap: they iterate correctly but do not inject `loop_index/loop_last_flag`, so an inner `#{loop_index}` expands empty where tmux prints the counter (cases 676–678, Appendix D; the C reference is `vendor/tmux/format.c:4776`). The lowercase per-object loops used above work; the gap is specifically the injected iteration variables in the capital-letter forms.

Literals and the `format_unescape` trap

The `l` modifier, `#{l:some text}`, means “this is a literal — unescape it and stop,” which is how you put otherwise-interpreted characters into a format. Its handler calls `format_unescape` (`src/format.rs`, citing `vendor/tmux/format.c:4281`), and this small function is the site of the most instructive bug in the port’s history. The C is a `for (; s != end; s++)` loop whose `s++` runs on every iteration, including the ones that hit a `continue`; an early Rust port translated it to a `while` loop that advanced `s` on most branches but forgot to on the normal path, so on a plain character it looped forever, writing past the `xmalloc(strlen + 1)` buffer and crashing the server. The parity case was a single `#{l:...}` string that tmux expanded and ztmux died on. The fix advances the pointer on every path, matching the C’s per-iteration increment, and the function now carries a comment documenting the trap so it cannot be reintroduced silently. It is the clean archetype of the whole methodology: a byte-for-byte parity case turned a mysterious crash into a one-line, C-anchored fix.

Quoting, escaping, and the `#[...]` style directive

Two mechanisms sit next to the `#{...}` operators and are easy to confuse with them. The first is *escaping*: because `#` introduces a format and `,` separates arguments, a literal `#` is written `##` and a literal comma inside a conditional branch is written `#,`. The `#{q:string}` operator (`FORMAT_QUOTE_SHELL`) does the inverse for shell contexts — it backslash-escapes a string so it is safe to hand to a shell, which is how `run-shell "grep #{q:pane_title}"` avoids being broken by a title containing spaces or quotes. A parallel `#{q:...}` style-quote form (`FORMAT_QUOTE_STYLE`) escapes for a style string instead. Getting the quoting exactly right matters because a format that under-quotes is a shell-injection bug and a format that over-quotes prints stray backslashes; the parity string cases (`037_str_quote` and `kin`) pin it.

The second mechanism is the `#[...]` style directive, which is not a `#{...}` expansion at all but an inline styling instruction the drawing layer (`format_draw.rs`, Chapter 20) interprets. `#[fg=red,bold]` sets the foreground and bold attribute for the text that follows in a status string; `#[default]` reverts; `#[reverse]` inverts. Styles and formats compose in the same string — a status line is typically a mix of `#{...}` values and `#[...]` directives — but they are processed by different machinery: `format_expand` resolves the `#{...}` into values, and `format_draw` walks the resulting string turning `#[...]` runs into actual cell attributes and recording the style *ranges* that make status-line regions clickable (Chapter 20). Keeping the two straight — expansion versus styling — is the last piece of reading a real status-line format fluently, and both are ported so a `.tmux.conf`’s status string renders identically, colors, clickable ranges, and all.

More operators: character, color, search, and re-expansion

Beyond the big families there is a tail of single-letter operators the modifier `match` handles, each small but real. `#{a:N}` (`FORMAT_CHARACTER`) turns a numeric code point into its character — `#{a:65}` is `A` — which is how a format produces a byte it could not write literally. `#{c:colour}` (`FORMAT_COLOUR`) resolves a color name or spec to its numeric value through the same `colour.rs` machinery the grid uses, useful for computing styles. `#{w:string}` (`FORMAT_WIDTH`) reports the display *width* of a string in columns, counting wide characters as two — distinct from `#{n:string}` (`FORMAT_LENGTH`), which counts characters — so a status line can budget columns correctly for CJK text.

Two operators control *re-expansion*, which is subtle enough to trip up format authors. `#{E:variable}` (`FORMAT_EXPAND`) expands the variable's value a second time, so a variable whose value itself contains `#{...}` gets those inner formats evaluated; `#{T:variable}` (`FORMAT_EXPANDTIME`) does the same but also enables time expansion for the inner pass. These exist because, by default, a variable's value is used verbatim, and sometimes you have stored a format *in* an option (a user status template, say) that you want evaluated when read. The `#{N:variable}` operator (`FORMAT_WINDOW_NAME` / `FORMAT_SESSION_NAME`, selected by a `/w` or `/s` argument) checks whether a named window or session exists, yielding truthy or empty — the building block for a status line that shows something only when a particular session is alive. And `#{C:pattern}` (the search modifier) searches the pane's content for a pattern and reports the matching line, which is how a format can surface “the line where the error is.” Each of these is one arm of the same big modifier `match`, ported from `tmux`, and each is a reminder that the format language is genuinely a language: it has character construction, type coercion, existence tests, and controlled re-evaluation, not just variable interpolation.

Why the format language is the parity suite's center of gravity

Step back and the reason formats dominate the parity suite is structural. A format case is a single deterministic string, expanded by `display-message -p` against a fixed session, diffed byte-for-byte — cheap to write, trivially reproducible, and, because formats read so much of the server's state, an enormous surface covered through one uniform mechanism. Arithmetic, comparisons, string ops, conditionals, padding, loops, and every per-object variable are all reachable as `.fmt` cases, which is exactly why the suite grew from 122 to 1107 mostly by adding formats, and exactly why the four format corners the expansion caught (`#{!:...}`, `#{s/.../\1/}`, `#{p-N:...}`, loop-var injection) were each a named case with a reproduction rather than an unknown — and are now green, each one still guarding its corner. The format engine is where the port is most thoroughly exercised, most thoroughly `unsafe`, and most cleanly measured — a microcosm of the whole project.

Chapter 17 – The Wire Protocol and `msg` Framing

Chapters 3 and 4 walked the server and the client as two halves of one binary. This chapter is about the seam between them: the socket messages that carry a client's attach, its keystrokes, its resizes, and its detach to the server, and the server's replies back. `tmux` does not invent a bespoke protocol framer — it uses OpenBSD's `msg` library, the same small message-passing layer OpenBSD daemons use to talk between privilege-separated processes — and `zmtx` ports the message set and the framing faithfully, because the wire format is an internal contract both halves must agree on exactly.

`msg`: length-prefixed messages with passed file descriptors

`msg` is a deliberately small idea: a message is a fixed header followed by a payload, buffered and framed over a Unix domain socket, with the ability to pass an open file descriptor alongside the bytes. The header carries a message *type* (a `u32` — the `MSG_*` values below), a *length*, a *peer id*, and a *pid*, and the library handles the buffering so that a partial read never delivers half a message: a reader pulls complete messages out of an internal buffer, and a writer queues messages that drain as the socket becomes writable. `zmtx` drives this through `src/proc.rs` (the `tmuxproc/tmuxpeer` abstraction) and the buffered event layer, with the `evbuffer_*` and `msg` calls appearing in the anti-drift allowlist (Chapter 12) precisely because they are library functions with no `vendor/tmux` definition to match — real functions, just not `tmux`'s own.

The file-descriptor-passing capability is not a nicety here; it is load-bearing to the whole architecture. When a client attaches, it does not *describe* its terminal for the server to open — it passes the actual open descriptors for its `stdin` and `stdout` across the socket, using `msg`'s `SCM_RIGHTS`-based fd transfer. That is what `MSG_IDENTIFY_STDIN` and `MSG_IDENTIFY_STDOUT` carry: duplicated file descriptors, not paths. The consequence, revisited from Chapter 10, is that the server's tty back end writes directly to the client's real terminal fd — it is driving hardware it received over a socket. No other part of the design would work without fd passing; it is why the server, a daemon with no controlling terminal of its own, can nonetheless paint your screen.

`PROTOCOL_VERSION = 8` and the version handshake

The very first thing the two halves do is agree they speak the same protocol. `src/tmux_protocol.h.rs` declares `pub const PROTOCOL_VERSION: i32 = 8`, and it is the first field checked: on connect the server sends `MSG_VERSION`, and the client compares the server's version against its own `PROTOCOL_VERSION`. On mismatch it prints `protocol version mismatch {client {}}, {server {}}` and refuses to proceed. This guard exists because the `msg` protocol is an *internal build contract*, not a stable ABI — the message set and payload layouts can change between builds, and a client from one build talking to a server from another could silently corrupt state. The version bump is how “these two processes must come from the same code” is enforced mechanically rather than hoped for. It is also why, in normal use, you never see the check fire: the client that autostarts a server is the same binary, so the versions always match unless you have mixed builds behind one socket.

The message catalogue

The `MSG_*` set is small and every message has a clear job. They group naturally:

- **Version and readiness.** `MSG_VERSION` (protocol check), `MSG_READY` (server signals the attach is complete), `MSG_WAKEUP` (nudge a waiting client).
- **The identify handshake** — the client's self-description, one fact per message: `MSG_IDENTIFY_LONGFLAGS` (64-bit client flags), `MSG_IDENTIFY_FLAGS`, `MSG_IDENTIFY_TERM` (the `TERM` string), `MSG_IDENTIFY_FEATURES` (negotiated terminal features), `MSG_IDENTIFY_TTYNAME`, `MSG_IDENTIFY_CWD` and `MSG_IDENTIFY_OLDPWD`, `MSG_IDENTIFY_TERMINFO` (one per terminfo capability), `MSG_IDENTIFY_STDIN` and `MSG_IDENTIFY_STDOUT` (the passed fds), `MSG_IDENTIFY_CLIENTPID`, `MSG_IDENTIFY_ENVIRON` (one per environment variable), and `MSG_IDENTIFY_DONE` to close the stream. The server assembles a client object as these arrive.
- **Command and control.** `MSG_COMMAND` (run a command line), `MSG_SHELL` (the shell-command path), `MSG_EXEC` (have the client `exec`l a program in its own process), `MSG_LOCK/MSG_UNLOCK` (run the lock command), `MSG_FLAGS` (client flag changes), `MSG_RESIZE` (new terminal geometry — sent on attach and on every `SIGWINCH`).
- **Exit and detach.** `MSG_DETACH` and `MSG_DETACHKILL` (detach, optionally killing the session), `MSG_SUSPEND` (stop the client with `SIGTSTP`), `MSG_EXIT`, `MSG_EXITED`, `MSG_EXITING`, `MSG_SHUTDOWN`.
- **File I/O over the socket.** `MSG_READ_OPEN/MSG_READ/MSG_READ_DONE/MSG_READ_CANCEL` and `MSG_WRITE_OPEN/MSG_WRITE/MSG_WRITE_READY/MSG_WRITE_CLOSE` — the messages behind `load-buffer/save-buffer` and control-mode file transfer, which let the server read and write files *in the client's context* (with the client's permissions and working directory) rather than its own.
- **Legacy fd carriers.** `MSG_OLDSTDIN/MSG_OLDSTDOUT/MSG_OLDSTDERR` — the older descriptor-passing messages, kept for the compatibility they name.

An attach, message by message

Putting the catalogue in motion, here is the sequence when you run `ztmux attach`:

```
client → connect() to <dir>/ztmux-<uid>/default
server → MSG_VERSION (8)
client → (checks version, aborts on mismatch)
client → MSG_IDENTIFY_LONGFLAGS, _TERM, _FEATURES, _TTYNAME, _CWD,
        _TERMINFO (*N), _STDIN(fd), _STDOUT(fd), _CLIENTPID,
        _ENVIRON (*N), _IDENTIFY_DONE
server → (builds the client, binds it to a session, sizes it)
server → MSG_READY
client → sets CLIENT_ATTACHED, sends initial MSG_RESIZE
... steady state: keys in, drawing out, MSG_RESIZE on SIGWINCH ...
client → MSG_DETACH (or server → MSG_DETACH / MSG_SHUTDOWN)
server → tears down the client; session keeps running
```

Two properties of this exchange are worth naming. It is *asymmetric*: the client front-loads everything about its terminal in the identify burst, then goes quiet and becomes a relay, because all intelligence lives server-side (Chapter 4). And it is *idempotent across re-attach*: a second `attach` from a different terminal runs the identical handshake with that terminal's facts, and the server rebinds the same session — the protocol has no notion of “resume,” because there is nothing to resume; the session never stopped. The wire protocol is thin for the same reason the client is thin: it carries a terminal's identity and a stream of keys and resizes, and lets the server do the rest.

Control mode: the protocol as a text stream

There is a second face of the protocol worth naming, because it is the seam the superset's integrations lean on hardest. A client can attach in *control mode* (`ztmux -[C]`), and instead of driving a terminal it exchanges a line-based text protocol with the server: it sends commands and receives structured %-prefixed notifications — `%output` when a pane produces bytes, `%window-add`, `%window-close`, `%layout-change`, `%session-changed`, and the rest — each corresponding to the internal notifications of Chapter 21. `src/control.rs` and `src/control_notify.rs` implement this, translating the server's internal events into the control-mode text lines a program can parse. Control mode is how a GUI front end embeds a tmux server: it drives the server programmatically and mirrors the server's state from the notification stream rather than scraping a rendered terminal.

This is precisely the seam `zterminal` uses today against the real tmux, and the reason the superset story (Chapter 13) is more than aspiration: a control-mode client and a server that share a codebase can evolve the protocol together — share types, share the socket convention — rather than one reverse-engineering the other. Porting control mode faithfully keeps ztmux a drop-in target for anything that already speaks tmux control mode, and porting the underlying notification vocabulary (Chapter 21) is what makes the control-mode stream identical. The text protocol and the `imsg` protocol are two views of the same event flow: `imsg` for the terminal-driving client, control mode for the programmatic one, both fed by the same server-side state changes.

Why porting the protocol byte-exactly matters

It would be tempting, in a Rust rewrite, to “improve” the protocol — use a nicer serialization, drop the legacy messages, restructure the identify burst. The project refuses, and the refusal is the same discipline as everywhere else. The protocol is observable at the boundary with `ztmux-core` (the sibling client engine that speaks the tmux wire protocol) and it is the seam the superset's `zterminal` integration (Chapter 13) will build on. Keeping `PROTOCOL_VERSION` and the `MSG_*` set faithful to tmux's — and evolving them deliberately rather than incidentally — is what keeps that seam legible. A protocol that drifted casually would be a protocol no sibling tool could trust, which is exactly the kind of un-owned dependency the whole project is built to avoid.

Chapter 18 — Copy Mode and the Grid Reader

Chapter 5 introduced copy mode as a cursor and a selection over the grid, and promised a closer look. This chapter is it: how `src/window_copy.rs` turns a pane into a navigable, searchable, extractable view of its own scrollbar, driven by the motion primitives in `src/grid_reader.rs`, and how the whole thing is a *mode* layered on the grid rather than a separate program. Copy mode is where the grid stops being a rendering buffer and becomes a data model you can walk.

The mode layer and the window mode stack

A pane is normally showing the live output of its program, but tmux can push a *mode* onto it — copy mode, the clock, the tree/menu choosers — that takes over the pane’s display and key handling until it is popped. The machinery is `src/mode_tree.rs` and the per-mode `window_mode` structs; copy mode’s is `window_copy_mode`. Entering copy mode (`copy-mode`, or the default prefix-`[`) calls `window_copy_init`, which snapshots the pane’s current screen into a copy-mode screen and starts a virtual cursor at the live cursor position. From that instant the keys you press are looked up in the `copy-mode` or `copy-mode-vi` key table (Chapter 9) rather than sent to the program — which is why `j` and `k` move the copy cursor under vi keys, and `C-Space` starts a selection under emacs keys. The program underneath keeps running and its grid keeps updating; you are navigating a view, not pausing the world.

The vi-versus-emacs split is not a special case in the copy-mode code — it is just which key table is active, set by the `mode-keys` option. `bind-key -T copy-mode-vi v` adds a vi-mode binding; the same motions dispatch to the same `window_copy_*` functions regardless. This is the payoff of the table-driven key system: copy mode inherits two full keymaps for free, and a user’s custom copy-mode bindings compose with the defaults exactly as under tmux.

The grid reader: motions over stored lines

The virtual cursor’s movements are the `grid_reader` — a `gd` grid pointer plus a `cx/cy` position — and `src/grid_reader.rs` holds the motion vocabulary, each a faithful port of tmux’s `grid-reader.c`. `grid_reader_start` initializes it; then the motions: `grid_reader_cursor_left/right/up/down` for single steps, `grid_reader_cursor_start_of_line/end_of_line` for line ends, `grid_reader_cursor_back_to_indentation` for the first non-blank, `grid_reader_cursor_next_word/previous_word/next_word_end` for word motion, `grid_reader_cursor_jump/jump_back` for in-line character search (the `f/F/t/T` family), and `grid_reader_line_length` and `grid_reader_in_set` as the primitives the others build on. Two details make these faithful rather than approximate. `grid_reader_handle_wrap` decides whether moving off the end of a line advances to the next line or stops, honoring the wrapped-line flag so a soft-wrapped long line is navigated as one logical line. And word motion consults the `word-separators` option through `grid_reader_in_set`, so what counts as a “word” is exactly tmux’s configurable set, not a hard-coded `\w`.

The reader also seamlessly crosses the boundary between the visible screen and the scrollbar above it, because it walks the grid’s full coordinate space including history — which is why scrolling up in copy mode flows smoothly from the current screen into stored lines with no seam. `window_copy_scroll_up/_scroll_`

`down/_scroll_to` move the viewport over that space, and the copy cursor's `cy` can range across the whole `hsize + sy` of stored lines.

Selection and extraction

A selection is a `screen_sel` on the copy-mode screen — `sx/sy` start, `ex/ey` end, a `rectangle` flag for block versus stream selection, and a cell template for how selected cells are drawn (Chapter 5). As the cursor moves with a selection active, `window_copy_update_selection` grows it, and the renderer calls `screen_check_selection(px, py)` for every cell to decide whether to draw it highlighted. Block selection (the rectangle flag, toggled by `v` under vi keys) selects a column range across multiple lines rather than a stream from start to end — the same distinction tmux makes, ported exactly.

Extraction — the “copy” in copy mode — is defined against the grid's string functions. `window_copy_copy_selection` walks the selected region and builds a string through `grid_string_cells`, whose `grid_string_flags` decide whether the copy preserves styling (embedding the SGR escape sequences via `grid_string_cells_fg/_bg/_us/_code`) or strips to plain text, and whether trailing spaces are trimmed. The result goes into a paste buffer (`src/paste.rs`), and the sibling `window_copy_copy_pipe` and `window_copy_copy_line` route it instead to a pipe command or a single line. This is why a copy-mode selection can round-trip color and attributes when you want it to and come out clean when you do not: the extractor can regenerate the exact escape sequences that produced each cell, or omit them, on a flag.

Search over the grid

Incremental search in copy mode (`/` and `?` under vi keys) is a scan over the grid, and `window_copy_search` with its `window_copy_search_compare` and `window_copy_search_back_overlap` helpers implement it. A search walks lines comparing against the query — plain or, with the regex option, as a pattern — and moves the copy cursor to the match, tracking the current-match and all-matches highlight styles (`copy-mode-current-match-style`, `copy-mode-match-style`, `copy-mode-mark-style` — real options in the table). Backward search and the overlap handling exist so a match spanning a wrap boundary, or overlapping the current position, is found correctly. Because search is just a grid scan, it searches the scrollbar too, which is the whole point: copy mode's search is how you find a line that scrolled off the screen a thousand lines ago.

Copy mode as proof that the grid is a data model

The recurring theme of Chapter 5 returns here fully realized: the grid is not merely a buffer that gets painted, it is a queryable, navigable, extractable data structure, and copy mode is the feature that exercises every one of those verbs. Navigation is the grid reader; query is search; extraction is `grid_string_cells`; selection is `screen_sel`. None of these required new storage — they are all operations on the same grid the input parser fills and the tty back end draws. That is why copy mode can scroll into history without a seam, preserve styling on copy, and search a thousand lines of scrollbar: it is not a separate subsystem bolted on, it is the grid used as what it always was — a model, not just a picture.

Chapter 19 – Spawning Panes and Windows

Every pane runs a real program in a real pseudo-terminal, and getting from a `split-window` command to a running shell in a new `pty` is the job of `src/spawn.rs`. It is a small module — `spawn_window`, `spawn_pane`, and the logging helper `spawn_log` — but it is where the server crosses from managing abstract state into managing operating-system processes, and it is the site of one of the three headline parity bugs (Chapter 11). This chapter walks the spawn path and the fork/exec dance that a faithful port has to get exactly right.

From command to child: `spawn_window` and `spawn_pane`

`spawn_window` creates a whole new window with its first pane; `spawn_pane` adds a pane to an existing window (the split case). Both take a `spawn_context` — the resolved target, the command to run (or `None` to run the default shell), the working directory, the environment, and flags — and both funnel into the same core work: allocate the window and/or pane structures, create the pane's screen and grid (Chapter 5), open a pseudo-terminal, fork a child, and in the child set up the environment and `exec` the program. `spawn_pane` is what `split-window` and `respawn-pane` call; `spawn_window` is what `new-window`, `new-session`, and `respawn-window` call. The `respawn_*` variants reuse an existing pane whose program has exited, which is why `remain-on-exit` can leave a dead pane in place and `respawn-pane` can bring it back to life in the same slot.

The environment handed to the child is assembled, not inherited raw. The `update-environment` option names the variables copied from the attaching client into the session environment (`src/envron.rs`), so that `DISPLAY`, `SSH_AUTH_SOCK`, and the rest reflect the client you attached from rather than the stale values the daemon was started with. `spawn_pane` merges the session environment, the window's overrides, and the per-spawn additions into the child's environment, and sets `TMUX` (well, the `ztermux` equivalent) so a program inside a pane can tell it is running under the multiplexer. This is faithful `tmux` behavior and it is why re-attaching from a new SSH session and opening a new window gets the *new* session's `SSH_AUTH_SOCK`.

The pseudo-terminal and the fork/exec window

A pane is backed by a `pty` pair: the server holds the master fd (watched by `libevent` — when the child writes, an event fires and the bytes flow to the input parser, Chapter 6), and the child runs with the slave fd as its controlling terminal. Between `fork` and `exec` the child does the delicate work that must happen in the child but before the new program takes over: it puts itself in a new session (`setsid`), makes the `pty` slave its controlling terminal, sets the terminal size to the pane's size, redirects `stdin/stdout/stderr` to the slave, restores default signal handling, and closes the inherited descriptors it must not leak to the child program.

That last step — closing inherited descriptors — is where the macOS parity bug lived, and it is worth retelling from the spawn side because it shows how load-bearing “faithful” is even in a corner this deep. The child called `closefrom` to close every descriptor at or above a low number, and the port's `closefrom` looped `close()` from that number up to `RLIMIT_NOFILE`. On the server, `RLIMIT_NOFILE` is raised to a huge value, so on macOS the child spun through *millions* of `close()` calls and hung before it could `exec` — leaving `{pane_current_command}` reporting `ztermux` (the not-yet-replaced server binary) instead of the real command. The parity case `294_pane_cmd.fmt` caught it, passing on Linux and failing on macOS, and the fix ports `tmux`'s macOS-specific `closefrom` that enumerates the actually-open descriptors with `proc_pidinfo(PROC_PIDLISTFDS)`

and closes only those. The child reaches `exec` immediately, and the format reports the true command. A whole-second hang between fork and exec, invisible on one platform, surfaced only because the parity suite runs cross-platform and compares bytes.

Reaping children and pane death

The other end of a pane's life is handled by the server's `SIGCHLD` path (Chapter 3): `server_child_signal` dispatches to `server_child_exited` or `server_child_stopped`, which find the pane by pid and mark it dead (recording the exit status and signal, exposed as `#{pane_dead}`, `#{pane_dead_status}`, `#{pane_dead_signal}`) or continued. What happens to a dead pane is policy: by default the pane and, if it was the last, the window are destroyed; with `remain-on-exit` set, the dead pane stays, its `remain-on-exit-format` shown, ready for `respawn-pane`. The whole lifecycle — spawn a child into a pty, watch its master fd, reap it on exit, apply the death policy — is the server acting as a process manager, and `spawn.rs` is the front door where new processes enter that management.

Chapter 20 – The Status Line and Screen Redraw

The visible frame a tmux user stares at all day is drawn by two subsystems working together: `src/screen_redraw.rs`, which composites the panes and their borders into the client's terminal, and `src/status.rs`, which builds the status line at the top or bottom. Both are downstream of everything the earlier chapters built — the grid supplies the pane contents, the layout supplies the rectangles, the format engine supplies the status text — and both are heavily driven by options and formats, which makes them a good closing look at how the faithful core turns state into pixels.

Screen redraw: compositing panes and borders

`screen_redraw_screen` is the top-level draw for a client: it walks the window's visible panes and, for each, calls `screen_redraw_draw_pane` to blit that pane's grid into its rectangle, then `screen_redraw_draw_borders` to draw the lines between panes. The border drawing is more subtle than it looks, because a border cell's glyph depends on its neighbors — a `+` where four panes meet, a `T` where three do, straight lines elsewhere — and `screen_redraw_cell_border` and `screen_redraw_check_cell` compute, per cell, which border character and style belong there. The active pane's border can be styled differently (`pane-active-border-style`), and the border can carry a per-pane status line (`pane-border-status`, drawn by `screen_redraw_draw_pane_status`) formatted from `pane-border-format`. This is why a tmux screen's borders “know” the shape of the layout: each border cell is computed from the geometry the layout tree (Chapter 7) defines.

Redraw is differential wherever it can be, tying back to Chapter 10's output path. The screen-write layer marks cells dirty as the grid changes; the redraw walks only what changed and hands minimal `tty_cmd_*` operations to the tty back end, which turns them into the shortest terminal sequences. A full redraw (`screen_redraw_check_is`) decides when one is needed — a resize, a mode change, a client refresh) repaints everything; the common case repaints a few cells. The status line, pane contents, and borders are all composited into the same client terminal through the same back end, per client, so two clients of different sizes each get a correct frame (Chapter 4's multi-client story, seen from the draw side).

The status line: `status.rs` and the format-driven bar

The status line is built by `status_redraw`, and it is almost entirely a format-expansion exercise. The bar has a left section (`status-left`, width-capped by `status-left-length`), a right section (`status-right`, `status-right-length`), and a middle list of window-status entries — one per window, formatted by `window-status-format`, with the current window using `window-status-current-format` and separators from `window-status-separator`. Every one of those is a `#{...}` format string expanded through Chapter 16's engine against the relevant session/window context, then laid out by `status_redraw` honoring `status-justify` (left/centre/right/absolute-centre) and `status-position` (top or bottom). `status_line_size` and `status_at_line` compute how many rows the status occupies (it can be multi-line) and where it sits, so the pane area and the status area never overlap.

Because the whole bar is formats, everything Chapter 16 described is available in it: a default `status-right` like `#{?client_prefix,^B ,}"#{=21:pane_title}" %H:%M` uses a conditional (highlight when the prefix is pending), a width limiter, and — where time is allowed — a strftime format. `format_draw.rs` is the piece

that renders an *already-expanded* status string with its embedded `#[...]` style directives into actual cells, handling the style ranges (`STYLE_RANGE_LEFT`, `STYLE_RANGE_WINDOW`, and the rest) that let a click on a window-status entry know which window it hit. The style-range machinery is what makes the status line *interactive* under the mouse, not just decorative.

Prompts, messages, and the status line's other lives

The status line row is time-shared with two other uses, and `status.rs` handles both. `status_message_redraw` shows the transient messages that `display-message` and command errors produce — the yellow bar that appears for `display-time` milliseconds and then reverts. `status_prompt_redraw`, with `status_prompt_key`, `status_prompt_update`, and `status_prompt_set`, implements the interactive command prompt (`command-prompt`, the prefix-`:` line) and the `confirm-before` and `command-prompt -p` prompts — reading a line of input in the status row with its own editing keys (`status-keys`, emacs or vi), history (`status_prompt_save_history`, persisted so your prompt history survives a server restart — one of the cleanup steps Chapter 3 named), and completion. So the single status row is, at different moments, an information display, a menu of windows, a transient message, and a line editor — three subsystems multiplexed onto one strip of terminal, each a faithful port of its tmux counterpart, and all of them ultimately drawing through the same grid-and-ty path as everything else in the program.

Chapter 21 — Hooks, Alerts, and Notifications

tmux is event-driven not only at the socket level but at the *semantic* level: when a window is renamed, a session is created, a pane changes mode, or a bell rings in a background window, tmux fires a named event that user configuration can hook and that attached clients can be told about. This chapter covers that layer — `src/notify.rs`, `src/alerts.rs`, and the hook machinery threaded through the command queue — because it is where a user's `.tmux.conf` reaches past static options into *reactive* behavior, and because it ties together the command queue (Chapter 8), options (Chapter 9), and the status line (Chapter 20).

The notification system: named events

`src/notify.rs` is the dispatcher for tmux's internal events. When something noteworthy happens, the code calls one of the typed `notify_*` entry points — `notify_session`, `notify_window`, `notify_pane`, `notify_client`, `notify_winlink`, `notify_paste_buffer`, `notify_session_window` — with the name of the event and the objects it concerns. Each queues a notification through `notify_add`, whose callback (`notify_callback`) does two things: it runs any hook bound to that event name, and it tells interested clients (control-mode clients especially) that the event occurred. The event names are a fixed vocabulary, and reading the list is reading tmux's model of “things worth reacting to”: `session-created`, `session-closed`, `session-renamed`, `session-window-changed`, `client-detached`, `client-session-changed`, `window-linked`, `window-unlinked`, `window-renamed`, `window-pane-changed`, `window-layout-changed`, `pane-mode-changed`, `paste-buffer-changed`, and `paste-buffer-deleted`.

The reason this indirection exists rather than each site directly doing the work is uniformity: every state change that matters flows through one dispatcher, so hooks and control-mode notifications compose the same way regardless of what triggered them. A control-mode client (Chapter 17's `-CC`) subscribes to these events to keep its own model of the server in sync — which is exactly how the `zterminal` integration of Chapter 13 would learn that a window was renamed without polling. The notification layer is the server's “something changed” broadcast, and `ztermux` ports the whole vocabulary and the dispatch.

Hooks: commands bound to events

A *hook* is a command list bound to run when an event fires — the reactive counterpart to a key binding (which runs on a keystroke). Hooks are set with `set-hook` (a sibling entry in `src/cmd_set_option.rs`, alongside `set-option`) and shown with `show-hooks`, and they live in the options tree as a special option kind, which is why they inherit and scope like options do. When `notify_hook` (or `notify_insert_hook/notify_insert_one_hook`) fires an event, it looks up the hook of that name in the relevant scope and, if present, queues its command list on the command queue with the event's context as the queue-item state (Chapter 8) — so a hook command knows the session, window, or pane the event concerned, and resolves `-t` targets against it.

Two families of hooks are worth naming. The event hooks share the notification names above: bind a command to `window-renamed` and it runs whenever a window is renamed, with the renamed window as its target. The *command* hooks — `after-<command>` and `before-<command>` — fire around the execution of specific commands (the `CMD_AFTERHOOK` flag on a `cmd_entry`, Chapter 8, governs the after-hook timing), so `set-hook -g after-split-window '...'` runs your command every time any `split-window` completes.

Because hooks are command lists on the command queue with resolved context, they compose with everything else: a hook can run a brace block, branch with `if-shell`, set an option, or send keys, exactly as a key binding can. This is how a `.tmux.conf` turns from a static configuration into a small reactive program.

Alerts: activity, bell, and silence

`src/alerts.rs` handles the three background-window alerts a user actually sees: *activity* (a background window produced output), *bell* (a program rang the terminal bell), and *silence* (a background window produced no output for a configured interval — the inverse alert, useful for watching a build that has stalled). The monitoring is gated per window by `monitor-activity`, `monitor-bell`, and `monitor-silence` (Appendix C), and when a monitored condition is met, `alerts_queue` schedules a check that `alerts_callback` services on the event loop. The check functions — `alerts_check_activity`, `alerts_check_bell`, `alerts_check_silence`, gathered by `alerts_check_all` and `alerts_check_session` — decide whether the condition has actually fired for a window and, if so, apply the configured *action*.

The action is where alerts meet the rest of the configuration surface. `activity-action`, `bell-action`, and `silence-action` (session options) choose *which* clients are alerted — any, current, or none, gated by `alerts_action_applies` — and `visual-activity`, `visual-bell`, and `visual-silence` choose *how*: a visible message in the status line (`alerts_set_message`, drawn by `status_message_redraw` of Chapter 20) versus an actual terminal bell versus both. When an alert fires it also sets the window's alert flags (`window_activity_flag`, `window_bell_flag`, `window_silence_flag`, Appendix B), which the `window-status-*`-style options then use to restyle that window's entry in the status line — so an active background window can glow in the status bar. `alerts_reset` and `alerts_reset_all` clear the state when the window is next viewed. The whole path — monitor option, condition check, action policy, visual/bell delivery, status-flag restyle — is a faithful port, because “which windows light up when and how” is exactly the kind of long-established behavior tmux users depend on.

Why this layer completes the picture

Hooks, alerts, and notifications are the reactive skin over the state machine the rest of the book described. The static layers — options, key tables, formats — configure what tmux *is*; this layer configures what tmux *does when things happen*. And it reuses every mechanism already built: hooks are command lists on the command queue with resolved context; alerts are options-gated checks that deliver through the status line; notifications are the broadcast that control-mode clients and the superset's integrations subscribe to. Nothing here is a new subsystem so much as the composition of the existing ones into event-driven behavior — which is precisely why porting it faithfully mattered: a hook that fired at the wrong time, or an alert that applied to the wrong clients, would break configs that have quietly relied on the exact semantics for years.

Chapter 22 — Two Lifecycles: A Keystroke and a Pane

The book has walked the subsystems one at a time; this capstone walks *through* all of them twice, following two end-to-end journeys — the life of a single keystroke, and the life of a pane from birth to death. Both cross nearly every module named so far, and tracing them is the best way to see how the faithful pieces compose into the whole machine. Nothing here is new mechanism; it is the existing mechanism in motion.

The life of a keystroke

You press `␣-b c` to make a new window. Follow the bytes. Your terminal sends the raw byte for `␣-b` to the ztmux *client*, which is in raw mode (Chapter 4) and forwards every byte to the server over the socket — the client decides nothing. On the server, the bytes arrive on the client's input and go to `tty_keys.rs`, which decodes the terminal's byte sequence into a tmux *key code* (`␣-b` becomes the internal code for control-B). The server looks that key up in the client's active key table — normally `root`, but because `␣-b` is the prefix key, the lookup instead switches the client's key table to `prefix` and consumes the key: the client's `client_key_table` is now `prefix`, which is exactly what `#{client_prefix}` reports and what the status-line recipe in Appendix J highlights.

Now you press `c`. Its byte is forwarded, decoded by `tty_keys.rs` into the key code for `c`, and looked up — this time in the `prefix` table, where `key_bindings_get` finds the default binding added by `key_bindings_init` (Chapter 9, Appendix I): `c` maps to the command list `{ new-window }`. That command list is dropped onto the command queue (Chapter 8) as a `cmdq_item` carrying a `cmdq_state` built from the resolved context — the client that pressed the key, and through it the current session, window, and pane (Chapter 8's target resolution). The client's key table resets to `root`.

The server's event loop (Chapter 3) pumps the queue on its next iteration: `cmdq_next` fires the item, which runs `new-window's exec` (`src/cmd_new_window.rs`). That resolves its target through `cmd_find`, then calls into `spawn.rs` (Chapter 19): `spawn_window` allocates a window and its first pane, creates the pane's screen and grid (Chapter 5), opens a pty, forks a child, and in the child sets up the environment (Chapter 19's `update-environment` merge) and `execs` your shell. The new window is inserted into the session's window list; the layout engine (Chapter 7) gives its single pane the full window rectangle. A `window-linked / session-window-changed` notification fires (Chapter 21), any hook bound to it runs, and the status line is marked for redraw.

Finally the frame comes back. The server recomputes the visible state, the screen-redraw layer (Chapter 20) composites the new window's pane and the updated status line, and the tty back end (Chapter 10) emits the minimal escape sequences — through the differential redraw and `tty_cursor/tty_attributes` minimization — to the client's borrowed terminal fd. The client writes those bytes to your actual screen. One keystroke pair crossed the client, the socket, the key decoder, two key tables, the command queue, target resolution, the spawn path, the pty and fork/exec, the layout engine, the notification system, and the redraw and tty layers — and every one of those was a faithful port of the corresponding tmux module. That traversal *is* the architecture.

The life of a pane

Now widen the lens from an instant to a whole lifetime. A pane is born when a command that spawns one — `new-window`, `split-window`, `new-session` — reaches `spawn_pane/spawn_window` (Chapter 19). Its screen and grid are created (`screen_init` → `grid_create`), sized to the rectangle the layout tree assigns it, and a `pty` is opened with the server holding the master fd. The child forks, does its between-fork-and-exec setup (new session, controlling terminal, size, descriptor closing — the macOS `closefrom` corner of Chapter 19), and `execs` a program. The server arms a libevent watch on the master fd. The pane is alive.

Through its life the pane is a pipe between a program and a grid. When the program writes, the master-fd event fires (Chapter 3), the bytes are read and fed to `input_parse` (Chapter 6), which drives the screen-write layer (Chapter 5) to mutate the grid — printing characters through the batched collect path, moving the cursor, scrolling the region, pushing lines into scrollback when they fall off the top (`grid_scroll_history`, trimmed by `grid_collect_history`). The changed cells are marked dirty; on the next redraw the `ty` layer paints only those cells to every client viewing the pane. If you enter copy mode (Chapter 18), a virtual cursor and a `grid_reader` navigate the pane's stored lines and scrollback while the program keeps running underneath; a selection extracts styled or plain text into a paste buffer. If you resize the window, the layout engine re-tiles, the pane's screen resizes and its grid reflows long lines to the new width (Chapter 5), and the pane redraws — all without the program noticing more than a `SIGWINCH`.

A pane can move without dying. `break-pane` promotes it into its own window, `join-pane` grafts it into another window's split, `swap-pane` exchanges it with another — all tree surgery on the layout (Chapter 7) that preserves the pane's screen, scrollback, and running program; only its rectangle changes. It can be zoomed to fill the window and un-zoomed back to the exact column it left, because zoom saves and restores the layout tree rather than rebuilding it (`window_zoom/window_unzoom`).

The pane dies when its program exits. The server's `SIGCHLD` path (Chapter 3) reaps the child and marks the pane dead, recording the exit status and signal (`#{pane_dead_status}`, `#{pane_dead_signal}`). Policy decides what happens next: by default the pane is destroyed and, if it was the last in its window, the window is closed and the layout collapses the freed space back to its siblings; with `remain-on-exit` set, the dead pane stays in place showing its `remain-on-exit-format`, ready for `respawn-pane` to bring it back to life in the same slot. When the last window of the last session is gone and the `server_loop` exit conjunction is finally satisfied (Chapter 3), the server itself exits — and the pane's whole lineage, from `grid_create` to the reaped child, has been a single coherent walk through the ported machine. The two lifecycles together are the book's argument in motion: a faithful port is faithful not module by module in isolation, but in the way the modules compose into the behavior a `tmux` user has always known.

Chapter 23 – Performance and the Hot Paths

A multiplexer sits in the path of every byte a program prints and every key you type, so it has to be fast, and tmux is — through a handful of specific optimizations in its hottest paths. Because ztmux ports those paths line for line rather than reinventing them, it inherits the performance characteristics along with the behavior. This chapter names the hot paths and the optimizations that make them fast, and explains why a faithful port has to keep them exactly rather than substitute a “more idiomatic” approach that might dispatch differently.

The character-printing fast path

The single most-traveled path in the program is a program printing text to a pane, and it is optimized end to end. When bytes arrive, `input_parse` (Chapter 6) special-cases the printable range so a run of ordinary characters never pays the collection-flush overhead of a control sequence. Printable characters route to `input_print`, which does *not* store each character in the grid individually; it accumulates them and hands the run to `screen_write_collect_add`, which batches them, and the batch is committed by `screen_write_collect_end` only when a non-print sequence arrives or the run must flush. The active SGR style rides along in the parser’s `input_cell`, so each batched character is stamped with the current color and attributes without re-deriving them. The result is that *cat*-ing a megabyte file through a pane turns a stream of thousands of printable bytes into a handful of batched grid writes, not thousands of individual cell stores. This batching is tmux’s, ported exactly, because it is not just a speed trick — the *order* in which the collected run is flushed relative to control sequences is observable, so a different batching strategy could change output.

Bounded reallocation and the packed cell

Two memory optimizations from Chapter 5 are performance choices as much as space choices. Grid lines grow through `grid_expand_line`, which snaps the allocation up through a bounded sequence of sizes — a quarter of the screen width, then half, then full — so a line that fills up reallocates a small, bounded number of times rather than once per character, turning what could be $O(\text{width})$ reallocations into $O(\log \text{width})$. And the packed/extended cell split (the five-byte packed `grid_cell_entry` for the common case, promotion to a `grid_extd_entry` only when needed) keeps a screenful of ordinary text compact, which means better cache behavior and less memory traffic on the redraw walk. Both are exact ports, because the allocation pattern is part of the behavior the parity discipline pins — but they are also why a grid of ten thousand scrollbar lines does not cost ten thousand full-width allocations.

Differential redraw and output minimization

The output path (Chapter 10, and the cell-by-cell walk earlier in this book) is where tmux spends the most optimization effort, because bytes on the wire are the bottleneck: an escape sequence to position the cursor and paint a cell is far larger than the cell. The server repaints only cells the screen-write layer marked dirty; `tty_cursor` emits the shortest cursor move from the tracked current position (a carriage return, a single “cursor right,” or an absolute `CUP` only when nothing cheaper reaches the target); and `tty_attributes` emits SGR only for the differences from the previously drawn cell. The `tty_cmd_*` family turns high-level changes

(a scrolled line, a cleared region) into the fewest terminal operations. Across a full-screen redraw of a busy editor these micro-optimizations are the difference between a responsive terminal and a laggy one, especially over a slow link — and ztmux ports them because the emitted byte stream is itself observable and, in principle, comparable.

Flow control: surviving a flood

A program can produce output faster than a terminal can consume it — a `yes` loop, a giant `cat` — and an unbounded output buffer would let one runaway pane exhaust memory and freeze the whole terminal. tmux applies flow control, and ztmux ports the exact thresholds. The tty's output goes into a libevent `evbuffer` drained to the client when writable; when the buffered bytes exceed `TTY_BLOCK_START` (computed as $1 + (sx * sy) * 8$, a multiple of a screenful), the tty enters a blocked state, stops producing more output, and arms a `TTY_BLOCK_INTERVAL` (100 ms) timer to resume. The runaway pane is throttled, the rest of the terminal stays responsive, and memory stays bounded. The specific constants matter — they are observable timing behavior — so the port keeps them rather than picking its own.

Housekeeping: the tidy timer and returning memory

Finally, the server does periodic housekeeping so long-lived sessions do not accumulate waste. The hourly tidy timer (`SERVER_EV_TIDY`, an `evtimer` set to 3600 seconds, whose callback `server_tidy_event` fires from the event loop) expires stale format jobs and, off macOS, calls `malloc_trim(0)` to return freed heap back to the operating system — so a server that once held a huge scrollbar and released it actually gives the memory back rather than holding the high-water mark forever. This is a small, faithful touch (the macOS build skips `malloc_trim`, which glibc provides and the macOS allocator does not), and it is the kind of long-run hygiene a daemon that may live for weeks needs. Performance in a multiplexer is not one clever algorithm; it is a dozen small, exact decisions in the hottest paths, and porting them faithfully is how ztmux is fast for the same reasons tmux is.

Chapter 24 – Colour: Names, Spaces, and the Nearest Match

Chapter 5 showed how a cell *stores* colour — a tagged `i32` that encodes an ANSI index, a 256-palette entry, or a 24-bit RGB value in one field. This chapter is about the machinery that sits underneath that storage: `src/colour.rs` (a port of tmux's `colour.c`), which turns colour *names* into those tagged integers, converts between the three colour spaces, finds the nearest palette entry when a terminal cannot show a true colour, and applies a window's palette overrides. It is small, unglamorous, and load-bearing — every `#[fg=...]` style directive, every `fg/bg` option, and every degraded-colour redraw passes through it.

Three colour spaces in one integer

The colour field's encoding is the fact everything else in the module is organized around, so it is worth restating precisely. A colour is an `i32`. The low values `0–7` and the bright `90–97` (aixterm) forms are direct ANSI palette indices; `8` is `COLOUR_DEFAULT` (the terminal's own default, and `9` is treated as default too); a value or-ed with `COLOUR_FLAG_256` (`0x01000000`) carries a 256-colour palette index in its low byte; and a value or-ed with `COLOUR_FLAG_RGB` (`0x02000000`) packs a 24-bit colour as `(r << 16) | (g << 8) | b`. The two flag bits are mutually exclusive and sit above any real index, so a single test on the tag bits tells `colour.rs` which space a value lives in before it unpacks anything.

The pack and unpack helpers are the primitives the rest of the module builds on. `colour_join_rgb(r, g, b)` folds three bytes into the RGB-tagged form, and `colour_split_rgb(c, &r, &g, &b)` reverses it, masking off the flag and shifting the components back out. These are trivial bit arithmetic, but they are *exactly* tmux's bit arithmetic — the same shift amounts, the same flag constants — because the packed value is not private: it is what gets stored in a grid cell (Chapter 5), passed through a format's `{c:...}` operator (Chapter 16), and written into an option, so any divergence in the packing would be observable downstream. The module keeps the C's representation to the bit.

Naming: `colour_fromstring` and `colour_tostring`

The bridge between the human world of config files and the integer world of the grid is a pair of functions. `colour_fromstring` parses a colour *name* or *spec* into the tagged integer, and it accepts the full tmux vocabulary: the bare ANSI names (`black`, `red`, ..., `white` and their `bright*` cousins), the numbered forms `colour0` through `colour255` (with the British spelling tmux uses, and `color/grey` variants accepted), the hex spec `#rrggbb`, the keyword `default`, and the large table of X11/web colour names (`orange`, `slateblue`, `darkolivegreen`, and the rest) that newer tmux added so a style string can name a colour the way CSS does. A `#rrggbb` parses through `colour_join_rgb`; a `colourN` becomes `N | COLOUR_FLAG_256`; a name is looked up in `colour_byname`, which returns the RGB-tagged value for the web names and the small index for the ANSI ones.

`colour_tostring` is the inverse, and it is what `show-options` and `{c:...}` print. Given a tagged integer it produces the most faithful string form: an RGB value comes back as `#rrggbb`, a 256-palette entry as `colourN`, an ANSI index as its name (`red`) or `colourN`, and `8` as `default`. The round-trip matters — a colour set as `#d75f5f` should read back as `#d75f5f`, and one set as `colour167` should read back as `colour167` — because configs and scripts compare these strings, and tmux's exact spelling (British `colour`, lowercase

hex, the specific name chosen for each ANSI index) is what a parity `.fmt` case pins. `colour.rs` ports the tables and the formatting so the strings are identical to the byte.

The nearest match: `colour_find_rgb` and `colour_256to16`

The most interesting function in the module is the one that runs when a terminal *cannot* show what the grid holds. When a cell carries a true 24-bit RGB colour but the client's terminal is only 256-colour (Chapter 10's degrade-late principle), the back end must approximate, and `colour_find_rgb(r, g, b)` computes the approximation: the index of the nearest colour in the xterm 256-colour palette. The algorithm is tmux's exact one, and it is cleverer than a linear scan. The 256-palette's colour cube is a 6×6×6 grid whose axis values are not evenly spaced but the specific ramp `{0x00, 0x5f, 0x87, 0xaf, 0xd7, 0xff}`; `colour_to_6cube` maps each input component to its nearest ramp step, giving a candidate cube colour, and separately the input is matched against the palette's 24-step greyscale ramp. The function then compares the squared distance from the input to the cube candidate and to the grey candidate and returns whichever is closer, or-ed with `COLOUR_FLAG_256`. Picking between the cube and the grey ramp is why a near-grey RGB value degrades to a clean grey rather than a muddy cube colour.

`colour_256to16` is the next step down, a straight table lookup from a 256-palette index to the nearest of the sixteen ANSI colours, used when even 256-colour is unavailable. The reason both of these are ported byte-for-byte rather than reimplemented with “a reasonable nearest-colour metric” is the recurring one: the *approximation itself is observable*. Two multiplexers that both “approximate reasonably” but choose different palette entries for the same RGB value produce visibly different output on a 256-colour terminal, and a user who tuned their theme against tmux's approximation would see it shift. The specific ramp values, the cube-versus-grey tie-break, the 256 → 16 table — all of it is tmux's, so ztmux degrades a colour to the *same* index tmux would, and the parity discipline can in principle diff the resulting SGR bytes.

Palettes: per-window overrides and OSC 4

The last piece is the *palette* — the per-window remapping of the 256 base colours that OSC 4 (Chapter 6) and the `pane-colours`/palette options install. A program inside a pane can redefine what `colour4` means by emitting an OSC 4 sequence, and a user can set palette entries through options; both feed a `colour_palette` structure that the back end consults before emitting a colour. `colour_palette_init` and `colour_palette_clear` manage its lifetime, `colour_palette_set(palette, n, c)` installs an override for index `n`, and `colour_palette_get(palette, c)` resolves a colour through the overrides — returning the remapped value when the palette has an entry for it and the original otherwise. The default foreground and background slots are tracked specially, because `fg/bg` “default” must resolve to the window's configured default rather than the terminal's when a palette sets them.

The palette is where the whole colour story closes its loop back to the input parser and the back end. OSC 4 comes *in* through `input.rs` (Chapter 6), lands in the window's `colour_palette`, and then every subsequent draw of a cell holding that palette index resolves through `colour_palette_get` on the way *out* through `tty_colours` (Chapter 10) — so redefining a palette entry retroactively recolours every cell already on screen that used it, exactly as on a real terminal. That end-to-end path, from an escape sequence a pane program emits to the bytes the outside terminal receives, is a colour subsystem in miniature, and `colour.rs` is the piece in the middle that every other stage leans on. It is a good example of a module that is tiny in lines and total in reach: get its nearest-match wrong and every degraded redraw is subtly off; get its name table wrong and every config's colours shift; port it to the bit and colour “just works” for the same reasons it works in tmux.

Chapter 25 – UTF-8, Wide Characters, and Combining Sequences

A grid cell holds a *character*, and in a Unicode world a character is not a byte. This chapter covers `src/utf8.rs` (a port of tmux's `utf8.c`), the module that decodes UTF-8 from the byte stream, measures how many terminal columns a character occupies, packs multi-byte sequences into the compact handle a grid cell stores, and handles the two hard cases every terminal must get right: *wide* characters that occupy two columns and *combining* characters that occupy none. Getting this wrong does not crash anything — it misaligns everything, which is worse, because a status line whose width is off by one and a TUI whose box borders drift are exactly the quiet breakage the parity discipline exists to prevent.

The `utf8_char` handle and the `utf8_data` it stands for

A grid cell cannot afford to store an arbitrary-length UTF-8 sequence inline — most cells hold a single ASCII byte, and Chapter 5's packed/extended split depends on the common cell staying small. So tmux represents a character in a cell as a `utf8_char`: a fixed-size handle (a `u32`) that stands for a UTF-8 sequence rather than containing it in general. The expanded form is `utf8_data` — a small struct carrying the raw bytes (`data`, up to `UTF8_SIZE`), the byte length `size`, and the display `width` (0, 1, or 2). `utf8_from_data` converts a decoded `utf8_data` into the packed `utf8_char` handle, and `utf8_to_data` expands a handle back into its bytes when the back end needs to emit them. Short sequences pack directly into the handle's bits; longer or unusual ones are interned through a lookup table so the handle stays a fixed `u32` no matter how many bytes the character actually took.

This indirection is the reason a cell can carry an arbitrary grapheme without bloating the common case, and it is why the `data` field of a packed `grid_cell_entry` (Chapter 5) can be a single byte for ASCII while a wide CJK glyph or an emoji-with-modifier lives as an extended cell whose `utf8_char` handle resolves through this table. `utf8.rs` ports the packing scheme exactly, because the handle values themselves are internal, but the *behavior* they produce — which sequences round-trip, how a combining mark attaches — is observable through copy-mode extraction and `capture-pane`, and must match tmux.

Decoding: `utf8_open`, `utf8_append`, and the incremental path

The input parser (Chapter 6) does not receive whole characters; it receives bytes, one at a time, and a multi-byte UTF-8 character arrives across several parser iterations. So decoding is *incremental*, driven by three functions. `utf8_open(ud, byte)` starts a new character on a lead byte, reading the high bits to learn how many continuation bytes to expect and stashing the first byte in a fresh `utf8_data`; it returns `UTF8_MORE` if more bytes are needed. `utf8_append(ud, byte)` adds each continuation byte, returning `UTF8_MORE` while the sequence is incomplete, `UTF8_DONE` when it completes (at which point the character's width is computed and it is ready to write to the grid), or `UTF8_ERROR` on a malformed sequence — an unexpected byte, an overlong encoding, a continuation without a lead. The parser holds a partial `utf8_data` across bytes and feeds it through this small state machine, which is why a UTF-8 character split across two reads from the pane's pty decodes correctly rather than tearing.

Error handling here is a fidelity detail that matters more than it looks. A broken UTF-8 stream — and real programs emit them, through bugs or binary output accidentally sent to a terminal — must degrade the *same* way tmux degrades it, not crash and not silently drop bytes, because what a terminal shows for invalid UTF-8 is observable. `utf8.rs` ports tmux’s exact treatment: an invalid sequence is handled deterministically (the offending bytes rendered as tmux renders them) so a pane fed garbage looks under ztmux the way it looks under tmux. The incremental decoder’s three return values and their exact triggers are the ported contract.

Width: the one-flag decisions that keep columns aligned

Once a character is decoded, the single most consequential question is: how many columns does it occupy? `utf8_width` answers it, returning 0 for a combining or zero-width character, 1 for an ordinary one, and 2 for a wide (East Asian fullwidth, or wide emoji) character. The answer drives everything visual: a width-2 character claims its cell *and* a following `GRID_PADDING_CELL` (Chapter 5) so the next real character starts two columns over; a width-0 combining mark does not advance the cursor at all but attaches to the base character already written; a width-1 character is the ordinary case. Get a width wrong and every character after it on the line is one column out of place — the box border of a TUI drifts, a status line’s right-aligned clock lands in the wrong spot, a CJK document renders as a diagonal smear.

The width data is a table — the classification of code points into zero/one/two columns — and tmux ships its own rather than trusting the host’s `wcwidth`, because `wcwidth` varies by platform and locale and would make width a non-portable, non-deterministic quantity. A ztmux built on the C library’s `wcwidth` would report a different width for the same emoji on macOS than on Linux, and the parity suite would light up with false diffs that are really libc differences. So `utf8.rs` ports tmux’s width determination, which is *why* the parity suite can pin `#{w:...}` (the display-width format operator, Chapter 16) across platforms: both binaries consult the same ported table, so the same string is the same number of columns everywhere.

Combining characters and the string helpers

Combining characters are the case that makes “a cell holds a character” subtle. A base letter followed by one or more combining marks — an `e` plus a combining acute accent, a family of emoji joined by zero-width joiners — is *one* grapheme occupying *one* cell, not several. When the parser decodes a width-0 character, it does not write a new cell; it appends the combining sequence to the `utf8_char` of the cell already under the cursor, extending that cell’s stored sequence (which is exactly why such a cell becomes *extended* in Chapter 5’s sense — its byte sequence no longer fits the packed form). `utf8.rs` handles the append, so a decomposed accented character and a precomposed one both end up as a single cell that copies and renders correctly.

Around the decoder sits a family of string helpers the rest of the code leans on, each UTF-8-aware where a naive byte routine would miscount. `utf8_strlen` and `utf8_cstrwidth` measure a string’s length in characters and its width in columns respectively — the distinction Chapter 16’s `#{n:...}` (length) versus `#{w:...}` (width) operators expose. `utf8_padcstr` and `utf8_rpadcstr` pad a string to a column width (not a byte count), which is how status-line fields align regardless of whether they contain CJK. `utf8_rtrimcstr` trims trailing columns. And `utf8_stravis/utf8_strvis` produce a *visible* rendering of a string with non-printable bytes escaped, used where tmux must show a control character rather than emit it. Every one of these is a place a byte-oriented shortcut would silently break on multi-byte input, and every one is ported so that width and length mean, to the column and to the character, what they mean in tmux. The alignment you never think about — because it just works — is this module doing its job.

Chapter 26 — Mouse Events and the Window Modes

Two subsystems that turn tmux from a keyboard program into an interactive one share this chapter because they meet: the *mouse*, which delivers clicks and drags and wheel scrolls to the right pane, and the *window modes*, the takeover screens (`choose-tree`, `choose-buffer`, `customize-mode`, and copy mode) that a pane can enter to become an interactive picker or editor. The mouse is how you point at them; the modes are much of what you point at. Both are faithful ports, and both are where the multiplexer’s UI, such as it is, actually lives.

A mouse click is a key: parsing and routing

The first thing to understand about mouse handling is that, to tmux, a mouse event is just another *key*. The outside terminal, when mouse reporting is on, encodes a click or a scroll as an escape sequence (the modern SGR form, `\x1b[<b;x;yM`, or the legacy X10/1000 forms), and that sequence arrives in the same input stream as the user’s keystrokes. `src/tty_keys.rs` recognizes it — `tty_keys_mouse` parses the button, the column and row, and the press/release/drag/wheel disposition out of the sequence — and packages it as a `key_code` carrying the `KEYC_MOUSE` marker with a `mouse_event` struct attached. From there it flows through the same key-dispatch path as any keystroke (Chapter 9’s key tables), which is the unifying trick: because a mouse event is a key, everything the key system already does — table lookup, binding to a command list, the command queue — applies to the mouse for free.

What makes a mouse event *usable* is that its coordinates must be resolved to an object. A click at column 40, row 12 means nothing until it is turned into “pane 2 of window 3.” The `mouse_event` carries the raw `x/y` (and the previous `lx/ly` for drags), and helpers `mouse_get_pane`, `mouse_get_window`, and the status-line hit-testing translate those coordinates against the current layout (Chapter 7) and status line (Chapter 20) into the pane, window, or status element under the cursor. This is how `MouseDown1Pane` knows which pane you clicked and `MouseDown1Status` knows which window in the status line you selected. The resolution is geometric — it walks the same layout tree the renderer used to place the panes — so the pane the click lands on is exactly the pane drawn there.

The default mouse bindings and forwarding to programs

Because a mouse event is a key, the *default* mouse behavior is expressed as bindings in the `root` key table (Chapter 9), on synthetic key names tmux defines for each event: `MouseDown1Pane` selects the pane clicked, `MouseDown1Pane` begins a copy-mode selection, `WheelUpPane` enters copy mode and scrolls up, `MouseDown1Status` selects a window, `MouseDown1Border` resizes panes, and so on. These are ordinary bindings — you can see them in `list-keys`, rebind them, or unbind them — which is why the mouse is configurable through the same mechanism as everything else and why `bind -n MouseDown3Pane ...` (a right-click binding) is a thing a user can write. The synthetic key names, the default set, and their command lists are all ported so that mouse behavior out of the box is tmux’s.

There is a crucial second path: a program *inside* a pane can itself request mouse reporting (an editor, a pager, a full-screen app that wants clicks). When it has, and the click lands in that pane, tmux does not act on the event — it *forwards* it, re-encoding the mouse event as the escape sequence the inner program expects and

writing it into the pane (through the `input_key` path, the outbound cousin of Chapter 6’s parser). So a click in `vim-with-mouse` goes to `vim`, while a click in a plain shell pane is handled by tmux’s own bindings. The `mouse` option gates the whole system (off by default in stock tmux, though most configs turn it on), and the decision of whether an event is tmux’s or the program’s — based on the pane’s mode and whether the inner program enabled reporting — is ported exactly, because “does my click go to tmux or to the app?” is behavior a user builds precise expectations around.

Window modes: a pane that becomes a UI

A *window mode* is what a pane enters when it stops showing its program and starts showing an interactive tmux surface instead. Copy mode (Chapter 18) is the most familiar, but it is one of several, and they share a structure. Each mode is a `window_mode` — a struct of function pointers (`init`, `free`, `resize`, `key`, and a `key_table` naming the key bindings that apply while the mode is active) — and a pane enters one through `window_pane_set_mode` and leaves it through `window_pane_reset_mode`. While a mode is active, the pane’s screen is drawn by the mode, not the program: the program keeps running underneath (its output still mutates its real screen), but what you see is the mode’s own screen, and keystrokes are dispatched through the mode’s key table rather than sent to the program. This is the same policy/mechanism separation the rest of the book keeps running into: the mode is a temporary alternate view and input handler bolted onto a pane.

The modes beyond copy mode are the interactive pickers, and tmux builds them on a shared toolkit — `mode-tree.c`, ported as the mode-tree helper — that provides a scrollable, filterable, taggable tree with a live preview. `choose-tree` (the `window_tree` mode) presents the whole session/window/pane hierarchy as a navigable tree and switches to whatever you select; `choose-client` (`window_client`) lists attached clients; `choose-buffer` (`window_buffer`) lists the paste buffers (Chapter 28) with a preview of each; and `customize-mode` (`window_customize`) is an interactive options-and-keys editor, letting you browse and change the scoped options (Chapter 9) and key bindings live. Because all four are built on the one mode-tree toolkit, they share their navigation keys, their `/`-search, their tagging for bulk actions, and their preview pane — which is why they feel like one interface, and why porting the toolkit once ports the consistency across all of them.

Why the modes and the mouse belong together

The mouse and the modes are the two halves of the same interactive loop, and a worked example shows them meeting. Scroll the wheel over a pane: the terminal sends a wheel escape, `tty_keys.rs` parses it into a `WheelUpPane` key with resolved coordinates, the `root` binding for `WheelUpPane` fires, and its command puts the pane into copy mode and scrolls up one step — a mouse event, routed as a key, entering a window mode. Now drag to select: each `MouseDownPane` event, coordinates resolved to grid cells, extends the copy-mode selection (Chapter 5’s `screen_sel`). Release, and the selection is copied into a paste buffer (Chapter 28). One gesture traversed the mouse parser, the key tables, a window mode, the grid’s selection model, and the buffer store — every stage a faithful port of the corresponding tmux piece.

That traversal is the argument for why these subsystems are ported rather than reinvented as “reasonable mouse support.” A reimplementation would almost certainly resolve a click to a slightly different pane at a border, or scroll a different number of lines per wheel notch, or bind the middle button somewhere else — small divergences that a mouse-driven tmux user would feel immediately as *wrong*, even if they could not say why. By porting the coordinate resolution, the synthetic key names, the default bindings, the forwarding decision, and the mode toolkit, ztmux inherits tmux’s exact feel under the mouse, the same way it inherits tmux’s exact behavior under the keyboard. The interactive surface is the part of a multiplexer users touch most directly, and faithfulness there is what makes the switch from tmux feel like nothing changed.

Chapter 27 — Jobs, run-shell, and pipe-pane

tmux runs other programs — not the ones in panes, which are the whole point, but *helper* processes: the shell command behind `run-shell`, the condition behind `if-shell`, the `#(...)` job in a status line, the sink of `pipe-pane`. All of them go through one small subsystem, `src/job.rs` (a port of tmux's `job.c`), which forks a child, wires its I/O to the event loop, and calls back when output arrives or the child exits — without ever blocking the server's single thread. This chapter is about that subsystem and the four features built on it, because “multiplexer that can shell out asynchronously” is a surprising amount of what makes tmux scriptable.

job_run: a child wired into the event loop

The core is one function, `job_run`, and its signature tells the story: it takes a command to run (as a shell string or an argv), an environment and working directory and originating session, and a set of *callbacks* — an update callback fired as the child produces output, a complete callback fired when it exits, and a free callback for teardown — plus flags and an initial size. It forks a child, sets up a socketpair (or a pty, if the `JOB_PTY` flag is set) connecting the child's stdout back to the server, `execs` the command in the child, and in the parent returns a `struct job` whose output end is registered with libevent. From then on the job is *asynchronous*: the child runs concurrently, its output arrives as libevent read events that fire the update callback, and its death arrives as a `SIGCHLD` that the server's child-reaping (Chapter 3's `server_child_signal`) routes to the job's complete callback. The server never waits on the child; it goes back to its event loop and services everything else.

The flags encode the variations the callers need. `JOB_NOWAIT` runs a fire-and-forget job whose output is discarded (a `run-shell -b` background command). `JOB_KEEPPWRITE` keeps the write end open so the job can be *fed* input, not just read from — the mechanism `pipe-pane` needs. `JOB_PTY` gives the child a pseudo-terminal instead of a pipe, for commands that behave differently when they think they are on a terminal. `job_free` tears a job down, and two functions matter to the rest of the server: `job_kill_all`, which the server calls on exit to terminate every outstanding job (Chapter 3), and `job_check_died`/the still-running check, which the server-loop exit decision consults so the server does not quit out from under a job that is still working (Chapter 3's exit conjunction). The jobs subsystem is thus wired into the server's lifecycle at both ends: started asynchronously, and accounted for when deciding whether the server is idle.

run-shell and if-shell: commands that wait

The two user-facing commands built directly on jobs are `run-shell` and `if-shell`, and they show how a *command* can be asynchronous without stalling the command queue (Chapter 8). `run-shell` runs a shell command and, unless backgrounded, waits for it — capturing its output and displaying it, or feeding it to a `-t` pane. `if-shell` runs a command purely for its *exit status* and then branches: on success it runs one command list, on failure another. Both are the reason the command queue models *waiting* (Chapter 8): when `if-shell`'s `exec` runs, it launches a job and then *suspends its queue item* rather than returning a result, and the item stays suspended — the server loop continuing to run everything else — until the job's complete callback fires with the exit status, at which point the item resumes and queues the chosen branch.

This is the property that makes tmux scriptable without freezing. A `.tmux.conf` that does `if-shell "test -n \"$SSH_CONNECTION\" " "source ~/.tmux-remote.conf"` runs a real subprocess during config load,

and a status line that shells out must not hang the server while it does. Because the job is asynchronous and the queue item merely suspends, a slow `if-shell` — a command that takes a second to decide — leaves every other client responsive; the server is not blocked, the item is just parked. `ztermux` ports the suspend/resume continuation exactly, because “does a slow `if-shell` freeze my other panes?” (it must not) is behavior, and getting it wrong would be a regression no format diff would catch but every user would feel.

Format jobs: `#(command)` in a status line

The format language's `#(command)` operator (Chapters 9 and 16) runs a shell command and interpolates its output into a format string — the classic status-line `#(uptime)` or `#(whoami)`. Naively that would fork a process every time the status line redraws, several times a second, which would be absurd; so `format.rs` keeps a *jobs cache* — an RB tree of format jobs keyed by the command string — so that `#(uptime)` runs *once*, its output is cached and interpolated on every redraw, and the job is only *re-run* on an interval (tied to `status-interval`). The first expansion of a `#(...)` starts the job and yields its last-known output (empty the first time); the job's update callback stores the fresh output; the next expansion picks it up. The stale-job expiry that the hourly tidy timer performs (Chapter 3, Chapter 23) is the reaping of format jobs whose commands have not been referenced recently.

This caching is also where the parity discipline meets the jobs subsystem, through a deliberate suppression. The parity suite (Chapter 11) cannot run `#(date)` and compare bytes — the output is non-deterministic — so format expansion carries a `FORMAT_EXPAND_NOJOBS` flag (Chapter 16) that tells the engine *not* to spawn jobs, leaving `#(...)` inert, and the suite runs with it set. The flag is not a parity hack; it is `tmux`'s own mechanism for the contexts where spawning a job would be unsafe or meaningless, ported along with everything else. So the same machinery that makes a status line's `#(...)` efficient in normal use makes it deterministic under the parity harness, which is the sort of dual-purpose fidelity the port aims for: the feature and its testability come from the same ported code.

`pipe-pane`: tee'ing a pane's output to a process

The fourth job-backed feature is `pipe-pane`, and it is the one that runs a job *fed* by a pane rather than read from. `pipe-pane "cat >> ~/pane.log"` opens a job whose stdin is connected to a *copy* of the target pane's output byte stream: every byte the pane's program writes is both processed normally (parsed into the grid, Chapter 6) and tee'd into the job's input, so the log file accumulates exactly what scrolled past. This is the `JOB_KEEPPWRITE` case from the first section — the write end is kept open so the pane can keep feeding the command — and it is wired at the pane's output path (`window-pane_start_output` / the pane's pipe fd) so the tee happens as output is read, not as a separate pass. Running `pipe-pane` again with no command toggles it off, closing the job.

`pipe-pane` completes the picture of what the jobs subsystem is *for*: it is the multiplexer's whole relationship with helper processes, in every direction. `run-shell` reads a job's output; `if-shell` reads a job's exit status; a format job caches a job's output for a status line; `pipe-pane` feeds a job a pane's output. Four different couplings, one `job_run`, all asynchronous, all reaped through the same `SIGCHLD` path, all accounted for in the same exit decision — and all ported from `job.c` with their callbacks and flags intact, so that a config or a script that leans on any of them behaves the way it does under `tmux`. The next chapter covers two more pieces of shared server state that these features touch — the paste buffers a copy fills and `run-shell` can read, and the environment a job inherits.

Chapter 28 — Paste Buffers and the Environment

Two small server-global stores round out the faithful core, and both are the kind of shared single-copy state Chapter 3 argued belongs in the server. The *paste buffers* are tmux's internal clipboard — the stack of text that copy mode fills and `paste-buffer` empties. The *environment* is the set of variables a session carries and hands to the programs it spawns. Neither is glamorous; both are ported carefully, because both have exact behaviors — buffer auto-naming and trimming, the `update-environment` merge — that scripts and configs depend on.

The paste buffer store: two orderings and automatic names

The paste buffers live in `src/paste.rs` (a port of `paste.c`), and the defining detail is that they are held in *two* red-black trees at once: one keyed by name (`paste_by_name`) and one keyed by an insertion order counter (`paste_by_time`), so a buffer can be found by its name *or* walked newest-first. Each `paste_buffer` carries its data and size, its name, an order number, and an `automatic` flag. That flag is the crux of the naming behavior: a buffer created *automatically* — by a copy-mode copy, which does not name its result — is given an auto-generated name (`buffer0000`, `buffer0001`, ...) from a counter, while a buffer created *explicitly* through `set-buffer -b <name>` carries the name the user gave it. The two kinds coexist in the same store, distinguished by the flag.

The automatic buffers are bounded, and the bound is a ported policy. The `buffer-limit` option (default 50) caps how many automatic buffers the store keeps; when a new automatic buffer pushes the count over the limit, `paste_add` frees the *oldest* automatic buffer (found through the by-time tree), so the clipboard is a bounded stack of recent copies rather than an unbounded accumulation. Explicit named buffers are *not* subject to the limit — you asked for them by name, so they persist until you delete them. This distinction (automatic buffers roll off, named buffers stay) is exactly tmux's, and it is the sort of thing a user relies on without articulating: your last fifty mouse-copies are there to paste, but the buffer you deliberately named for a script does not silently vanish. The operators around it — `paste_get_top` (the most recent), `paste_get_name` (by name), `paste_set`, `paste_replace`, `paste_rename`, `paste_walk` (iterate for `list-buffers`) — are the ported API the buffer commands sit on.

The buffer commands and a fixed parity gap

The user-facing buffer commands are a family, and they all route through the `paste.rs` store: `set-buffer` (create or replace a buffer from a string), `load-buffer/save-buffer` (from and to a file), `show-buffer` (print the top buffer), `paste-buffer` (insert a buffer's contents into a pane, optionally with `-p` bracketed-paste and `-r` no-newline-translation), `delete-buffer`, and `list-buffers` (walk them with a `-F` format). `choose-buffer` (Chapter 26's `window_buffer` mode) is the interactive picker over the same store. Copy mode's copy commands (Chapter 18) create buffers through `paste_add`, which is the loop that connects “select and yank in copy mode” to “paste on the command line.”

This store is also where one of the parity-suite gaps (Chapter 11, Appendix D) lived and was fixed, which makes it a concrete example of the method. Creating a *named* buffer with `set-buffer -b <name>` reported

“empty buffer name” in an early state of the port — the named-buffer path was not correctly wired — where tmux accepts it and stores the buffer under that name. The suite’s expansion caught it as a deterministic, pinned failure; the fix wired the `-b <name>` path through `paste_set/paste_rename` to match the C, and the case turned green and now guards the corner. It is the same shape as every other gap the expansion surfaced: a bounded, mechanical divergence from `paste.c`, caught by a byte comparison, fixed against the source of truth, and kept as a regression test forever.

The environment: per-session variables and `update-environment`

The other store is the environment, in `src/envIRON.rs` (a port of `envIRON.c`), and it exists because a session must carry a set of variables to hand to the programs it spawns — and *which* variables, and from *when*, is a genuinely subtle question a multiplexer has to answer well. The environment is a red-black tree of `envIRON_entry` (a name, a value, and flags), and there is a *global* environment (built from the server process’s own environment at startup, via `envIRON_create` and `envIRON_copy`) plus a per-session environment layered on top. The operators are the expected set: `envIRON_set/envIRON_put` to add, `envIRON_unset` to remove, `envIRON_get` to read, and `envIRON_update` to merge one environment into another. When a pane is spawned (Chapter 19), the effective environment is assembled from these layers and applied to the child with `envIRON_push` just before `exec`, so the program starts with exactly the variables tmux intends.

The behavior that makes this subsystem worth a section is `update-environment`, and it solves a real problem detach-survival creates. A long-lived session outlives the client that started it, but some environment variables — `DISPLAY`, `SSH_AUTH_SOCK`, `SSH_CONNECTION`, `SSH_AGENT_PID` — are tied to the *client*’s connection, not the session, and go stale the moment you detach and re-attach from a different terminal or a new SSH connection. The `update-environment` option is a *list of variable names* that tmux, on each attach or client switch, copies from the *attaching client*’s environment into the session’s environment (through `envIRON_update`), so the session picks up the new client’s `DISPLAY` and SSH agent rather than clinging to the ones from whenever it was first created. This is why you can detach an X or SSH session, re-attach from a fresh connection, open a new pane, and have `ssh` find the *current* agent and a GUI program find the *current* display — the session’s environment was refreshed from the client that just attached. `envIRON.rs` ports the option, the merge, and the timing exactly.

Why these small stores are worth porting exactly

It would be easy to treat the buffers and the environment as trivia — a clipboard and some variables — and reimplement them “reasonably.” The reason ztmux does not is that both have precise, depended-on behaviors hiding in their simplicity. The paste store’s automatic-versus-named distinction and its `buffer-limit` roll-off govern whether a script’s named buffer survives and whether your recent copies are where you expect; get the trimming policy or the auto-naming scheme wrong and a paste-buffer-driven workflow breaks in ways that are maddening to diagnose. The environment’s `update-environment` timing governs whether a re-attached session can reach the X server and the SSH agent; get the merge or its trigger wrong and re-attaching over SSH silently loses your agent forwarding. These are exactly the “reasonable but not identical” divergences (Chapter 29’s argument) that a byte-for-byte port avoids and a from-spec rewrite invites.

Both stores also close loops the rest of the book opened. The paste buffers are the sink of copy mode (Chapter 18) and the source of `paste-buffer`, the thing a mouse drag (Chapter 26) ultimately fills, and a store `run-shell` (Chapter 27) can read; the environment is what a job (Chapter 27) inherits and what a spawned pane (Chapter 19) starts with, assembled from the global and session layers and refreshed by the client that attaches. They are small, but they are *connected* — shared server-owned state that several subsystems read and write — which is precisely why they live in the server (Chapter 3) and precisely why porting them faithfully matters: a divergence here would surface not in one feature but in every feature that touches them. With the buffers and the environment ported, the account of the faithful core is complete down to its smallest

shared stores, and the remaining chapter steps back from mechanism to answer the sharp questions the whole approach invites.

Chapter 29 – Design Decisions and Objections

A port this opinionated invites questions, and the honest thing is to answer the sharp ones directly rather than let them sit as unstated doubts. This chapter takes the objections a skeptical reader would raise — about the `unsafe`, the FFI, the vendored trees, the byte-for-byte bar — and answers each from the project's actual reasoning. None of the answers is “trust us”; each is a consequence of the method the earlier chapters laid out.

“Why is it full of `unsafe`? Isn't the point to be safe Rust?”

The point is to be safe Rust *eventually*, and the `unsafe` is the honest cost of getting there faithfully (Chapter 14). `ztmux` reproduces `tmux`'s manual-memory model — raw-pointer intrusive trees, `#[repr(C)]` structs, `static mut` globals, `libc/libevent` FFI — precisely because that is what lets the behavior be checked against the C line for line. A grid cell that is a `#[repr(C)]` struct with `tmux`'s exact field layout can be reasoned about against the C directly; the same cell rewritten as an idiomatic Rust enum could not. The `unsafe` is a phase, not a destination: with parity green and pinned as a specification, the tree gets swept subsystem by subsystem toward safe Rust, each conversion guarded by the parity suite. Starting idiomatic would have thrown away the one thing that makes the port checkable. So the `unsafe` is not a failure of the safety goal — it is the disciplined route to it.

“Why link the real `libevent` through FFI instead of a Rust `async` runtime?”

Because a faithful port of an event-loop program must dispatch on the *same* event loop. The exact order in which `tmux` services fds, fires timers, and drains its command queue is observable behavior — the difference between a redraw landing this iteration or the next — and a reimplemented `async` runtime, however good, would make subtly different scheduling decisions. So `ztmux` links `libevent` (`event_core`, statically on macOS, dynamically elsewhere) and calls its C API directly, registering the same `struct event` values and the same `extern "C-unwind"` callbacks `tmux` registers. The FFI is inherently `unsafe`, but it will never go fully to zero, and that is fine: the right amount of FFI `unsafe` is a small, audited set of wrapper functions, not a reimplemented runtime that might diverge.

“Why vendor two whole source trees? That bloats the repo.”

Self-containment is worth the bytes. `vendor/tmux/` (the C source of truth) and `vendor/tmux-rs/` (the pristine seed) are both committed, SHA-pinned, plain copies so that a clone of `ztmux` carries everything needed to build the port, build the reference, run the parity suite, and audit every function against its origin — with no network access and no version ambiguity (Chapter 2). “Faithful to `tmux`” then means faithful to a *specific, committed* `tmux`, not to whatever `tmux` a contributor happens to have installed, which would produce false parity diffs from version drift. And keeping the seed pristine makes the port's own contribution a single `git`

`diff --no-index vendor/tmux-rs/src src` away — the divergence diff that makes inflation impossible to hide. The repo size buys auditability that cannot be had any cheaper.

“Byte-for-byte is fanatical. Isn’t ‘behaves reasonably’ enough?”

No, and the three headline bugs (Chapter 11) are the proof. The `39|40-versus-40|39` layout rounding, the `pane_current_command` reporting `zterm` on macOS, the `{1:}` crash — each produced output that looked perfectly *reasonable* in isolation. They were only visible as *divergences from tmux*, and only a byte-for-byte comparison against the reference found them. The whole value of a superset is muscle-memory compatibility: the configs, keybindings, and status-line format strings people have accumulated over years must keep working unchanged, and those depend on exact widths, exact format output, exact defaults. “Almost compatible” is the state in which a status line prints a subtly wrong width and a script silently breaks. Byte-for-byte is not fanaticism; it is the only bar that guarantees the compatibility the superset rests on.

“A green suite is easy to fake — why trust zterm’s 1107/1107?”

Because the number is *earned*, and the method is designed to make faking it obvious. The suite reached 122/122, and rather than bank a green bar that might only mean “the suite is too small to fail,” the project grew it to 1107 — deliberately into corners it suspected were weak — until it *did* fail, pinned each of the twenty failures to a specific gap (Appendix D), and then fixed every one against the C. A parity case is an asset: a byte-exact statement of tmux’s behavior with a minimal reproduction attached, and it is never deleted — the twenty that once failed now stand as regression guards. So 1107/1107 is a stronger claim than the original 122/122 was, not merely a bigger one — it is measured against a harder target that was deliberately made to fail first, and every corner it caught is named, reproducible, and now green.

“Why keep the C function names and structure? Rust has better idioms.”

It does, and they are the endpoint, not the starting point (Chapter 14). Keeping the C names is what makes the anti-drift gate, the C-signature citations, and a reviewer’s side-by-side diff all possible — the shared name is the join key between the Rust and the C. Rewrite `grid_create` as `Grid::new` and the citation has nothing to point at, the parity comparison can no longer localize a divergence to a function, and the port becomes “inspired by” tmux rather than a checkable transliteration of it. The idiomatic cleanup comes *after* a subsystem is verified, with the parity cases as the specification the rewrite must satisfy — so the auditable phase and the idiomatic phase are distinct in time. Having maximal C correspondence and maximal Rust idiom *simultaneously* is the mistake; having them in sequence is the plan.

Appendix A – The Command Reference

This appendix lists the commands ztmux ports, grouped by what they act on, with each command's canonical name, its alias in parentheses, and a one-line description of what it does. The list is drawn from `CMD_TABLE` in `src/cmd.rs` — an array of ninety command entries — and the per-command files `src/cmd_*.rs`, each a mirror of tmux's corresponding `cmd-*.c`. A command with no alias listed has none. Every entry here corresponds to a real tmux command; the point of the port is that these behave identically, so this doubles as a map of the tmux command surface that ztmux covers.

Session commands

Sessions are the top of the state tree — a collection of windows a client attaches to.

- **new-session (new)** — create a new session (and, on first use, autostart the server) and optionally attach to it.
- **attach-session (attach)** — attach the current client to an existing session.
- **detach-client (detach)** — detach one or more clients from their sessions, leaving the sessions running.
- **has-session (has)** — test whether a named session exists (exit status only).
- **kill-session** — destroy a session and everything in it.
- **rename-session (rename)** — change a session's name.
- **switch-client (switchc)** — point an attached client at a different session, window, or pane without detaching.
- **suspend-client (suspendc)** — stop the client process with `SIGTSTP`.
- **lock-session (locks)** — lock a session, running the lock command.
- **list-sessions (ls)** — list sessions, one per line, formatted by `-F`.
- **choose-client** — interactive chooser for attached clients.
- **choose-tree** — interactive session/window tree chooser.

Window commands

A window is a collection of panes tiled by a layout, with one current window per session.

- **new-window (neww)** — create a new window in the current session.
- **kill-window (killw)** — destroy a window and its panes.
- **rename-window (renamew)** — change a window's name (also affects automatic-rename).
- **select-window (selectw)** — make a window the current window.
- **next-window (next)** — switch to the next window by index.
- **previous-window (prev)** — switch to the previous window by index.
- **last-window (last)** — switch to the most recently current window.
- **move-window (movew)** — renumber a window (move it to another index/session).
- **link-window (linkw)** — link an existing window into another session.
- **unlink-window (unlinkw)** — remove a linked window from a session.
- **swap-window (swapw)** — exchange two windows' positions.
- **rotate-window (rotatew)** — rotate the panes of a window through their positions.
- **respawn-window (respawnw)** — restart the command in a window whose program exited.
- **list-windows (lsw)** — list a session's windows, formatted by `-F`.
- **find-window (findw)** — search window names/titles/content and select a match.

Pane commands

A pane is a single pseudo-terminal running a program, backed by a screen and grid.

- **split-window (splitw)** — split a pane into two, **-h** horizontally or **-v** vertically.
- **kill-pane (killp)** — destroy a pane.
- **select-pane (selectp)** — make a pane active, or move to the pane in a direction (**-L/-R/-U/-D**), or mark a pane (**-m**).
- **last-pane (lastp)** — switch to the most recently active pane.
- **resize-pane (resizp)** — grow or shrink a pane, or zoom it (**-Z**).
- **swap-pane (swapp)** — exchange two panes' positions in the layout.
- **break-pane (breakp)** — promote a pane into its own new window.
- **join-pane (joinp)** — pull a pane from another window into this one as a split.
- **move-pane (movep)** — the **join-pane** machinery targeted as a move.
- **respawn-pane (respawnp)** — restart the command in a pane whose program exited.
- **capture-pane (capturep)** — dump a pane's visible content or scrollback as text (optionally with escape sequences) to a buffer or stdout.
- **clear-history (clearhist)** — drop a pane's scrollback history.
- **pipe-pane (pipep)** — pipe a pane's output to a shell command in real time.
- **display-panes (displayp)** — overlay each pane with its index for keyboard selection.
- **list-panes (lisp)** — list a window's panes, formatted by **-F**.

Layout commands

- **select-layout (selectl)** — apply a preset layout or a saved layout string.
- **next-layout (nextl)** — cycle to the next preset layout.
- **previous-layout (prevl)** — cycle to the previous preset layout.

Buffer and copy-mode commands

Paste buffers are server-global clipboards; copy mode is the grid navigator of Chapter 18.

- **copy-mode** — enter copy mode in a pane.
- **clock-mode** — display the big clock in a pane.
- **list-buffers (lsb)** — list paste buffers.
- **set-buffer (setb)** — set the contents of a paste buffer (named with **-b**).
- **delete-buffer (deleteb)** — delete a paste buffer.
- **load-buffer (loadb)** — load a file into a paste buffer (over the socket, in the client's context — Chapter 17).
- **save-buffer (saveb)** — write a paste buffer to a file.
- **show-buffer (showb)** — print a paste buffer's contents.
- **paste-buffer (pasteb)** — paste a buffer's contents into a pane.
- **choose-buffer** — interactive paste-buffer chooser.

Key-binding commands

- **bind-key (bind)** — bind a key in a table to a command list.
- **unbind-key (unbind)** — remove a binding.
- **list-keys (lsk)** — print the key tables and their bindings.
- **send-keys (send)** — send keys (or literal input) to a pane as if typed.
- **send-prefix** — send the prefix key to a pane.
- **list-commands (lscm)** — print the command list with usage strings.

Option, hook, and environment commands

- **set-option (set)** — set a server/session option (**-g** global, **-s** server).
- **set-window-option (setw)** — set a window option.
- **show-options (show)** — print options and their values.
- **show-window-options (showw)** — print window options.
- **set-hook** — set a hook (a command list run on an event).
- **show-hooks** — print hooks.
- **set-environment (setenv)** — set a variable in the session environment.
- **show-environment (showenv)** — print the session environment.

Server, client, and display commands

- **start-server (start)** — start the server without attaching.
- **kill-server** — terminate the server and all sessions.
- **lock-server (lock)** — lock all clients.
- **server-access** — manage the access-control list for the server socket.
- **list-clients (lsc)** — list attached clients.
- **refresh-client (refresh)** — force a client redraw, resize, or control-mode update.
- **display-message (display)** — show a message (or, with **-p**, print an expanded format to stdout — the parity suite's workhorse).
- **show-messages (showmsgs)** — print the server's recent message log.
- **display-menu (menu)** — show an interactive menu overlay.
- **display-popup (popup)** — show a popup running a command in an overlay.

Control-flow and configuration commands

- **run-shell (run)** — run a shell command, optionally capturing its output.
- **if-shell (if)** — run a command list conditionally on a shell command's exit status.
- **confirm-before (confirm)** — prompt for confirmation before running a command.
- **command-prompt** — open the interactive command prompt in the status line.
- **wait-for (wait)** — block on, or signal, a named synchronization channel.
- **source-file (source)** — load and run another configuration file.

Mode and history commands

- **customize-mode** — enter the interactive options/keys customizer.
- **show-prompt-history (showphist)** — print the saved command-prompt history.
- **clear-prompt-history (clearphist)** — clear the saved command-prompt history.

Appendix B – The Format Variable Reference

The `#{...}` format language (Chapters 9 and 16) exposes nearly every field of the server's state as a variable. This appendix groups the variables by the object they describe. The dynamic per-object variables are resolved from live state at expansion time; the callback variables live in `FORMAT_TABLE` in `src/format.rs`. This is not the entire set tmux defines, but it is the substantial majority the port carries, and it is the surface the parity `.fmt` cases probe. Flag variables (`*_flag`, and booleans like `pane_active`) expand to `1` or `0` (or empty), which is exactly why the `#{?...}` conditional tests them.

Client variables

Describe an attached client — a terminal lending itself to the server.

- `client_name`, `client_tty` — the client's name and controlling tty path.
- `client_pid`, `client_uid`, `client_user` — the client process's pid, uid, and user name.
- `client_width`, `client_height` — the client terminal's size in cells.
- `client_cell_width`, `client_cell_height` — the pixel size of a cell, when known.
- `client_termname`, `client_terminfo`, `client_termfeatures` — the `TERM` value, type, and negotiated feature set.
- `client_session`, `client_last_session` — the current and previous session names.
- `client_key_table` — the client's current key table (e.g. `prefix` while pending).
- `client_prefix` — flag: is the prefix key currently pending?
- `client_readonly`, `client_control_mode` — flags: read-only client, control mode.
- `client_activity`, `client_created` — timestamps.
- `client_flags`, `client_written`, `client_discarded` — flag string and I/O byte counters.

Session variables

Describe a session — the top of the state tree.

- `session_name`, `session_id` — name and numeric id.
- `session_windows` — number of windows in the session.
- `session_attached`, `session_many_attached`, `session_attached_list` — how many clients are attached, whether more than one, and their list.
- `session_created`, `session_activity`, `session_last_attached` — timestamps.
- `session_path` — the session's working directory.
- `session_marked` — flag: does this session contain the marked pane?
- `session_stack` — the window stack (most-recent order).
- `session_grouped`, `session_group`, `session_group_size`, `session_group_list`, `session_group_attached`, `session_group_attached_list`, `session_group_many_attached` — session-group membership and sizes.
- `session_alerts` — pending alerts on the session's windows.
- `session_format` — flag used internally when formatting a session line.

Window variables

Describe a window within a session.

- **window_name**, **window_index**, **window_id** — name, index, and unique id.
- **window_width**, **window_height** — the window's size in cells.
- **window_panes** — number of panes in the window.
- **window_active** — flag: is this the session's current window?
- **window_layout**, **window_visible_layout**, **window_raw_flags** — the layout string, the visible-portion layout, and the raw flag characters.
- **window_flags** — the summary flag string (***** current, **-** last, **Z** zoomed, **!** bell, and so on).
- **window_activity**, **window_activity_flag**, **window_bell_flag**, **window_silence_flag** — activity/bell/silence state and timestamps.
- **window_zoomed_flag** — flag: is a pane zoomed?
- **window_last_flag**, **window_marked_flag**, **window_linked** — last-window, contains- marked-pane, and linked-into-multiple-sessions flags.
- **window_start_flag**, **window_end_flag**, **window_bigger** — used when the window is larger than the client and scrolled.
- **window_offset_x**, **window_offset_y** — scroll offset when bigger than the client.
- **window_cell_width**, **window_cell_height** — pixel cell size.
- **window_stack_index** — position in the session's most-recent-use stack.
- **window_active_clients**, **window_active_clients_list**, **window_active_sessions**, **window_active_sessions_list**, **window_linked_sessions**, **window_linked_sessions_list** — which clients/sessions have this window active or linked.
- **active_window_index**, **last_window_index** — the session's current and last window indices.

Pane variables

Describe a pane — a single `pty` running a program.

- **pane_id**, **pane_index** — unique id (**%N**) and index within the window.
- **pane_width**, **pane_height** — size in cells.
- **pane_left**, **pane_right**, **pane_top**, **pane_bottom** — the pane's edges in window coordinates.
- **pane_at_left**, **pane_at_right**, **pane_at_top**, **pane_at_bottom** — flags: is the pane against each edge? (**pane_at_top/pane_at_bottom** are two of the current parity gaps — see Appendix D.)
- **pane_active**, **pane_last**, **pane_marked**, **pane_marked_set** — active, last, marked, and any-marked-set flags.
- **pane_current_command**, **pane_current_path** — the foreground command and its cwd.
- **pane_start_command**, **pane_start_path** — the command and path the pane was spawned with.
- **pane_pid**, **pane_tty** — the child process pid and the pane's `pty` path.
- **pane_title** — the pane title (set by OSC 0/2).
- **pane_dead**, **pane_dead_status**, **pane_dead_signal**, **pane_dead_time** — for a pane whose program exited (with **remain-on-exit**), the death flag, exit code, signal, and time.
- **pane_in_mode**, **pane_mode** — whether the pane is in a mode (copy/clock) and which.
- **pane_synchronized**, **pane_input_off**, **pane_pipe** — synchronize-panes, input-disabled, and pipe-pane states.
- **pane_fg**, **pane_bg** — the pane's default foreground/background colors.
- **pane_tabs** — the pane's tab-stop columns.
- **pane_search_string**, **pane_unseen_changes** — copy-mode search string and the changed-since-last-visible flag.
- **cursor_x**, **cursor_y**, **cursor_character**, **cursor_flag** — the pane cursor's position, the character under it, and visibility.
- **scroll_region_upper**, **scroll_region_lower** — the pane's scroll region rows.
- **alternate_on**, **alternate_saved_x**, **alternate_saved_y** — alternate-screen state.

- **history_size**, **history_limit**, **history_bytes**, **history_all_bytes** — scrollbar line count, limit, and byte sizes.
- **wrap_flag**, **insert_flag**, **origin_flag**, **keypad_flag**, **keypad_cursor_flag** — the pane's terminal mode flags.
- **pane_format**, **pane_key_mode**, **pane_path** — internal/format helpers.

Mouse variables

Set when a format is expanded in the context of a mouse event.

- **mouse_x**, **mouse_y** — cell coordinates of the event.
- **mouse_button_flag**, **mouse_standard_flag**, **mouse_sgr_flag**, **mouse_all_flag**, **mouse_any_flag** — which mouse-reporting mode is active.
- **mouse_pane**, **mouse_line**, **mouse_word**, **mouse_hyperlink** — the pane, line, word, and hyperlink under the pointer.
- **mouse_status_line**, **mouse_status_range** — status-line row and style range hit.

Server and host variables

Global to the server or the host, and largely *excluded* from parity cases because they are non-deterministic (Chapter 11).

- **host**, **host_short** — full and short hostname.
- **pid** — the server pid.
- **version** — the tmux/ztmux version string (deliberately the port's own, hence excluded from parity).
- **socket_path** — the server socket path (the **ztmux-<uid>** one).
- **uid**, **user** — the server's uid and user name.
- **start_time** — when the server started.
- **server_sessions** — total session count across the server.
- **next_session_id** — the id the next session will get.
- **config_files** — the configuration files that were loaded.

Appendix C – The Option Reference

Options are the typed, scoped configuration values of Chapter 9, defined in `src/options_table.rs` (123 entries) and read through `src/options.rs`. This appendix groups them by their primary scope and notes types and notable defaults where they are load-bearing. Scope determines which tree an option lives in and how it inherits: **server** options are global to the server; **session** options apply per session (and fall back to the global session tree); **window** options apply per window; **pane** options apply per pane and inherit from the window. Remember the caveat from Chapter 15: every option here is *accepted and stored* so a real `.tmux.conf` parses, but not every one is yet wired to behavior — the configuration surface is complete before the behavior behind all of it is.

Server options

Global settings that govern the server process and its interaction with terminals.

- **buffer-limit** (number) — maximum number of paste buffers kept.
- **command-alias** (array) — user-defined command-name aliases.
- **default-terminal** (string) — the `TERM` the server advertises to panes (commonly `screen-256color` or `tmux-256color`).
- **escape-time** (number, ms) — how long to wait after an `Escape` to distinguish it from a function-key sequence.
- **exit-empty** (flag) — exit the server when no sessions remain (a `server_loop` exit conjunct, Chapter 3).
- **exit-unattached** (flag) — exit when no client is attached.
- **extended-keys**, **extended-keys-format** (choice/flag) — the CSI-u / modified-key reporting mode.
- **focus-events** (flag) — pass focus-in/out reporting to panes.
- **history-file** (string) — file for command-prompt history persistence.
- **message-limit** (number) — how many server messages `show-messages` keeps.
- **prompt-history-limit** (number) — command-prompt history depth.
- **set-clipboard** (choice) — whether/how to set the system clipboard via OSC 52.
- **terminal-features**, **terminal-overrides** (array) — the terminfo correction machinery (Chapter 10).
- **user-keys** (array) — user-defined key sequences.

Session options

Per-session behavior, the largest group, covering the status line, keys, and policy.

- **base-index** (number) — the first window index (default 0; many users set 1).
- **default-command**, **default-shell** (string) — the program a new pane runs.
- **default-size** (string) — the size for a detached/unattached session (`80x24`).
- **destroy-unattached** (flag) — destroy a session when its last client detaches.
- **detach-on-destroy** (choice) — what a client does when its session is destroyed.
- **display-time** (number, ms) — how long `display-message` stays up.
- **display-panes-time**, **display-panes-colour**, **display-panes-active-colour** — the `display-panes` overlay.
- **history-limit** (number, default 2000) — scrollbar lines per new pane (Chapter 5).
- **key-table** (string) — the default key table (normally `root`).
- **lock-after-time** (number), **lock-command** (string) — idle auto-lock.

- **mouse** (flag) — enable mouse support.
- **prefix** (key, default **C-b**), **prefix-timeout** (number) — the prefix key and its timeout.
- **renumber-windows** (flag) — renumber windows when one closes.
- **repeat-time** (number, ms) — the window during which a **-r** binding repeats.
- **set-titles**, **set-titles-string** — set the outer terminal's title.
- **status** (choice: off/on/2..5) — status line on and how many rows.
- **status-bg**, **status-fg**, **status-style** — status colors/style.
- **status-format** (array) — the raw per-row status format.
- **status-interval** (number) — seconds between status refreshes.
- **status-justify** (choice), **status-position** (choice: top/bottom) — layout.
- **status-keys** (choice: emacs/vi) — editing keys in the status prompt.
- **status-left**, **status-left-length**, **status-left-style** — the left section.
- **status-right**, **status-right-length**, **status-right-style** — the right section.
- **message-style**, **message-command-style**, **message-line** — the message bar.
- **update-environment** (array) — variables copied from the client on attach (Chapter 19).
- **visual-activity**, **visual-bell**, **visual-silence** — alert display modes.
- **activity-action**, **bell-action**, **silence-action** — alert policies.
- **word-separators** (string) — copy-mode word boundaries (Chapter 18).
- **buffer/editor/set-clipboard-adjacent** helpers — **copy-command**, **editor**.

Window options

Per-window appearance and behavior.

- **aggressive-resize** (flag) — resize a window to the smallest client actively viewing it rather than the session size.
- **automatic-rename**, **automatic-rename-format** — auto-title a window from its active pane's command.
- **clock-mode-colour**, **clock-mode-style** — the **clock-mode** display.
- **main-pane-width**, **main-pane-height** — the main pane size for the main-horizontal/ main-vertical layouts (**main-pane-width** under main-vertical is a current parity gap — Appendix D).
- **other-pane-height**, **other-pane-width** — the non-main panes' size.
- **mode-keys** (choice: emacs/vi) — copy-mode key table selection.
- **mode-style** — the highlight style for selections and menus.
- **monitor-activity**, **monitor-bell**, **monitor-silence** — per-window alert monitoring.
- **pane-base-index** (number) — the first pane index.
- **pane-border-status** (choice), **pane-border-format**, **pane-border-lines**, **pane-border-indicators**, **pane-border-style**, **pane-active-border-style** — pane border drawing (Chapter 20).
- **window-size** (choice: largest/smallest/latest/manual) — how a session sizes to differing clients (Chapter 4).
- **window-status-format**, **window-status-current-format**, **window-status-separator** — the middle of the status line.
- **window-status-style**, **window-status-current-style**, **window-status-last-style**, **window-status-activity-style**, **window-status-bell-style** — window-status colors.
- **window-style**, **window-active-style** — default cell style for inactive/active panes.
- **wrap-search** (flag) — wrap copy-mode search at the ends.

Window-and-pane options

Options that live in a combined scope, settable per window or per pane.

- **allow-passthrough** (flag) — permit the DCS passthrough escape.
- **allow-rename** (flag) — permit programs to rename the window via the rename string (Chapter 6).
- **allow-set-title** (flag) — permit OSC 0/2 title changes.

- **alternate-screen** (flag) — honor the alternate-screen switch (Chapter 5).
- **cursor-colour**, **cursor-style** — the pane cursor appearance.
- **remain-on-exit**, **remain-on-exit-format** — keep a dead pane in place (Chapter 19).
- **scroll-on-clear** (flag) — push cleared lines into scrollbar.
- **synchronize-panes** (flag) — send input to all panes of a window at once.
- **fill-character** (string) — the character used to fill unused pane area.
- **pane-colours** (array) — the pane's palette overrides.

How scope resolution works

When you type a bare `set-option foo bar`, ztmux must decide *which* scope `foo` belongs to, and it does so with `options_scope_from_name` (ported with its C citation `vendor/tmux/options.c:863`): the option's name determines its natural scope from the table, and the command flags (`-s` server, `-g` global session, `-w` window, `-p` pane) constrain it further. This is why `set -g status on` sets the global session option, `setw -g window-status-format ...` sets the global window option, and `set -p allow-rename on` sets it on the current pane. Getting this resolution byte-identical to tmux matters because a config that sets an option in the wrong scope silently does nothing, and tmux users have years of precise expectations about which `set` lands where.

Appendix D – The Parity Methodology and the Current Gap List

Chapter 11 argued that the parity suite is the operational definition of correct. This appendix is the reference companion: how to run the suite, how a case is structured, the determinism rules a case must obey, and — the record that the expansion caught twenty gaps which the port then closed — the now-green cases that each still guard a specific corner.

Running the suite

The runner is `parity/run_parity.sh`. It builds the vendored tmux reference (`vendor/tmux/tmux`, the `next-3.7` line) and a release ztmux if they are missing, then runs every case under both and compares:

```
# per-case OK/FAIL plus totals
bash parity/run_parity.sh
# totals only, for CI
bash parity/run_parity.sh --summary
# emit a machine-readable summary
bash parity/run_parity.sh --json parity/parity_summary.json
# test a debug build instead of release
ZTMUX=target/debug/ztmux bash parity/run_parity.sh
```

Per-case failure detail — both outputs plus a unified diff — lands in `parity/parity_failures.log` (gitignored, truncated each run), so a **FAIL** is always accompanied by the exact bytes that differed. The reference can be overridden with `TMUX_REF=/path/to/tmux`, but the default — and the only reference the numbers are reported against — is the vendored tmux, because version drift against a system tmux produces false diffs (Chapter 11).

Case anatomy

Cases live in `parity/cases/` in two flavors:

- `*.fmt` — a single tmux FORMAT string. The runner expands it with `display-message -p` against a fresh detached session and diffs the two binaries' output. Example: `010_arith_add.fmt` contains `{e|+|:2,3}` and both must print 5.
- `*.sh` — a shell scenario for multi-command behavior. `$TM` is exported as the binary bound to a private socket; the script runs `$TM <cmd>` lines and prints deterministic output. Example: `100_list_windows_after_neww.sh` runs `$TM new-window` then `$TM list-windows -F '#{window_index}'`.

For each case the runner starts an isolated server per binary (`-L <uniq>`, `-f /dev/null`, fixed 80×24 geometry), runs the case under a `timeout`, captures combined stdout and stderr, kills the server, and compares with a raw byte comparison. One differing byte fails the case.

Determinism rules

Byte comparison only works on deterministic output, so cases must avoid host/time/version/ pid/random state. Forbidden inputs include `{host}`, `{host_short}`, `{version}`, `{pid}`, `{client_pid}`, wall-clock times, and socket paths — all of which differ between the reference and the port (the port's version and socket prefix are deliberately its own) or between runs. The runner pins geometry (80×24), locale (`LC_ALL=C`), and options (`-f /dev/null`) so widths, collation, and defaults are stable. The discipline is to exclude non-deterministic *inputs* rather than normalize non-deterministic *output*, which keeps the comparison exact.

The gaps the expansion caught — and how each was closed (now 1107/1107)

The suite reached 122/122, was expanded to 1107, and the expansion surfaced twenty deterministic failures — each pinned to a real, bounded port gap. Every one has since been fixed against the C, and the suite now reports **1107 of 1107 passing**. The cases stay in the suite as regression guards, per policy. Grouped by the gap the expansion exposed, with the fix that closed it:

- `{!:...}` **logical-not operator** — cases 548–550. tmux prints 1 for the negation of a false value; ztmux printed empty because the `!` operator was not wired into the format modifier parse. *Fixed*: the `!` arm was added to the modifier `match` in `src/ported/format.rs`.
- `{s/.../.../}` **regex backreferences** — cases 566–568, 572. `\1/\2` in a substitution replacement were not substituted; on `abcd` a pair-swap that should yield `badc` emitted the literal digit next to the capture (`b2a1d2c1`). *Fixed*: backreference expansion was implemented in the replacement builder (`src/ported/regsub.rs` / the `s` modifier).
- `{p-N:...}` **left padding** — cases 589, 592. The left-pad form padded to `N` *plus* the content length instead of a total field width of `N`. *Fixed*: the pad is now computed as a total-width target, matching tmux's field-width semantics.
- `{pane_at_top}` / `{pane_at_bottom}` — cases 644–645. Formatted as Rust `true/false` (`format!("{}", flag)`) instead of the C's `%d` → `1/0`. *Fixed*: the flags now format as integers, as tmux does for the other `pane_at_*` flags.
- `{S:}` / `{W:}` / `{P:}` **loop variables** — cases 676–678. The capital-letter loop modifiers iterated but did not inject `loop_index` / `loop_last_flag` (real tmux format vars, `vendor/tmux/format.c:4776`), so an inner `{loop_index}` expanded empty. *Fixed*: the two variables are now injected per iteration, as the lowercase loop forms already did.
- **Named buffers** — cases 720–724. `set-buffer -b <name>` reported “empty buffer name”; named-buffer set/show/list/delete/rename were broken. *Fixed*: the `-b <name>` argument is now wired through the paste-buffer commands.
- `main-vertical` / `main-pane-width` — case 754. The main-vertical layout ignored `main-pane-width`; the main pane filled the width instead of being pinned to it. *Fixed*: the main-vertical builder now honors `main-pane-width`.

Each was a small, mechanical fix against the C, not an architectural hole — the profile a healthy expansion produces. The authoritative, current record is `parity/PARITY_ROADMAP.md` and the machine artifact is `parity/parity_summary.json` (passed 1107, failed 0); this appendix is a snapshot, and like every number in this book it is exactly as current as the code it was written against.

Why keep the cases forever

It bears repeating because it is counterintuitive: a parity case is an *asset* — even, or especially, after it goes green. It is a byte-exact statement of tmux's behavior with a minimal reproduction attached, and deleting it to shorten the suite would throw away the guard that keeps the corner from silently regressing. The green count means something only because the suite was *made to fail first*: the twenty cases the expansion broke on, now green, are the proof that 1107/1107 is a real number rather than the artifact of a suite too small to fail. A suite that never fails is a suite that has stopped measuring.

Appendix E – The Anti-Drift Gate and the Fake-Fn Categories

Chapter 12 described the anti-drift gate that keeps the port from quietly growing functions tmux does not have. This appendix is the reference detail: the gate’s mechanism, the exact categories the allowlist documents, and how the C-signature citations are generated and checked.

The gate mechanism

The gate is one test, `tests/ported_fn_names_match.rs`, run in CI like any other. It has two halves. `collect_free_fns` parses the Rust source character by character, tracking brace depth so it counts only depth-zero free functions (not methods inside `impl/trait` blocks, which correspond to C’s struct-of-function-pointers idiom), skipping braces inside comments, strings (including raw strings), and char literals, stripping `pub/unsafe/async/extern/const`, normalizing raw identifiers (`r#loop` → `loop`), and skipping `mod tests` wholesale. `c_fn_names` builds an index of every function *definition* in `vendor/tmux/*.c`, `vendor/tmux/compat/*.c`, and `cmd-parse.y` by scanning for line-initial identifiers followed by `(` with a brace nearby (a definition, not a `;-`terminated prototype), excluding C keywords, and asserting the index has more than 500 names as a sanity check that the vendored tmux is actually present.

The check: every collected Rust free-function name must be in the C index or in the allowlist. There is **no** trailing-underscore leniency — `foo_` is not auto-accepted as an implementation of C’s `foo`; the underscore convention is exactly how the port marks a split or a macro helper, and those must be listed rather than waved through. On failure the gate prints one greppable line per violation, `FAKE-FN <name> (<file>:<line>) – no tmux C counterpart`, and panics, so a developer sees precisely which functions are unaccounted for.

The allowlist categories

`tests/data/fake_fn_allowlist.txt` holds roughly three hundred accepted exceptions, and its header frames it as “an AUDIT TRAIL, not a free pass,” with the explicit goal to shrink it. The entries fall into documented categories, so a reader sees not just *that* a function is excepted but *why*:

1. **Macro-paired helpers.** C variadic functions (`format_add(fmt, ...)`) are ported as a Rust `macro_rules!` carrying the real C name plus a `_`-suffixed helper the macro forwards to (`format_add_`). Rust macros and functions occupy different namespaces, so the macro keeps the C name and the helper takes the underscore.
2. **C-function splits.** One C function ported as several typed Rust overloads — `xstrdup` as `xstrdup_`/`xstrdup__`/`xstrdup___` — to be merged back into one signature later. These are exactly the deviations the no-underscore-leniency rule is designed to surface.
3. **libc and libevent shims.** Wrappers around library functions with no `vendor/tmux` definition at all: `evtimer_*`, `evbuffer_*`, the `img` calls, `fnmatch`, `basename`, `dirname`. Real functions — just not tmux’s — so they cannot match the C index and must be listed.
4. **BSD intrusive-data-structure macros.** The red-black tree family (`rb_insert`, `rb_remove`, `rb_foreach`, `rb_min`, `rb_next`, ...), the tail-queue family (`tailq_foreach`, `tailq_insert_tail`, `tailq_remove`, ...), and the list family. These are C *macros* in tmux (no function definition to match)

ported as Rust functions because Rust has no equivalent of C's token-pasting intrusive-structure macros. They are a large share of the allowlist, and Chapters 3, 5, 7, and 9 all depend on them (`WINDOWS` is an `RB` tree, `CLIENTS` a `TAILQ`, key tables are `RB` trees).

5. **Genuine Rust glue.** The handful of truly Rust-only helpers — `cstr_to_str`, `assume_init_ref` — that exist because Rust needs them and C did not.

The allowlist is version-controlled, so its growth or shrinkage over time is itself a signal: it should trend toward zero as splits are merged, macros are replaced with safe Rust equivalents (Chapter 14), and shims are consolidated behind thin boundaries. Shrinking the allowlist and shrinking the `unsafe` surface are nearly the same project.

The C-signature citations

The name gate checks names; `scripts/annotate_c_links.py` checks *signatures*. For every Rust function whose name matches a tmux C function, it writes or refreshes the doc-comment citation this book quotes throughout — `/// C \vendor/tmux/c: " — including the C function's return type and full parameter list with names, so a reviewer can diff Rust parameters against C parameters and catch a renamed, reordered, dropped, or invented argument (a “fake param,” the argument-level analog of a fake fn). There are more than two thousand such citations.`

The script is careful about *which* C definition it cites when a name appears in several files: it prefers the C file whose stem matches the Rust file's stem (`grid.rs` → `grid.c`, `cmd_kill_pane.rs` → `cmd-kill-pane.c`, mapping underscores to hyphens); failing that, if the name is defined in exactly one C file it cites that; if it is ambiguous with no stem match, it *skips* rather than guess. It is idempotent — it replaces an existing citation in place rather than duplicating it — so re-running it after a vendored-tmux refresh updates every line number and signature to match the new C. A `--check` mode writes nothing and exits non-zero if any citation is missing or stale, so citation freshness can itself be a CI gate.

Together the name gate, the signature citations, and the allowlist form a structural-fidelity system that parallels the behavioral-fidelity system of the parity suite: names must correspond, interfaces must match, exceptions must be documented and bounded, and all of it is derived from the vendored C rather than a hand-maintained list, so none of it can drift out of sync with the source of truth.

Appendix F – The Source Tree, Module by Module

The chapters walked the subsystems in a reading order; this appendix is the map, grouping `src/` by role so you can find where a behavior lives. Every module named here is a port of the same-named tmux C file (with underscores where the C uses hyphens), carrying its C citations. The `cmd_*.rs` files are catalogued in Appendix A and omitted here except as a group.

Entry points and process model

- **main.rs** — the binary entry; installs the libc-shim global allocator and calls `tmux_main`.
- **tmux.rs** — argument handling, socket-label construction (`make_label`, the `ztmux-<uid>` path), and the client-vs-server decision.
- **proc.rs** — the `tmuxproc/tmuxpeer` process and peer abstraction over `msg`; the event dispatch (`proc_loop`) both server and client run on.
- **tmux_protocol_h.rs** — `PROTOCOL_VERSION` and the `MSG_*` message set (Chapter 17).
- **event.rs** — the libevent FFI surface used across the tree.

The server

- **server.rs** — the event loop (`server_loop`), the socket (`server_create_socket`), the accept path (`server_accept`), the global state trees, and the lifecycle (`server_start`, signal handling).
- **server_client.rs** — per-client servicing (`server_client_loop`), client creation, and the client-side of key and redraw handling.
- **server_fn.rs** — server helper functions (marked pane, message log, redraw requests).
- **server_acl.rs** — the socket access-control list (`server-access`).
- **client.rs** — the client half: connect, autostart, the identify handshake, the dispatch table (`client_dispatch_wait/_attached`), exit reasons.
- **control.rs**, **control_notify.rs** — control mode (`-CC`) and its notifications.

Grid, screen, and drawing

- **grid.rs** — the cell grid: creation, line growth, packed/extended cells, scrollbar (`grid_scroll_history`, `grid_collect_history`), reflow, string extraction.
- **grid_view.rs** — the coordinate-translated view used for writing into a region.
- **grid_reader.rs** — the copy-mode motion primitives (Chapter 18).
- **screen.rs** — the stateful view over a grid: cursor, scroll region, modes, tabs, selection, alternate screen, resize/reflow.
- **screen_write.rs** — the mutation API the input parser drives (`screen_write_*`), with the batched collection fast path.
- **screen_redraw.rs** — compositing panes and borders into a client (Chapter 20).
- **attributes.rs** — attribute-name ↔ bitset conversion (`attributes_tostring/_fromstring`).
- **colour.rs** — color pack/unpack across the 16/256/RGB tag scheme.

- **style.rs** — parsing and applying `#[...]` style strings.
- **format_draw.rs** — rendering an expanded status string with its style ranges into cells.

Input and terminal

- **input.rs** — the VT parser state machine (Chapter 6): states, transition tables, CSI/OSC/ESC/DCS dispatch, SGR.
- **input_keys.rs** — encoding tmux key codes into the byte sequences sent to panes.
- **tty.rs** — the output back end: differential redraw, flow control, cursor/attribute optimization, the output buffer.
- **tty_term.rs** — terminfo capability lookup and overrides.
- **tty_keys.rs** — decoding the outside terminal's keyboard bytes into key codes.
- **tty_features.rs** — the runtime feature registry (truecolor, synchronized output, ...).
- **tty_acs.rs** — the alternate-character-set (line-drawing) mapping.

Layout and windows

- **layout.rs** — the pane-tiling tree, splits, resize, navigation.
- **layout_set.rs** — the five preset layouts and their cycling.
- **layout_custom.rs** — the layout string: dump, parse, checksum.
- **window.rs** — the window object, panes, zoom (`window_zoom/window_unzoom`).
- **session.rs** — the session object and its window list/stack.
- **resize.rs** — whole-window and client resize orchestration.
- **spawn.rs** — creating windows/panes, the pty, and the fork/exec path (Chapter 19).
- **names.rs** — automatic window naming.

The command language

- **cmd.rs** — the `cmd_entry` model, `CMD_TABLE`, command lookup and aliases.
- **cmd_parse.lalrpop** / **cmd_parse.rs** — the grammar and the generated parser plus the hand-written lexer.
- **arguments.rs** — argument-spec interpretation (`args_has/_get/_count`).
- **cmd_find.rs** — target resolution (`session:window.pane, {marked}`, defaults).
- **cmd_queue.rs** — the command queue, per-item state, and asynchronous continuations.
- ****cmd_*.rs**** — one file per command, mirroring `cmd-*.c` (Appendix A).

Configuration, formats, keys

- **format.rs** — the `#{...}` engine (Chapters 9 and 16); the single largest module.
- **regsub.rs** — regex substitution behind the `s` modifier.
- **options.rs** / **options_table.rs** — the scoped, typed options system and its default table (Appendix C).
- **key_bindings.rs** — key tables and bindings (add/remove/lookup, reference counting).
- **key_string.rs** — parsing key names (`C-b, M-left`) into key codes.
- **keyc_mouse_key.rs** — mouse key-code mapping.
- **cfg.rs** — loading config files (`start_cfg, load_cfg`) and collecting causes.
- **environ.rs** — the session/global environment and `update-environment`.
- **notify.rs, alerts.rs** — hooks/notifications and the activity/bell/silence alerts.

Copy mode, modes, and buffers

- **window_copy.rs** — copy mode (Chapter 18): motions, selection, search, extraction.
- **window_clock.rs** — clock mode.
- **window_buffer.rs**, **window_client.rs**, **window_tree.rs**, **window_customize.rs** — the interactive choosers and customizer (**choose-***, **customize-mode**).
- **mode_tree.rs** — the shared tree/list UI the choosers are built on.
- **menu.rs**, **popup.rs** — **display-menu** and **display-popup** overlays.
- **paste.rs** — paste buffers.
- **status.rs** — the status line, messages, and the command prompt (Chapter 20).

Utilities and platform

- **utf8.rs**, **utf8_combined.rs** — UTF-8 handling and combining characters.
- **hyperlinks.rs** — OSC 8 hyperlink storage.
- **image.rs**, **image_sixel.rs** — sixel graphics (the one fully-implemented “extra”).
- **job.rs** — background jobs behind **#{shell}** formats and **run-shell**.
- **file.rs** — file I/O over the socket (Chapter 17’s read/write messages).
- **bitstr.rs** — the bit-string used for tab stops.
- **xmalloc.rs** — the checked-allocation wrappers (**xmalloc**, **xstrdup**, ...).
- **libc.rs**, **osdep.rs**, **compat/** — libc bindings and per-OS shims (the macOS **closefrom** fix of Chapter 19 lives in this territory).
- **log.rs** — logging, including the tty output log.
- **structured.rs** — structured-logging helpers.
- **utempter.rs** — utmp/wtmp login records where supported.
- **lib.rs** — the shared struct and constant definitions (**grid_cell**, **screen**, **layout_cell**, the flag sets) the rest of the tree is built on.

Appendix G — Glossary

A compact glossary of the terms this book uses in their tmux-specific senses. Where a term names a source construct, the module is noted.

- **Alternate screen** — a saved-and-restored secondary screen a full-screen program switches to so the shell's scrollbar is untouched when it exits (`screen.rs`, `screen_alterate_on/_off`).
- **Anti-drift gate** — the CI test that fails the build if a Rust free function has no tmux C counterpart and is not allowlisted (`tests/ported_fn_names_match_c.rs`, Appendix E).
- **Attach** — connecting a client to a session; involves the identify handshake and fd passing (Chapters 4, 17).
- **Cell** — one character position in the grid, with its character, attributes, and colors; stored *packed* or *extended* (`grid.rs`, Chapter 5).
- **Client** — the thin half of the binary that lends its terminal to the server and relays keys (`client.rs`, Chapter 4).
- **Command queue** — the ordered stream through which every state mutation passes (`cmd_queue.rs`, Chapter 8).
- **Copy mode** — a mode that turns a pane into a navigable, searchable, extractable view of its grid and scrollbar (`window_copy.rs`, Chapter 18).
- **Detach** — ending a client's terminal loan while the session keeps running (Chapter 4).
- **Extended cell** — a cell too rich for the packed form (wide/combining glyph, RGB color, non-default underscore color, hyperlink), stored via a `grid_extd_entry`.
- **Fake fn** — an invented Rust helper with no C counterpart that would inflate apparent completeness; caught by the anti-drift gate (Chapter 12).
- **Format** — a `#{...}` expression evaluated against server state; the configuration and introspection language (`format.rs`, Chapters 9, 16).
- **Grid** — the two-dimensional cell array plus scrollbar that backs a pane (`grid.rs`).
- **Grid reader** — the virtual cursor and its motions over stored grid lines (`grid_reader.rs`).
- **msg** — the OpenBSD message-passing library tmux uses to frame socket messages and pass file descriptors (Chapter 17).
- **Identify handshake** — the burst of `MSG_IDENTIFY_*` messages by which a client describes its terminal to the server on attach.
- **Layout cell** — a node in the pane-tiling tree: a left-right split, a top-bottom split, or a leaf window pane (`layout.rs`).
- **Layout string** — the checksummed serialization of a layout, printed by `{window_layout}` (`layout_custom.rs`).
- **Marked pane** — the server-global pane set by `select-pane -m`, used as an implicit source by `join-pane/swap-pane`.
- **Mode** — a display/key state pushed onto a pane (copy, clock, chooser) that takes over until popped (`mode_tree.rs`).
- **Option** — a typed, scoped configuration value (server/session/window/pane) (`options.rs`, Appendix C).
- **Packed cell** — the compact five-byte form for the common cell (ASCII, palette color, low attributes).
- **Pane** — a single pty running a program, backed by a screen and grid; the leaf of the state tree.
- **Parity suite** — the runner that diffs ztmux against the vendored tmux byte-for-byte; the operational definition of correct (`parity/`, Chapter 11, Appendix D).
- **Prefix key** — the key (default `C-b`) that switches the active key table to `prefix` so the next key is a tmux binding.
- **Reflow** — re-wrapping grid lines to a new width on resize (`grid_reflow_*`).

- **Screen** — the stateful view over a grid (cursor, scroll region, modes) (`screen.rs`).
- **Screen-write** — the mutation API (`screen_write_*`) the input parser drives; the mechanism half of the parser's policy/mechanism split.
- **Scrollbar** — the bounded history of past lines above the visible screen, trimmed by `grid_collect_history`.
- **Seed** — the pristine `vendor/tmux-rs/` port ztmux was forked from, kept untouched so the divergence diff stays meaningful (Chapter 2).
- **Server** — the long-lived daemon that owns all state and outlives every client (`server.rs`, Chapter 3).
- **Session** — a collection of windows a client attaches to; the top of the state tree.
- **Source of truth** — the vendored `vendor/tmux/` C, the reference every module is validated against.
- **Superset** — the goal of being a strict extension of tmux: everything tmux does, exactly, plus additive features (Chapter 13).
- **Target** — the `-t session:window.pane` (or `{marked}`, `-`, relative) argument resolved by `cmd_find.rs`.
- **terminfo** — the terminal-capability database the tty back end consults to emit the right escape sequences (`tty_term.rs`).
- **Window** — a collection of panes tiled by a layout, with one active pane; a member of a session.
- **Zoom** — temporarily making one pane fill the window without touching the layout tree (`window_zoom/window_unzoom`).
- **Activity / bell / silence** — the three background-window alerts, gated by `monitor-activity/monitor-bell/monitor-silence` and delivered per `*-action / visual-*` options (`alerts.rs`, Chapter 21).
- **Control mode** — a text, line-based protocol (`-CC`) by which a program drives the server and mirrors its state from `%`-prefixed notifications (`control.rs`, Chapter 17).
- **Differential redraw** — repainting only the cells that changed since the last frame, minimized further by `tty_cursor/tty_attributes` (Chapter 10).
- **Divergence diff** — `git diff --no-index vendor/tmux-rs/src src`, the port's original contribution against the pristine seed (Chapter 2).
- **Flow control** — throttling a pane whose output floods faster than the terminal can consume, via `TTY_BLOCK_START/TTY_BLOCK_INTERVAL` (Chapter 23).
- **Hook** — a command list bound to run when a named event fires; the reactive counterpart of a key binding (`notify.rs`, Chapter 21).
- **Key code** — the internal representation of a key plus its modifiers and flags, produced by `tty_keys.rs / key_string.rs`.
- **MSG *** — the `imsg` message types the client and server exchange over the socket (Chapter 17).
- **Notification** — the server's internal "something changed" broadcast, consumed by hooks and control-mode clients (`notify.rs`, Chapter 21).
- **Pending wrap** — the deferred-wrap state that keeps the cursor on the last column after a full-width write until the next character arrives (Chapter 6).
- **Semantic prompt** — OSC 133 markers a shell emits to delimit prompts and command output, recorded as grid line flags (Chapter 6).

Appendix H – Building ztmux and the Vended References

This appendix collects the build-side facts a reader who wants to compile, test, or audit the port needs: the dependencies, the feature flags, how the C libraries are linked, and how the two vended reference trees are kept and refreshed.

Dependencies

ztmux's `Cargo.toml` is deliberately lean, because a faithful port of a C program should not smuggle in a parallel stack of Rust libraries that change behavior. The runtime dependencies are:

- **libc** — the raw libc bindings the port calls directly (the custom allocator, the syscalls, the terminal ioctls).
- **bitflags** — the flag-set types (`grid_attr`, `format_modifiers`, and the rest) that mirror the C's `#define` flag constants.
- **lalrpop-util** — the runtime half of the LALRPOP parser generator (Chapter 8); the build-dependency **lalrpop** compiles `cmd_parse.lalrpop` to `cmd_parse.rs`.
- **memchr** — fast byte scanning used in the parser and format hot paths.
- **num_enum** — deriving the integer $\leftrightarrow$ enum conversions the ported tagged unions need.
- **paste** — token pasting for the macro-generated code.
- **terminfo-lean** — the one non-libc, non-parser dependency doing real domain work: it reads the terminfo database for the tty back end (Chapter 10).

That is the whole list. There is no async runtime (the port uses the real libevent through FFI), no Rust terminal-emulation crate (the port *is* the emulator), and no UI framework. The dependency list is short on purpose: every library that is not libc, the parser generator, or the terminfo reader is a potential source of behavioral divergence from tmux, and the port keeps that surface minimal.

Feature flags

The `[features]` block exists mostly, as the manifest itself notes, “to make matching upstream’s configuration options easier” — they let a `.tmux.conf` that references a build-time capability parse without error. The flags are `cgroups`, `hyperlinks`, `iutf8`, `ncurses`, `nokerninfo`, `sixel`, `sys_signame`, `systemd`, `utempter`, and `utf8proc`, with `default = ["sixel"]`. The important exception the book has stressed is **sixel**: it is not just an accepted flag, it is genuinely implemented (`src/image_sixel.rs`), and its presence changes an observable behavior — the Device Attributes reply advertises sixel (`\x1b[?1;2;4c` instead of `\x1b[?1;2c`, Chapter 6). The remaining flags are the honest “configuration surface exists, behavior is being wired” story of Chapter 15 in build form.

Linking the C libraries

`build.rs` handles linking libevent, and its policy is faithful to how the two platforms prefer to link: **static on macOS, dynamic elsewhere by default**, each overridable with the additive `static/dynamic` feature flags. On macOS the build locates the Homebrew libevent prefix (`brew_link_prefix("libevent")`) and adds

it to the link search path, then links `event_core` — statically (`cargo::rustc-link-lib=static=event_core`) unless the `dynamic` feature is set, dynamically otherwise. The reasoning is pragmatic: a statically linked libevent on macOS avoids the runtime-path headaches of a Homebrew dylib, while Linux systems generally have a system libevent worth linking dynamically. Either way the port uses the *real* libevent, because a faithful port of an event-loop program should dispatch on the same event loop, not a reimplemented one.

The two vendored references

Under `vendor/` sit two complete, plain-copied, SHA-pinned source trees, documented in `vendor/VENDOR.md`:

- **`vendor/tmux/`** — the upstream `tmux C source`, the `next-3.7` development line, and the *source of truth*. It is what every ported module is diffed against, what the parity runner builds and runs as its reference, and what the anti-drift gate indexes. Pinning to a specific upstream commit rather than “whatever `tmux` the system has” is essential because `tmux`’s behavior drifts between versions.
- **`vendor/tmux-rs/`** — Collin Richards’ `tmux-rs`, the Rust port `ztmux`’s `src/` was seeded from, kept pristine so the divergence diff (`git diff --no-index vendor/tmux-rs/src src`) always shows exactly what the port has changed since the fork.

Both carry their upstream ISC license and their pin metadata. The refresh procedure, per `VENDOR.md`, is mechanical: export a fresh `git archive` of the upstream with no `.git`, rsync it over the vendored path, and record the new `git rev-parse HEAD`. After a `vendor/tmux/` refresh, re-running `scripts/annotate_c_links.py` updates every one of the two-thousand-plus C-signature citations to the new line numbers and signatures, and the parity suite re-measures against the new reference — so a version bump is a controlled, re-verifiable operation rather than a leap of faith.

Appendix I – The Default Key Bindings

tmux ships a default `prefix` key table so that a fresh install is usable before any `.tmux.conf`, and ztmux builds the identical table at server start in `key_bindings_init` (`src/key_bindings.rs`), each binding added by `key_bindings_add` with the same key, note, and command list as tmux. This appendix lists the notable `prefix`-table bindings (the ones reached by pressing the prefix, default `C-b`, then the key). Each carries a `-N` note in the source, quoted here as its description. This is not the whole table, but it is the working set a user meets daily, and it is byte-for-byte tmux's.

Prefix table – windows and sessions

- **c** — create a new window (`new-window`).
- **&** — kill the current window, with confirmation (`confirm-before ... kill-window`).
- **,** — rename the current window (`command-prompt ... rename-window`).
- **.** — move the current window to a new index (`command-prompt ... move-window`).
- **n / p** — next / previous window (`next-window / previous-window`).
- **l** — last (previously current) window (`last-window`).
- **0-9** — select window by index (`select-window -t:=N`).
- **'** — prompt for a window index to select.
- **w** — choose a window from the tree chooser (`choose-tree`).
- **f** — find a window by name/content (`command-prompt ... find-window`).
- **\$** — rename the current session (`command-prompt ... rename-session`).
- **s** — choose a session from a list (`choose-tree -Zs`).
- **d** — detach the current client (`detach-client`).
- **D** — choose and detach a client (`choose-client -Z`).
- **(/)** — previous / next client (`switch-client -p / -n`).
- **L** — switch to the last client (`switch-client -l`).

Prefix table – panes

- **%** — split the window horizontally (`split-window -h`).
- **"** — split the window vertically (`split-window -v`).
- **o** — select the next pane (`select-pane -t:.`).
- **C-o** — rotate through the panes (`rotate-window`).
- **;** — move to the previously active pane (`last-pane`).
- **!** — break the pane into a new window (`break-pane`).
- **m / M** — toggle / clear the marked pane (`select-pane -m / -M`).
- **x** — kill the current pane, with confirmation.
- **z** — zoom / unzoom the current pane (`resize-pane -Z`).
- **q** — display pane numbers for selection (`display-panes`).
- **{ / }** — swap the pane with the previous / next (`swap-pane`).
- **Space** — cycle to the next preset layout (`next-layout`).
- **E** — spread panes out evenly (`select-layout -E`).
- **Arrow keys** — select the pane in a direction (`select-pane -L/-R/-U/-D`).
- **C-Arrow / M-Arrow** — resize the current pane (repeatable).

Prefix table — copy, buffers, and misc

- **[** — enter copy mode (`copy-mode`).
- **]** — paste the most recent buffer (`paste-buffer -p`).
- **#** — list all paste buffers (`list-buffers`).
- **=** — choose a paste buffer from a list (`choose-buffer -Z`).
- **-** — delete the most recent paste buffer (`delete-buffer`).
- **:** — open the command prompt (`command-prompt`).
- **?** — list key bindings (`list-keys -N`).
- **/** — describe a key binding (`command-prompt -kpkey { list-keys ... }`).
- **i** — display window information (`display-message`).
- **r** — redraw the current client (`refresh-client`).
- **C** — customize options interactively (`customize-mode -Z`).
- **t** — display the big clock (`clock-mode`).
- **~** — show the server message log (`show-messages`).
- **C-z** — suspend the current client (`suspend-client`).
- **C-b** — send the prefix key to the pane (`send-prefix`).

The `root` table (bindings that fire without the prefix) carries the mouse bindings and any user additions; the `copy-mode` and `copy-mode-vi` tables carry the navigation and selection keys of Chapter 18. Because all four are built by the same `key_bindings_add` calls as `tmux` and looked up the same way, a user's muscle memory transfers unchanged — which is exactly the compatibility the superset promise (Chapter 13) rests on.

Appendix J — Format Recipes

The best way to internalize the format mini-language (Chapters 9 and 16) is to read real compositions. This appendix works through a few, building from the operators the parity suite pins. Each can be tested directly with `display-message -p`.

A status-right with conditional prefix highlight

```
#{?client_prefix,#[reverse] PREFIX #[default],}%H:%M
```

`#{?client_prefix,THEN,ELSE}` tests whether the prefix key is currently pending. When it is, the `THEN` branch emits a reverse-video `PREFIX` badge (using the `#[reverse]` / `#[default]` style directives that `format_draw.rs` renders); when it is not, the `ELSE` branch is empty. Either way it is followed by `%H:%M`, a strftime time (evaluated only when time expansion is enabled — hence excluded from parity). The recipe shows the two most common building blocks together: a boolean-tested conditional and inline style directives.

A window-status entry with a zoom flag

```
#{window_index}:{window_name}#{?window_zoomed_flag,Z,}
```

Each window's status entry shows its index and name, and appends a `Z` only when a pane in that window is zoomed — again a `#{?...Z,}` conditional over a flag variable. Set this as `window-status-format` and every window in the middle of the status line gains a zoom indicator with no scripting.

Truncating a long title to fit

```
#{=/20/...:pane_title}
```

The `#{=/N/marker:string}` form limits `pane_title` to twenty columns and, if it had to truncate, appends the marker `....`. The plain `#{=20:pane_title}` truncates silently; the `/.../` variant tells the reader the title was cut. This is how status lines keep a fixed width regardless of how long a program sets its title.

Counting panes and branching on it

```
#{?#{==:#{window_panes},1},[solo],#{window_panes} panes}
```

Reading inside out: `#{window_panes}` is the pane count; `#{==:#{window_panes},1}` is a comparison that yields `1` when there is exactly one pane; and the outer `#{?...}` prints `[solo]` in that case, otherwise the count followed by `panes`. It composes three operators — a variable, a comparison, and a conditional — and every layer is re-expanded by the same engine, which is why arbitrary nesting works.

A per-window loop in a summary

```
#{W:#{window_index}#{?window_active,*, } }
```

The `#{W:fmt}` loop expands `fmt` once per window in the current session. Here `fmt` prints each window's index followed by `*` for the active window and a space otherwise, then a trailing space — producing a compact `0 1* 2` summary of the session's windows. It is the loop modifier of Chapter 16 in a single line. (Note that the capital-letter loop's injected `loop_index/loop_last_flag` are a current parity gap, Appendix D; the per-object variables used here — `window_index`, `window_active` — work.)

Arithmetic in a format

```
used #{e|/|:#{history_size},1000}k of #{e|/|:#{history_limit},1000}k lines
```

Two `#{e|/|:a,b}` divisions turn the raw scrollbar line counts into rounded thousands, so a pane with 5,300 lines of a 20,000-line limit reads `used 5k of 20k lines`. Arithmetic formats are how a status line reports computed quantities — memory, ratios, elapsed counts — without an external shell call, and because `#{e}` is deterministic it is heavily represented in the parity suite (cases 010–017, 200–215).

Appendix K — The Audit Workflow

The book’s central claim is that ztmux’s faithfulness is *checkable*, not merely asserted. This appendix is the how-to: the concrete steps a reviewer takes to verify a claim, using the machinery Chapters 2, 11, and 12 built. None of it requires trusting the author.

Verifying one function against the C

Pick any Rust function — say `grid_scroll_history` in `src/grid.rs`. Read its citation, the doc comment directly above it: `/// [\vendor/tmux/grid.c: " . Openvendor/tmux/grid.c at that line and you are looking at the exact C the Rust claims to implement. Compare the two by eye: the parameters (names and order should match the cited signature), the control flow (loops that increment on continue', early returns, the order of side effects), and the constants. Because both trees are committed in the repo, this is a local, offline operation — no fetching, no version guessing. If the Rust diverges from the C in a way the parity suite does not yet cover, you have found either a bug or a missing case.`

Checking the whole tree’s structural honesty

Run the anti-drift gate: `cargo test --test ported_fn_names_match_c`. It collects every Rust free function, indexes every C function in `vendor/tmux/`, and fails if any Rust name has no C counterpart and no allowlist entry — printing `FAKE-FN <name> (<file>:<line>)` for each violation (Appendix E). A green run means every function in the tree corresponds to something real in tmux or is a documented exception. Then run `python3 scripts/annotate_c_links.py --check`: it exits non-zero if any citation is missing or stale, so a green run means every ported function’s C citation is present and current.

Measuring behavioral fidelity

Run the parity suite: `bash parity/run_parity.sh`. It builds the vendored tmux reference and ztmux, runs all 1107 cases through both, and reports per-case OK/FAIL and totals. For any FAIL, `parity/parity_failures.log` holds both outputs and a unified diff, so the divergence is the exact bytes. To investigate a single case by hand, expand its format string in both binaries — `vendor/tmux/tmux display-message -p '<fmt>'` versus `ztmux display-message -p '<fmt>'` — and compare. The case is the reproduction (Chapter 11), so debugging starts from a minimal input.

Reading the port’s own contribution

To see what the port has actually done since the seed, run the divergence diff: `git diff --no-index vendor/tmux-is/src src`. Because the seed is kept pristine, this diff is the port’s original work — every re-reported function, every bug fixed against the C, every advance past where the seed left off. It can only under-state the work (untouched seed files count as unchanged), never overstate it, which is exactly the anti-inflation property a reviewer wants. Pair it with the generated `port_report.html` (from `scripts/gen_port_report.py`) for coverage — how much of tmux’s function surface has a ported counterpart — and you have both halves of the completeness picture: contribution (the diff) and coverage (the report).

The whole loop

Put together, the audit is: read a citation and diff one function against the C; run the anti-drift gate for structural honesty; run the citation `--check` for interface freshness; run the parity suite for behavioral fidelity; and run the divergence diff and port report for contribution and coverage. Every step consults a committed reference and produces an artifact, so the conclusion does not rest on the author's word at any point. That is what "faithfulness you can check" means operationally — a fixed sequence of commands, each with a pass/fail or a diff, that a skeptic can run start to finish.

Appendix L – Coexisting with tmux and Operational Notes

ztmux is designed to run alongside a real tmux without collision, and to be diagnosable when something goes wrong. This appendix collects the operational facts that do not belong to any one subsystem chapter.

The socket namespace

ztmux opens its own socket namespace — `ztmux-<uid>` rather than tmux's `tmux-<uid>` — built by `make_label` in `src/tmux.rs`. The default socket is `<dir>/ztmux-<uid>/default`, with the directory created `S_IRWXU` and subject to the same owner-and-permission safety checks tmux performs (the directory must be owned by the caller and carry no world-accessible bits). The `ztmux-` prefix is the one place the path deliberately differs from tmux, and it is what lets a running tmux and a running ztmux coexist on the same machine: two servers, two socket namespaces, two sets of sessions, neither aware of the other. You can attach to your tmux sessions and your ztmux sessions independently from the same shell. A custom socket is selected with `-L <label>` (a named socket in the same directory) or `-S <path>` (an explicit path), exactly as in tmux.

Diagnostics: the panic file and the output log

Two diagnostic aids are worth knowing. First, the server installs a Rust panic hook that, on any panic in the daemon, dumps a backtrace to `server-panic-<pid>.txt` — a small original touch that turns a Rust panic in a detached daemon (which has no terminal to print to) into a diagnosable artifact rather than a silent death. If a ztmux server ever vanishes unexpectedly, that file is where to look. Second, the tty layer can mirror the exact byte stream it sends to a client into an output log (`tty_create_log`), a faithful port of tmux's own debugging aid, so a developer can capture precisely what escape sequences the server emitted — invaluable when a terminal renders something wrong and the question is whether ztmux sent the wrong bytes or the terminal interpreted the right ones wrongly.

Signals the server honors

The server's `server_signal` handler wires the operational signals faithfully (Chapter 3): `SIGINT/SIGTERM` begin an orderly shutdown (kill jobs, save prompt history, exit); `SIGCHLD` reaps pane children; `SIGUSR1` recreates the socket, so a moved or deleted socket can be restored without losing the server; and `SIGUSR2` toggles logging at runtime. Knowing these lets an operator recover a server whose socket file was accidentally removed (`kill -USR1`) or turn on logging for a misbehaving server (`kill -USR2`) without restarting it.

The environment a pane sees

A pane's environment is assembled, not inherited raw (Chapter 19). The `update-environment` option names the variables refreshed from the attaching client into the session environment on each attach — `DISPLAY`, `SSH_AUTH_SOCK`, and the rest — so a program launched in a new window after you re-attach over a fresh

SSH connection sees the *new* connection's values, not the stale ones the daemon was started with. This is why agent forwarding and X display forwarding "just work" across re-attaches, and it is a faithful port of behavior tmux users rely on without thinking about.

Appendix M – A Reading Guide to the Source

For a reader who wants to move from this book into the code, this appendix suggests an order, because two hundred kilobytes of `format.rs` is not where to start. The tree rewards being read in roughly the book's order, with a few practical entry points.

Where to start

Begin with `src/lib.rs`, which holds the shared struct and constant definitions — `grid_cell`, `screen`, `layout_cell`, the flag sets — that every other module is built on. Reading the data model first makes the code that operates on it legible. Then read `src/tmux.rs` and `src/main.rs` for the entry points and the client-versus-server decision, and `src/server.rs` for the event loop that everything hangs off.

Following a subsystem

To understand one subsystem deeply, read its module next to its C origin: open `src/grid.rs` and `vendor/tmux/grid.c` side by side, and use the citations to jump between them. The one-to-one function correspondence means you can read the C for the algorithm and the Rust for the port, and the citation on each function tells you exactly which lines to compare. The `cmd_*.rs` files are the easiest to read in isolation — each is a single command, small and self-contained, mirroring its `cmd-*.c`.

Using the tooling as a map

The generated `port_report.html` is a map of what is ported and what is pending, per function. The divergence diff (`git diff --no-index vendor/tmux-rs/src src`) shows what the port changed from the seed, which is a good way to find the most actively-worked code. And the parity cases in `parity/cases/` are, collectively, a specification-by-example of the format language and the command surface — reading them is a fast way to learn what behavior is pinned and how the format operators actually compose. Between the citations, the report, the diff, and the cases, the source is unusually navigable for a program of its size — which is, after all, the whole point of a port that sells itself on being checkable.